

e-Stat API 入門

政府統計の総合窓口 API で学ぶ 公的統計データの取得と分析

河合 勝彦

名古屋市立大学大学院経済学研究科

kkawai@econ.nagoya-cu.ac.jp

2026 年 6 月 30 日

概要

本書は、政府統計の総合窓口「**e-Stat**」が提供する **API 機能 (バージョン 3.0)** を使い、Python から公的統計データを検索・取得・整形・可視化する手順を学ぶ入門チュートリアルである。第 1 部では、利用登録、最初のリクエスト、レスポンス構造、データの絞り込み、コードから名称への変換、ページング、エラー処理までを順を追って説明する。第 2 部では、人口・労働・物価・家計・GDP・産業・企業・教育などの分野を対象に、**各種データを入手するための 30 個の例題**を示す。第 3 部 (付録) では、例題作成に使えるコーディングエージェント向けプロンプト集と、Flet による GUI アプリ、FastAPI と HTMX による Web アプリの作例を扱う。すべての Python スクリプトは **uv** と **PEP 723** (インラインスクリプトメタデータ) 形式で記述されており、追加の依存管理を行わずに **uv run** の 1 コマンドで実行できる。

出典: 本書で取得・表示する統計データは「政府統計の総合窓口 (e-Stat)」(<https://www.e-stat.go.jp/>) によるものである。

目次

第 I 部	e-Stat API の基礎	5
1	はじめに	5
1.1	e-Stat とは	5
1.2	API 機能で何ができるか	5
1.3	なぜ Web 画面ではなく API を使うのか	5
1.4	REST API の仕組みをざっくり理解する	6
1.5	本書の構成と対象読者	7
1.6	準備するもの	7
2	準備①：アプリケーション ID (appId) の取得	7
3	準備②：実行環境 (uv と PEP 723)	8
3.1	uv のインストール	8
3.2	PEP 723：スクリプト 1 枚に依存を書く	8
3.3	uv run の仕組み	9
3.4	appId の安全な扱い	9
4	練習問題と解答：準備編	10
5	e-Stat API の全体像	11
5.1	ベース URL とバージョン	11
5.2	7つの機能	11
6	最初のリクエスト：統計表を検索する	12
7	レスポンスの構造を理解する	12
7.1	メタとデータの対応	13
8	統計表を探す：statsCode と検索条件	13
9	メタ情報を取得する：getMetaInfo	15
10	統計データを取得する：getStatsData と絞り込み	15
11	大きなデータを分割取得する：ページネーション	16
12	コードを名称に変換し pandas で扱う	16
13	欠損・秘匿・注釈を扱う	17
14	出力形式の選択：JSON・CSV・gzip	17
15	エラー処理と STATUS コード	18
16	グラフの日本語表示：matplotlib の準備	18
17	利用上の注意とマナー	19
18	練習問題と解答：API 基礎編	19
第 II 部	各種データを入手する例題 30 選	20
19	まず動かす：最小のスクリプト	21

19.1	最小例 1：統計表を検索する	21
19.2	最小例 2：数値を 1 つ取り出す	22
19.3	最小例 3：CSV を pandas で読む	22
19.4	最小例 4：グラフにする	23
20	共通モジュール	25
21	グループ A：API 操作の基礎	28
21.1	例題 1：キーワードで統計表を検索する	28
21.2	例題 2：政府統計コードで絞り、CSV に保存する	30
21.3	例題 3：分野と複合キーワードで横断検索する	32
21.4	例題 4：メタ情報（分類の定義）を取得する	34
21.5	例題 5：取得件数を先に確認する（cntGetFlg）	36
21.6	例題 6：生の JSON 構造を観察する	38
22	グループ B：getStatsData の絞り込み技法	39
22.1	例題 7：時間軸で絞る（cdTimeFrom・lvTime）	39
22.2	例題 8：地域で絞る（lvArea・cdArea）	41
22.3	例題 9：分類事項で絞る（cdCat02）	42
22.4	例題 10：階層レベルで絞る（lvCat01）	43
22.5	例題 11：ページネーション（NEXT_KEY）	44
22.6	例題 12：データとメタを別々に取得して結合する	46
22.7	例題 13：CSV と JSON を比べる	47
23	グループ C：実データの取得と分析・可視化	49
23.1	例題 14：人口ピラミッド	49
23.2	例題 15：都道府県別人口ランキング	51
23.3	例題 16：出生数と合計特殊出生率	53
23.4	例題 17：3 大都市圏の転入超過	55
23.5	例題 18：完全失業率の月次推移	57
23.6	例題 19：現金給与総額の推移	59
23.7	例題 20：年齢階級別の賃金プロファイル	61
23.8	例題 21：消費者物価指数の前年同月比（インフレ率）	63
23.9	例題 22：費目別消費支出の構成	65
23.10	例題 23：実質 GDP 成長率（年率換算）	67
23.11	例題 24：鉱工業生産指数（IIP）	69
23.12	例題 25：業種別商業販売額指数	71
23.13	例題 26：経常利益と設備投資	73
23.14	例題 27：進学率の長期推移	75
24	グループ D：応用	77
24.1	例題 28：データカタログで公開ファイルを探す（getDataCatalog）	77

24.2	例題 29：複数表を一括取得する (getStatsDatas)	79
24.3	例題 30：再利用クライアントと英語メタ (lang=E)	80
25	練習問題と解答：例題・応用編	81
第 III 部	付録	83
A	コーディングエージェント用プロンプト集	83
A.1	共通の前提プロンプト	83
A.2	グループ A：API 操作の基礎のプロンプト	84
A.3	グループ B：getStatsData の絞り込みのプロンプト	85
A.4	グループ C：取得と可視化のプロンプト	87
A.5	グループ D：応用のプロンプト	89
B	Flet で作る GUI アプリ	90
B.1	アプリ 1：統計表をキーワード検索する	91
B.2	アプリ 2：メタ情報（分類の定義）を表示する	94
B.3	アプリ 3：統計データをランキング表示する	97
C	FastAPI で作る Web アプリ	101
C.1	アプリ 1：統計表をキーワード検索する	102
C.2	アプリ 2：メタ情報（分類の定義）をブラウズする	105
C.3	アプリ 3：統計データのダッシュボード	108

第 I 部 e-Stat API の基礎

1 はじめに

1.1 e-Stat とは

e-Stat（政府統計の総合窓口）は、日本の各府省が公表する統計データを一つの窓口に集約した政府統計のポータルサイトである（総務省統計局が整備、独立行政法人統計センターが運用）。国勢調査・労働力調査・消費者物価指数・家計調査・国民経済計算など、国の基幹統計をはじめとする膨大な統計表が、Web 上で横断的に検索・閲覧・ダウンロードできる。

<https://www.e-stat.go.jp/> をブラウザで開けば、人手で表を探して Excel や CSV をダウンロードできる。しかし、毎月更新される指標を継続的に取り込んだり、多数の統計表をまとめて処理したりするには、手作業では限界がある。そこで用意されているのが **e-Stat API 機能** である。

1.2 API 機能で何ができるか

API は **Application Programming Interface** の略で、ここでは「プログラムから e-Stat に問い合わせるための入口」と考えればよい。通常の Web 画面は人間が読むために HTML として表示されるが、API は Python などのプログラムが読み取りやすい形式で結果を返す。

e-Stat API は、統計データを機械可読な形式（JSON・XML・CSV）でプログラムから直接取得できる Web API である。これにより、

- ・キーワードや統計分野で**統計表を検索**する（どんな表があるか調べる）
- ・統計表の**メタ情報**（分類項目・地域・時間軸の一覧）を取得する
- ・条件で絞り込んで**統計数値そのもの**を取得する
- ・取得したデータを pandas で整形し、CSV 保存やグラフ化など**自動処理**する

といったことが、再現可能なスクリプトとして実現できる。最新値の自動更新、複数地域・複数期間の一括取得、独自の指標計算など、研究・業務のデータ基盤として活用できる。

1.3 なぜ Web 画面ではなく API を使うのか

はじめて API に触れると、URL、パラメータ、JSON などの言葉が並び、必要以上に難しく見えるかもしれない。しかし、やっていることは「**条件を指定して統計表を取り寄せる**」だけである。ブラウザで e-Stat を操作するときは、検索窓にキーワードを入れ、画面上の分類や期間を選び、CSV や Excel をダウンロードする。API では、その同じ作業をスクリプトに書いて再実行できるようにする。

手作業によるダウンロードは、1 回だけなら十分便利である。一方で、次のような場面では API の利点大きい。

- 毎月公表される指標を、同じ条件で継続的に更新したい。
- 47 都道府県や長い時系列をまとめて取得し、同じ形に整えたい。
- 取得条件や加工手順をコードとして残し、後から再現できるようにしたい。
- CSV 保存、pandas での集計、グラフ作成までを一連の流れにしたい。

本書では、最初から巨大なプログラムを書くことはしない。まず「統計表を探す」「分類を調べる」「数値を取る」という小さな作業に分け、それぞれを短い Python スクリプトで確認していく。

1.4 REST API の仕組みをざっくり理解する

e-Stat API は **REST 方式** の Web API である。REST という言葉自体は抽象的だが、初心者のうちは次のように理解すれば十分である。

決まった URL に、条件を添えてリクエストを送り、サーバからレスポンスを受け取る。ブラウザで Web ページを開くときも、実は URL に対してリクエストを送り、サーバからページの内容を受け取っている。API の場合も基本は同じである。違いは、返ってくるものが人間向けの画面ではなく、プログラムで扱いやすい JSON や CSV だという点にある。

たとえば、統計表を検索するリクエストは次のような URL になる。

```
https://api.e-stat.go.jp/rest/3.0/app/json/getStatsList?appId=【appId】&searchWord=人口&limit=3
```

一見長いが、部品に分けると読みやすい。

表 1 REST API の URL を構成する主な部品

部品	意味
https://api.e-stat.go.jp	e-Stat API のサーバ
/rest/3.0	REST API のバージョン 3.0 を使う指定
/app/json	アプリケーション ID を使い、JSON 形式で受け取る指定
/getStatsList	統計表情報を検索する機能（エンドポイント）
?appId=...	誰の利用かを示す認証用の値
&searchWord=人口&limit=3	検索語と取得件数という条件（クエリパラメータ）

REST API では、このような機能ごとの URL を **エンドポイント** と呼ぶ。e-Stat API の基本操作では、Python から `requests.get(...)` を使って GET リクエストを送り、返ってきた JSON を辞書やリストとして読む。後半で扱う一括取得のように、条件が複雑な場合は POST リクエストを使うこともあるが、本書の入口では「URL に条件を付けて取得する」と考えてよい。

URL エンコード

ブラウザに URL を直接貼り付ける場合、日本語や空白などは UTF-8 で URL エンコードする必要がある。Python の `requests` では、`params={"searchWord": "人口"}` のように辞書で渡せば

自動でエンコードされる。文字列連結で URL を組み立てるより、`params` を使う方が安全である。

2つの STATUS

API を使うときは、HTTP 通信そのものの成否と、e-Stat 側の処理結果を分けて見る。`requests` の `res.raise_for_status()` は通信の失敗を確認し、レスポンス中の `RESULT.STATUS` は e-Stat API の処理結果を確認する。本書のスクリプトでは、この2段階をどちらも確認する。

1.5 本書の構成と対象読者

本書は、基礎、例題、付録の三部で構成される。第1部（本パート）では、API の利用登録から REST API の基本、最初のリクエスト、レスポンスの読み方、データの絞り込み、コードから名称への変換、エラー処理までを、順を追ってやさしく解説する。第2部では、実際に各種データを入手するための30個の例題を、分野横断で示す。第3部（付録）では、コーディングエージェントに例題生成を依頼するためのプロンプト集と、同じ API 呼び出しを画面アプリに発展させる Flet GUI、FastAPI + HTMX の Web アプリを扱う。

対象読者は「Python の基本文法（変数・関数・リスト・辞書）は分かるが、Web API や e-Stat は初めて」という方を想定している。難しい環境構築は要求しない。後述する `uv` さえ入れれば、各スクリプトは1行のコマンドで動く。

1.6 準備するもの

1. e-Stat のアプリケーション ID (`appId`) : API 利用に必須。無料。取得方法は第2節。
2. `uv` : Python スクリプトを依存ごと自動実行するツール。導入は第3節。
3. インターネット接続とテキストエディタ。

2 準備①：アプリケーション ID (`appId`) の取得

e-Stat API を呼び出すには、リクエストごとにアプリケーション ID (`appId`) を付ける必要がある。これは利用者ごとに発行される無料の鍵で、次の手順で取得する。

1. <https://www.e-stat.go.jp/api/> にアクセスし、画面の案内に従って利用登録（メールアドレスの登録）を行う。確認メールから本登録を完了する。
2. e-Stat にログインし、マイページの「API 機能（アプリケーション ID の取得）」を開く。
3. アプリケーションの名称・URL・概要を入力して登録する。登録すると `appId`（英数字の文字列）が発行される。
4. 発行された `appId` を控える。これをスクリプトから利用する。

ヒント

公開予定のないアプリやローカルでの学習用途でも登録できる。URL 欄には `http://test.localhost/` のようなローカルアドレスを入力してよい。名称・URL・概要は後から変更できる。

appId は秘密情報として扱う

appId をソースコードに直接書き込んで公開リポジトリにコミットすると、第三者に悪用されかねない。本書では appId を環境変数 `ESTAT_APPID` から読み込む方式を採り、ファイル `.env` に保存して Git 管理から除外する (`.gitignore` に `.env` を追加)。具体的な扱いは次節で示す。

3 準備②：実行環境 (uv と PEP 723)

3.1 uv のインストール

uv は、Python のパッケージ管理とスクリプト実行を高速に行うツールである。本書のスクリプトは uv の機能を使い、依存パッケージの手動インストールを不要にしている。未導入なら、公式の案内 (<https://docs.astral.sh/uv/>) に従って導入する。代表的な方法は次のとおり。

```
# macOS / Linux (公式インストーラ)
curl -LsSf https://astral.sh/uv/install.sh | sh

# Homebrew を使う場合
brew install uv

# Windows (PowerShell)
powershell -ExecutionPolicy Bypass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

導入後、`uv --version` でバージョンが表示されれば成功である。

3.2 PEP 723：スクリプト 1 枚に依存を書く

従来、外部ライブラリを使う Python スクリプトを配るには「requirements.txt を別途用意し、仮想環境を作って pip install する」といった手順が必要だった。PEP 723 (インラインスクリプトメタデータ) は、この依存情報をスクリプトファイルの先頭コメントに直接書けるようにする仕組みである。

```
# /// script
# requires-python = ">=3.11"
# dependencies = [
#     "requests",
#     "pandas",
# ]
# ///
```

```
import requests # 上で宣言したので利用できる
```

ルールは単純である。# /// script で始まり # /// で終わるコメントブロックを、ファイル先頭（シバン行があればその直後）に置く。中身は TOML 形式で、requires-python（必要な Python のバージョン）と dependencies（必要なパッケージの一覧）を書く。

3.3 uv run の仕組み

このメタデータ付きスクリプトは、次の 1 行で実行できる。

```
uv run script.py
```

uv run は実行時に先頭のメタデータを読み取り、一時的な仮想環境を自動生成して依存パッケージをインストールし、その環境でスクリプトを走らせる。ユーザーが事前に pip install する必要はない。2 回目以降はキャッシュが効くため高速である。

スクリプトへ後から依存を追加するには、次のコマンドが使える（メタデータブロックが自動で書き換わる）。

```
uv add --script script.py matplotlib
```

3.4 appId の安全な扱い

第 2 節の警告のとおり、appId は環境変数から読む。リポジトリにはひな形 .env.example を置いてあるので、まずコピーして .env を作る。

```
cp .env.example .env
```

リポジトリ直下の .env に、次のように書く（このファイルはコミットしない）。

```
# .env
ESTAT_APPID=ここに発行されたappIdを書く
```

スクリプト実行前に、シェルでこのファイルを環境変数として読み込む。

```
set -a; source .env; set +a # .env の内容を環境変数に展開
uv run scripts/ex01_search_statslist.py
```

Python 側では os.environ["ESTAT_APPID"] で受け取る。本書のスクリプトは、未設定のときに分かりやすいメッセージで終了するようにしている。

ヒント

set -a は「以後に定義した変数を自動的に export する」設定、set +a はその解除である。間に source .env を挟むことで、.env 内の KEY=VALUE がすべて環境変数になる。Windows の PowerShell では \$env:ESTAT_APPID="..." のように設定する。

4 練習問題と解答：準備編

問 1.問題 uv が利用できるか確認せよ。

解答 バージョン確認コマンドを実行し、バージョンが表示されればよい。

実行結果（抜粋）

```
$ uv --version
uv 0.11.7 (x86_64-unknown-linux-gnu)
```

問 2.問題 .env を読み込み、ESTAT_APPID が環境変数として設定されたか確認せよ。

解答 appId そのものは表示せず、設定済みかどうかだけを確認する。

実行結果（抜粋）

```
$ set -a; source .env; set +a
$ python3 -c "import os; print(\"OK\" if os.environ.get(\"ESTAT_APPID\") else \"NG\")"
OK
```

問 3.問題 scripts/min3_csv_to_pandas.py の PEP 723 メタデータを確認し、必要な依存を答えよ。

解答 先頭の dependencies に、このスクリプトで使う外部パッケージが列挙されている。

実行結果（抜粋）

```
$ head -n 4 scripts/min3_csv_to_pandas.py
# /// script
# requires-python = ">=3.11"
# dependencies = ["requests", "pandas"]
# ///
```

問 4.問題 最小の検索スクリプト scripts/min1_hello_search.py を実行し、API 検索が成功することを確認せよ。

解答 getStatsList が「人口」を含む統計表を返せば成功である。

実行結果（抜粋）

```
$ uv run scripts/min1_hello_search.py
25016 件ヒットしました。先頭3件：
- 人口及び人口増加（29），男女別（3）人口数－総人口，日本人人口，外国人人口，全国
- 人口及び人口増加（29），男女別（3）人口数－総人口，日本人人口，外国人人口，全国
```

問 5.問題 最小の作図スクリプト scripts/min4_quick_plot.py を実行し、図が保存されることを確認せよ。

解答 スクリプト実行後、figures/min4_cpi.pdf が作成される。

実行結果（抜粋）

```
$ uv run scripts/min4_quick_plot.py
グラフを figures/min4_cpi.pdf に保存しました。
$ ls figures/min4_cpi.pdf
figures/min4_cpi.pdf
```

これで準備は整った。次節から、API の全体像と最初のリクエストに進む。

5 e-Stat API の全体像

e-Stat API は **REST 方式** の Web API である。HTTP で特定の URL（エンドポイント）にアクセスし、パラメータを与えると、統計情報が JSON などの形式で返ってくる。すべてのリクエストには共通パラメータとして `appId`（必須）と `lang`（言語、J=日本語／E=英語、既定は J）を与えられる。公式仕様では、バージョン 3.0 から CSV 形式、集計地域区分（`collectArea`）、解説情報（`explanationGetFlg`）、注釈情報（`annotationGetFlg`）などが整備されている。本書はこの 3.0 仕様に合わせて説明する。

5.1 ベース URL とバージョン

本書では現行のバージョン 3.0 を用いる。エンドポイントの基本形は次のとおり。

```
https://api.e-stat.go.jp/rest/3.0/app/<機能名>      ← XML（既定）
https://api.e-stat.go.jp/rest/3.0/app/json/<機能名> ← JSON
https://api.e-stat.go.jp/rest/3.0/app/jsonp/<機能名> ← JSONP
https://api.e-stat.go.jp/rest/3.0/app/getSimpleStatsData ← CSV（簡易版の例）
```

本書では扱いやすい JSON（`json/` を挟む形）を主に用いる。CSV が必要なときは、統計表情報・メタ情報・統計データの簡易版（`getSimpleStatsList`・`getSimpleMetaInfo`・`getSimpleStatsData`）を使う。

5.2 7つの機能

API には次の 7 つの機能がある（本書では主に上の 3 つを使う）。CSV 簡易版は出力形式の違いであり、独立した機能としては数えない。

表 2 e-Stat API 3.0 の機能一覧

機能名	和名	用途
<code>getStatsList</code>	統計表情報取得	どんな統計表があるか検索する
<code>getMetaInfo</code>	メタ情報取得	表の分類・地域・時間軸の定義を得る
<code>getStatsData</code>	統計データ取得	数値データ本体を取得する
<code>postDataset</code>	データセット登録	絞り込み条件を保存する
<code>refDataset</code>	データセット参照	保存済みの取得条件を参照する
<code>getDataCatalog</code>	データカタログ情報取得	公開ファイル（XLS/CSV 等）を検索する
<code>getStatsDatas</code>	統計データ一括取得	複数表をまとめて取得する

典型的な利用の流れは、(1) `getStatsList` で表を探し、(2) `getMetaInfo` で中身の分類を調べ、(3) `getStatsData` で必要な数値を絞り込んで取得する、の 3 ステップである。なお、`getStatsData` は便利だが、返却セル数が多い用途には向かない。一括取得では全リクエスト合計の返却セル数に制限があるため、大きな表を扱う場合は `getStatsData` を個別に呼び、`NEXT_KEY` でページングする方が制御しやすい。

6 最初のリクエスト：統計表を検索する

まず `getStatsList` で「人口」を含む統計表を検索してみよう。リクエストは URL に `?キー=値&キー=値` の形でパラメータを並べるだけである。

```
https://api.e-stat.go.jp/rest/3.0/app/json/getStatsList?appId=【appId】&searchWord=人口&limit=3
```

Python では `requests` ライブラリを使うと、パラメータを辞書で渡せて簡単である。

```
import os, requests

url = "https://api.e-stat.go.jp/rest/3.0/app/json/getStatsList"
params = {"appId": os.environ["ESTAT_APPID"], "searchWord": "人口", "limit": 3}
res = requests.get(url, params=params, timeout=60)
res.raise_for_status()
body = res.json()["GET_STATS_LIST"]
if body["RESULT"]["STATUS"] != 0:
    raise RuntimeError(body["RESULT"]["ERROR_MSG"])
for table in body["DATALIST_INF"]["TABLE_INF"]:
    title = table["TITLE"]
    title = title["$"] if isinstance(title, dict) else title
    print(table["@id"], title)
```

`getStatsList` の応答は `GET_STATS_LIST` を頂点とし、検索でヒットした各表が `DATALIST_INF` → `TABLE_INF`（配列）に入る。各表には統計表 ID (`@id`) が付いており、これが次のステップで使う `statsDataId` である。詳しくは第 2 部の例題 1～3 で実際に確認する。

7 レスポンスの構造を理解する

API 理解の山場は、`getStatsData`（統計データ取得）の応答構造である。JSON は入れ子（ネスト）になっており、大きくメタ情報（分類の定義）とデータ（数値）に分かれる。全体像は次のとおり。

```
GET_STATS_DATA
├─ RESULT          … 処理結果   STATUS (0=データありで正常), ERROR_MSG, DATE
├─ PARAMETER       … 送ったパラメータの控え
└─ STATISTICAL_DATA
   ├─ RESULT_INF   … 件数情報   TOTAL_NUMBER, FROM_NUMBER, TO_NUMBER, NEXT_KEY
   ├─ TABLE_INF   … 表の基本情報 (表題・調査名・更新日など)
   ├─ CLASS_INF    … ★メタ情報: 分類の定義
   └─ CLASS_OBJ[]  各次元 … @id, @name
```

└─ CLASS[]	選択肢 … @code, @name, @level, @unit
└─ DATA_INF	… ★データ：数値の集合
└─ VALUE[]	1セル … @tab,@cat01…,@area,@time,@unit, "\$"(値)

7.1 メタとデータの対応

ここが最重要である。DATA_INF の各 VALUE は、@cat01・@area・@time といった属性にコードを持ち、実際の数値は \$（ドル記号のキー）に入る。そのコードが何を表すかは、CLASS_INF の対応する CLASS_OBJ（同じ @id を持つ）の CLASS を引けば分かる。

表3 VALUE の属性とメタ情報（CLASS_OBJ）の対応

VALUE の属性	対応する CLASS_OBJ の @id	意味の例
@tab	tab	表章事項（人数・金額など）
@cat01	cat01	分類事項 1（男女・産業など）
@cat02…@cat15	cat02…cat15	分類事項 2～15
@area	area	地域（全国・都道府県・市区町村）
@time	time	時間軸（年・四半期・月）
@unit	—	単位（人・円・% など）
@annotation	ANNOTATION	注釈コード（ある場合のみ）
\$	—	数値そのもの

たとえば、ある VALUE が {"@cat01":"001","@time":"2020000000","\$":"1234"} であり、CLASS_OBJ (@id="cat01") の中に {"@code":"001","@name":"男"} があれば、そのセルは「2020 年・男・1234（単位はメタの通り）」と読める。このコード→名称の引き当てを自動化する方法は第 12 節で示す。

要素数 1 のときは配列にならない

e-Stat の XML から JSON への変換仕様上、CLASS_OBJ や VALUE が 1 個しかないときは配列 [] ではなく単一のオブジェクト {} として返る。プログラムでは「リストでなければリストに包む」処理を入れて両対応にすると安全である（例題 6・12 で実装する）。

8 統計表を探す：statsCode と検索条件

目的の数値を取るには、まずその表の statsDataId を知る必要がある。getStatsList の主な絞り込み条件は次のとおり。

政府統計コード（statsCode） は統計調査ごとに割り当てられた 8 桁の番号で、これを指定するとその調査の統計表だけに絞り込む。代表的な統計のコードを挙げる（本書の例題で使用する。コードは e-Stat 上または getStatsList の応答中の STAT_NAME の @code で確認できる）。

表 4 getStatsList の主な検索パラメータ

パラメータ	説明
searchWord	検索キーワード。空白や AND/OR/NOT で論理結合できる
statsCode	政府統計コード。5 桁（作成機関）または 8 桁（統計調査）で指定
statsField	統計分野コード。2 桁（大分類）または 4 桁（小分類）
surveyYears, openYears	調査年月／公開年月。yyyy・yyyymm・yyyymm-yyyyyy
searchKind	1: 統計情報（既定）／ 2: 小地域・地域メッシュ
collectArea	集計地域区分 1: 全国／ 2: 都道府県／ 3: 市区町村
updatedAt	更新日付。yyyy・yyyymm・yyyymmdd・範囲指定
explanationGetFlg	解説情報を返すか（Y=既定／ N）
limit, startPosition	取得件数と開始位置

表 5 本書で扱う主な統計と政府統計コードの例

統計調査名	作成機関	statsCode
国勢調査	総務省	00200521
人口推計	総務省	00200524
住民基本台帳人口移動報告	総務省	00200523
労働力調査	総務省	00200531
家計調査	総務省	00200561
消費者物価指数	総務省	00200573
人口動態調査	厚生労働省	00450011
毎月勤労統計調査	厚生労働省	00450071
賃金構造基本統計調査	厚生労働省	00450091
国民経済計算	内閣府	00100409
学校基本調査	文部科学省	00400001

9 メタ情報を取得する：getMetaInfo

statsDataId が分かったら、getMetaInfo でその表の**分類の定義**を取得できる。これは「この表にはどんな項目（男女別・年齢別・地域別・期間など）があり、それぞれどんなコードを指定できるか」の一覧である。返る構造は第 7 節の CLASS_INF と同じく、CLASS_OBJ（次元）→ CLASS（選択肢）の二段である。

各 CLASS には @level（階層の深さ）が付くことがある。たとえば地域なら「全国＝level1、都道府県＝level2、市区町村＝level3」のように階層化されており、後述の lvArea で「都道府県だけ」を選べる。また、分類によっては @unit（単位）、@parentCode（親コード）、@addInf（追加情報）を持つ。階層構造をたどる、または詳細な分類表を作る場合は、名称だけでなくこれらの属性も確認するとよい。

10 統計データを取得する：getStatsData と絞り込み

getStatsData は、statsDataId と絞り込み条件を与えて数値を取得する。表全体は巨大なことが多いので、**必要な部分だけに絞る**のが基本である。絞り込みは「コードで選ぶ」「範囲で選ぶ」「階層レベルで選ぶ」の 3 通りがある。

表 6 getStatsData の主な絞り込み・制御パラメータ

パラメータ	説明
statsDataId	取得する統計表 ID（必須）
cdTab, cdCat01 ... cdCat15, cdArea, cdTime	単一コードで絞り込む（カンマ区切りで複数可、最大 100）
cd...From / cd...To	コードの範囲で絞り込む（例 cdTimeFrom, cdTimeTo）
lvTab, lvCat01..., lvArea, lvTime	階層レベルで絞り込む（2、1-2、-3、2- の形式）
metaGetFlg	メタ情報を併せて返すか（Y=既定 / N）
cntGetFlg	件数のみ取得（Y）。事前に規模を知るのに便利
annotationGetFlg	注釈情報を返すか（Y=既定 / N）
explanationGetFlg	統計表や分類の解説情報を返すか（Y=既定 / N）
replaceSpChar	数値以外の特殊記号の置換（0: なし～3: "NA"）
startPosition, limit	取得開始位置と件数（次節）

getStatsData は statsDataId の代わりに、postDataset で登録した dataSetId を指定することもできる。ただし、statsDataId と dataSetId を同時に指定するとエラーになる。

ヒント

コード値には特殊キーワード min / max が使える。たとえば cdTimeFrom=2010000000&cdTimeTo=max とすれば「2010 年以降の最新まで」を表せる。単一コード指定（cdTime=max など）で min / max を使う場合は、カンマ区切りの複数コード

指定と併用できない。

11 大きなデータを分割取得する：ページネーション

1 回のリクエストで返るデータには上限（limit 省略時は 10 万件）がある。これを超える場合、応答の RESULT_INF に NEXT_KEY（次の開始行番号）が入る。これを startPosition に渡して繰り返し取得する。startPosition は 1 から始まる行番号であり、NEXT_KEY はページ番号ではなく、次回取得を始める行番号である点に注意する。

```
def as_list(x):
    return x if isinstance(x, list) else ([x] if x is None else [x])

records, start = [], 1
while True:
    params = {"appId": app_id, "statsDataId": sid, "startPosition": start}
    res = requests.get(url, params=params, timeout=60)
    res.raise_for_status()
    body = res.json()["GET_STATS_DATA"]
    if body["RESULT"]["STATUS"] != 0:
        raise RuntimeError(body["RESULT"]["ERROR_MSG"])
    sdata = body["STATISTICAL_DATA"]
    records.extend(as_list(sdata["DATA_INF"].get("VALUE")))
    next_key = sdata["RESULT_INF"].get("NEXT_KEY")
    if not next_key:
        # NEXT_KEY が無ければ全件取得完了
        break
    start = next_key
```

TOTAL_NUMBER（総件数）と FROM_NUMBER・TO_NUMBER（今回返した範囲）も併せて確認できる。例題 11 で実装する。

12 コードを名称に変換し pandas で扱う

取得した VALUE は、そのままでは @cat01 などがコードのままで読みにくい。CLASS_INF の対応表を使って名称に変換し、分析しやすいタテ持ち（tidy）の DataFrame にするのが定石である。本書の例題で繰り返し使う変換関数の骨子を示す（完全版は例題 12）。

```
import pandas as pd

def to_dataframe(sdata: dict) -> pd.DataFrame:
    # 1) メタ情報を {次元id: {コード: 名称}} の辞書に
    objs = sdata["CLASS_INF"]["CLASS_OBJ"]
    objs = objs if isinstance(objs, list) else [objs]
    names, code2name = {}, {}
    for o in objs:
        names[o["@id"]] = o["@name"]
        cl = o["CLASS"]; cl = cl if isinstance(cl, list) else [cl]
        code2name[o["@id"]] = {c["@code"]: c["@name"] for c in cl}
```

```
# 2) データを DataFrame 化し、@xxx 列を名称列に変換
vals = sdata["DATA_INF"]["VALUE"]
vals = vals if isinstance(vals, list) else [vals]
df = pd.DataFrame(vals)
for col in [c for c in df.columns if c.startswith("@")]:
    key = col[1:]
    if key in code2name:
        df[names[key]] = df[col].map(code2name[key])
df = df.rename(columns={"$": "値", "@unit": "単位"})
df["値"] = pd.to_numeric(df["値"], errors="coerce")
return df
```

13 欠損・秘匿・注釈を扱う

統計表によっては、数値セルに通常の数値ではなく特殊文字が入る。たとえば「-（該当数値なし）」「...（表章単位に満たない等）」「X（秘匿）」のような記号である。これらの意味は、応答中の NOTE に凡例として含まれる。

特殊文字は勝手に 0 にしない

- や X は、0 とは意味が異なる。分析用に `pd.to_numeric(..., errors="coerce")` で欠損値にするのはよいが、理由なく 0 へ置換すると集計値や平均値をゆがめる。置換が必要な場合だけ `replaceSpChar` を指定し、置換後の意味を記録しておく。

`replaceSpChar` は、特殊文字をそのまま返すか、0・空文字・NA に置換するかを選ぶパラメータである。既定は 0（置換しない）である。本書のスクリプトでは、原則として API の値をそのまま受け取り、pandas 側で数値化できないものを欠損として扱う。

また、バージョン 3.0 では統計数値に注釈が付くことがある。`annotationGetFlg=Y`（既定）の場合、注釈の定義は ANNOTATION に入り、各 VALUE には `@annotation` として注釈コードが付く。公表資料に近い表を作るときは、数値だけでなく注釈コードも残すと解釈ミス避けられる。

14 出力形式の選択：JSON・CSV・gzip

JSON (`json/`) はメタ情報と数値を構造的に含み、プログラム処理に向く。一方 CSV (`getSimpleStatsData`) は表形式で直接 `pandas.read_csv` に渡せて手軽である (`sectionHeaderFlg=2` でヘッダ説明行を省ける)。ただし、コードと名称の対応や階層情報を丁寧に扱う処理では JSON の方が確実である。用途に応じて使い分ける（比較は例題 13）。CSV 形式では `metaGetFlg` や `cntGetFlg` は無効であり、メタ情報を別途 `getMetaInfo` から取得して結合するような処理には JSON の方が向いている。

通信量を抑えたい場合は、リクエストヘッダに `Accept-Encoding: gzip` を付けると gzip 圧縮された応答が得られる (`requests` は自動で展開する)。文字コードはいずれも UTF-8 である。

15 エラー処理と STATUS コード

応答の `RESULT.STATUS` は、e-Stat 側の処理結果を表す。0 はデータありで正常終了、1 は該当データなし、2 は一括取得で一部リクエストに該当データなしを意味する。100 以上はエラーである。原因は `RESULT.ERROR_MSG` に文章で入るので、必ず確認する。代表的な失敗原因は次のとおり。

- 認証エラー：appId が未指定・誤り・無効。
- パラメータエラー：必須項目（statsDataId 等）の欠落、値の誤り。
- 該当データ無し：絞り込み条件に合うデータが存在しない。

実務では、ネットワーク不調やタイムアウトにも備える。本書のスクリプトでは、`requests` に `timeout` を設定し、期待したデータが得られない場合はメッセージを表示して終了する、という最小限の処理を入れている。本書の多くの例題は「データが存在する条件」を前提にしているため、簡潔さを優先して `STATUS` が 0 でない場合は処理を止める実装にしている。検索ツールや公開アプリを作る場合は、`STATUS=1` を例外ではなく「検索結果なし」として画面に表示するとよい。

節度ある利用を

短時間に大量のリクエストを送るとサーバに負荷をかける。必要な範囲に絞り込み、取得結果はローカルに保存して再利用し、ループでの連続取得には適度な間隔を空けること。

16 グラフの日本語表示：matplotlib の準備

matplotlib は既定では日本語フォントを持たず、ラベルが豆腐（□）になる。本書では `scripts/jpfont.py` に、日本語フォントを自動選択する小さな補助モジュールを置く。例題 14 以降などの作図スクリプトでは `import jpfont` するだけで、環境にある Noto Sans CJK JP、IPAexGothic、Yu Gothic などの候補から利用可能なものを設定する。

```
# dependencies = ["matplotlib"]
import matplotlib
matplotlib.use("Agg") # 画面を使わずファイルに保存する
import matplotlib.pyplot as plt
import jpfont # noqa: F401 import 時に日本語フォントを設定する
```

フォントが見つからない環境では図は作成できるが、日本語が欠けることがある。その場合は OS 側に日本語フォントを追加するか、`jpfont.py` の候補リストを調整する。

17 利用上の注意とマナー

- 出典の明示：取得データを用いた成果物には、出典「政府統計の総合窓口 (e-Stat)」を明記する。
- 利用規約：API の[利用規約](#)に従う。
- 再取得の抑制：同じデータを繰り返し取らず、保存して使い回す。
- 分割取得：大量データは limit / ページングで分割し、間隔を空ける。

18 練習問題と解答：API 基礎編

問 1.問題 getStatsList で「人口」を検索し、統計表 ID と表題が得られることを確認せよ。

解答 例題 1 を実行する。統計表 ID (@id) は、後続の statsDataId として使う。

実行結果 (抜粋)

```
$ uv run scripts/ex01_search_statslist.py
ヒット総数: 25016 件 (先頭 10 件を表示)

[ID:0000150041] 人口推計
  表題: 人口及び人口増加 (29), 男女別 (3) 人口数 - …全国 (調査:199310)
```

問 2.問題 消費者物価指数の表について、分類軸の数と代表的な軸名を確認せよ。

解答 例題 4 を実行し、getMetaInfo の CLASS_OBJ を読む。

実行結果 (抜粋)

```
$ uv run scripts/ex04_getmetainfo.py
対象の統計表: [ID:0002050001] 消費者物価指数 (平成17年基準)

この表は 4 個の分類軸 (次元) を持つ:
● tab    = 表章項目 (選択肢 7 件)
● cat01   = 平成17年基準品目 (選択肢 802 件)
● area    = 地域 (選択肢 65 件)
● time    = 時間軸 (年・月) (選択肢 587 件)
```

問 3.問題 巨大な表を取得する前に、件数だけを問い合わせる方法を確認せよ。

解答 cntGetFlg=Y を使う例題 5 を実行する。

実行結果 (抜粋)

```
$ uv run scripts/ex05_count_records.py
対象の統計表: [ID:0003217721] 就業状態, 年齢階級別15歳以上人口(2018年1-3月期〜)
この表の総データ件数 (セル数): 41,474 件
1回 100,000 件取得とすると、全件には約 1 回のリクエストが必要。
→ 1回のリクエストで全件取得できる規模です。
```

問 4.問題 NEXT_KEY と startPosition を使ったページングの流れを説明せよ。

解答 例題 11 を実行する。NEXT_KEY は次の開始位置であり、ページ番号ではない。

実行結果（抜粋）

```
$ uv run scripts/ex11_pagination.py
ページ1: TOTAL=13,496,127 件中 1~1000 を取得 (NEXT_KEY=1001)
ページ2: TOTAL=13,496,127 件中 1001~2000 を取得 (NEXT_KEY=2001)
ページ3: TOTAL=13,496,127 件中 2001~3000 を取得 (NEXT_KEY=3001)
```

問 5.問題 CSV 簡易版と JSON 版で同じ条件の値が一致することを確認せよ。

解答 例題 13 を実行し、最新値の一致確認を見る。

実行結果（抜粋）

```
$ uv run scripts/ex13_csv_vs_json.py
■ CSV(getSimpleStatsData) の列: tab_code, 表章項目, cat01_code, ..., value, annotation
■ JSON の直近3件:
2026年3月 112.7
2026年4月 113.0
2026年5月 113.5

最新値の一致確認: CSV=113.5 / JSON=113.5 → 一致
```

以上で基礎は完了である。第 2 部では、これらを組み合わせて各種データを入手する 30 個の例題に取り組む。

第 II 部 各種データを入手する例題 30 選

第 2 部では、e-Stat API を使って実際に各種データを入手する例題を 30 個示す。各スクリプトは PEP 723 形式の単体実行可能なファイルで、リポジトリの scripts/ 以下に置かれている。実行は次のとおり（ESTAT_APPID を設定済みとする）。

```
set -a; source .env; set +a
uv run scripts/ex01_search_statslist.py
```

掲載するコードと実行結果は、本書執筆時点で実際に API を呼び出して得られたものである（統計値・公開時点・検索ヒット件数は更新されうる）。実行結果は要点を示すための抜粋であり、同じスクリプトでも API の更新後には直近月や件数が変わることがある。例題は次の 4 グループに分かれる。

グループ A（例題 1-6） API 操作の基礎：検索・メタ情報・レスポンス構造。

グループ B（例題 7-13） getStatsData の絞り込み技法。

グループ C（例題 14-27） 実データの取得と pandas/matplotlib による分析・可視化。

グループ D（例題 28-30） データカタログ・一括取得・再利用可能なクライアント。

プログラミングが初めての方は、まず次の第 19 節「まず動かす」で、十数行の最小スクリプトを

動かして雰囲気をつかむとよい。

19 まず動かす：最小のスク립ト

ここでは、共通モジュールも関数も使わず、十数行で動く最小のスク립トを4本示す。まずは「APIを叩くとデータが返ってくる」感覚をつかむのが目的なので、エラー処理などは省いている。仕組みを理解したうえで本格的に書くときは、第20節以降の書き方（STATUS 確認・コード→名称変換・ページング）を参考にしてほしい。

最小例の前提

いずれも実行前に `set -a; source .env; set +a` で `ESTAT_APPID` を環境変数に読み込んでおくこと（第3節参照）。

19.1 最小例1：統計表を検索する

`getStatsList` に `appId` と `searchWord` を渡すだけ。返ってきた JSON からヒット件数と表題を取り出して表示する。

scripts/mini_hello_search.py	最小の検索
<pre>1 # /// script 2 # requires-python = ">=3.11" 3 # dependencies = ["requests"] 4 # /// 5 # 【超入門1】e-Stat API で統計表を検索する、最小のスク립ト。 6 # 実行: set -a; source .env; set +a (で ESTAT_APPID を読み込む) 7 # uv run scripts/mini_hello_search.py 8 import os 9 import requests 10 11 app_id = os.environ["ESTAT_APPID"] # appId を環境変数から取得 12 13 # 「人口」というキーワードで統計表を検索する (JSON形式で受け取る) 14 url = "https://api.e-stat.go.jp/rest/3.0/app/json/getStatsList" 15 params = {"appId": app_id, "searchWord": "人口", "limit": 3} 16 data = requests.get(url, params=params).json() 17 18 # ヒット件数と、先頭3件の表題を表示する 19 info = data["GET_STATS_LIST"]["DATA_LIST_INF"] 20 print(info["NUMBER"], "件ヒットしました。先頭3件:") 21 for table in info["TABLE_INF"]: 22 print(" -", table["TITLE"]["\$"] if isinstance(table["TITLE"], dict) else table["TITLE"])</pre>	

実行結果（抜粋）

25016 件ヒットしました。先頭3件：

- 人口及び人口増加（29）、男女別（3）人口数 - 総人口、日本人人口、外国人人口、全国
- [参考表]年齢各歳（94）、男女別（3）、…全国

19.2 最小例2：数値を1つ取り出す

getStatsData で消費者物価指数を取得し、VALUE の先頭セルの \$（数値）を表示する。

scripts/min2_get_one_value.py

最小のデータ取得

```
1 # /// script
2 # requires-python = ">=3.11"
3 # dependencies = ["requests"]
4 # ///
5 # 【超入門2】統計データから「数値を1つ」取り出す、最小のスクリプト。
6 #   消費者物価指数（総合・全国）の最新の指数を表示する。
7 #   実行: set -a; source .env; set +a
8 #       uv run scripts/min2_get_one_value.py
9 import os
10 import requests
11
12 app_id = os.environ["ESTAT_APPID"]
13
14 url = "https://api.e-stat.go.jp/rest/3.0/app/json/getStatsData"
15 params = {
16     "appId": app_id,
17     "statsDataId": "0003427113", # 消費者物価指数（2020年基準）
18     "cdCat01": "0001",          # 品目: 総合
19     "cdArea": "00000",          # 地域: 全国
20     "cdTab": "1",               # 表章項目: 指数
21     "limit": 10,                # 新しい順に10件取得
22 }
23 data = requests.get(url, params=params).json()
24
25 # VALUE は数値セルのリスト。この表は新しい順なので [0] が最新。"$" がその数値。
26 value = data["GET_STATS_DATA"]["STATISTICAL_DATA"]["DATA_INF"]["VALUE"][0]
27 print("消費者物価指数（総合・全国）の最新値:", value["$"], "（2020年平均=100）")
```

実行結果（抜粋）

消費者物価指数（総合・全国）の最新値: 113.5 （2020年平均=100）

19.3 最小例3：CSV を pandas で読む

CSV 版（getSimpleStatsData）なら、返ってきた文字列をそのまま pandas.read_csv に渡すだけで表になる。

```

1 # /// script
2 # requires-python = ">=3.11"
3 # dependencies = ["requests", "pandas"]
4 # ///
5 # 【超入門3】CSV形式で取得して pandas の表として読む、最小のスクリプト。
6 # getSimpleStatsData は CSV を返すので、そのまま read_csv に渡せる。
7 # 実行: set -a; source .env; set +a
8 # uv run scripts/min3_csv_to_pandas.py
9 import io
10 import os
11 import requests
12 import pandas as pd
13
14 app_id = os.environ["ESTAT_APPID"]
15
16 url = "https://api.e-stat.go.jp/rest/3.0/app/getSimpleStatsData" # ← CSV版
17 params = {
18     "appId": app_id,
19     "statsDataId": "0003427113", # 消費者物価指数（2020年基準）
20     "cdCat01": "0001", # 総合
21     "cdArea": "00000", # 全国
22     "cdTab": "1", # 指数
23     "sectionHeaderFlg": "2", # 説明ヘッダ行を省く
24     "limit": 5,
25 }
26 csv_text = requests.get(url, params=params).text
27 df = pd.read_csv(io.StringIO(csv_text)) # 文字列を pandas の表に
28
29 print(df[["時間軸（年・月）", "value"]]) # 時点と値だけ表示

```

実行結果（抜粋）

	時間軸（年・月）	value
0	2026年5月	113.5
1	2026年4月	113.0
2	2026年3月	112.7
3	2026年2月	112.2
4	2026年1月	112.9

19.4 最小例4：グラフにする

取得した CSV を pandas で読み、matplotlib で折れ線にして保存する。日本語が表示できない（□になる）ときは、フォント名を環境に合わせて変える（本文第 16 節参照）。

```

1 # /// script
2 # requires-python = ">=3.11"
3 # dependencies = ["requests", "pandas", "matplotlib"]
4 # ///
5 # 【超入門4】取得したデータをグラフにする、最小のスクリプト。

```

```

5 # 消費者物価指数（総合・全国）の月次推移を折れ線にして figures/ に保存する。
6 # 実行: set -a; source .env; set +a
7 # uv run scripts/min4_quick_plot.py
8
9 import io
10 import os
11 import requests
12 import pandas as pd
13 import matplotlib
14 matplotlib.use("Agg") # 画面を使わず画像ファイルに保存するための設定
15 import matplotlib.pyplot as plt
16
17 # 日本語が□（豆腐）になる場合は、環境にあるフォント名に変えてください
18 plt.rcParams["font.family"] = "Noto Sans CJK JP"
19
20 app_id = os.environ["ESTAT_APPID"]
21
22 url = "https://api.e-stat.go.jp/rest/3.0/app/getSimpleStatsData"
23 params = {
24     "appId": app_id, "statsDataId": "0003427113",
25     "cdCat01": "0001", "cdArea": "00000", "cdTab": "1",
26     "lvTime": "4", "cdTimeFrom": "2024000101", # 2024年1月以降の月次
27     "sectionHeaderFlg": "2",
28 }
29 df = pd.read_csv(io.StringIO(requests.get(url, params=params).text))
30 df = df.sort_values("time_code") # 時点コードの昇順に並べ替え
31
32 plt.plot(range(len(df)), df["value"], marker="o")
33 plt.title("消費者物価指数（総合・全国）の推移")
34 plt.ylabel("指数（2020年平均=100）")
35 plt.xlabel("月（古い→新しい）")
36
37 os.makedirs("figures", exist_ok=True)
38 plt.savefig("figures/min4_cpi.pdf", bbox_inches="tight")
39 print("グラフを figures/min4_cpi.pdf に保存しました。")

```

実行結果（抜粋）

グラフを figures/min4_cpi.pdf に保存しました。

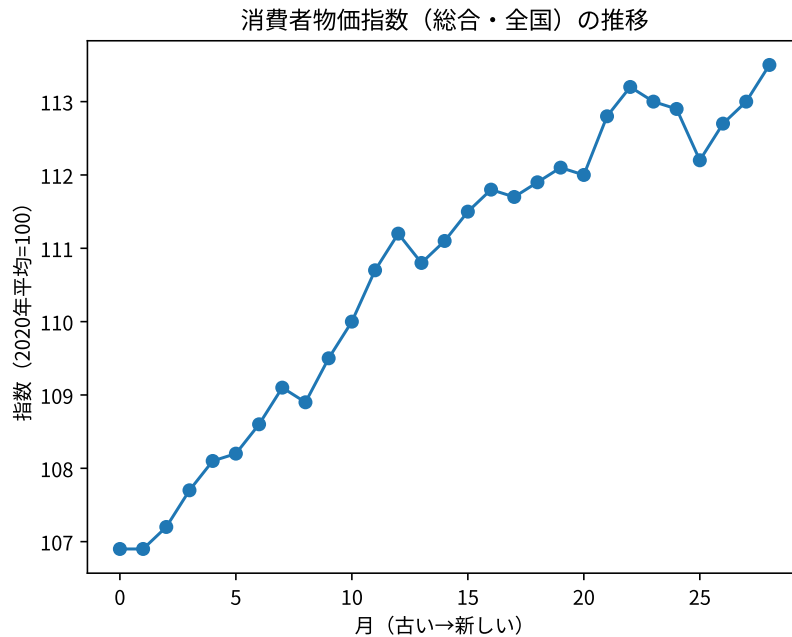


図1 最小例4の出力：消費者物価指数の推移

20 共通モジュール

例題7以降では、繰り返し使う処理を2つの小さなモジュールにまとめて再利用する。`estat_api.py` は API 呼び出しとコード→名称変換を担い、`jpfont.py` は `matplotlib` の日本語表示を有効にする。

scripts/estat_api.py
API 呼び出しの共通処理

```

1  """e-Stat API 用の小さな共通モジュール（例題7以降で再利用する）。
2
3  このファイルは他のスクリプトから import して使う。import 専用のため PEP 723 の
4  メタデータは持たない（利用側スクリプトの dependencies に requests, pandas を書く）。
5
6  主な関数:
7      get_app_id()                環境変数 ESTAT_APPID を取得
8      get_stats_list(app_id, **params)  getStatsList → TABLE_INF のリスト
9      get_meta_info(app_id, sid)      getMetaInfo → CLASS_OBJ のリスト
10     get_stats_data(app_id, sid, ...)  getStatsData → STATISTICAL_DATA (fetch_all で全件)
11     stats_data_to_frame(stat_data)    コードを名称に変換した tidy な DataFrame
12
13     from __future__ import annotations
14
15     import os
16     import sys
17

```

```

18 import pandas as pd
19 import requests
20
21 BASE = "https://api.e-stat.go.jp/rest/3.0/app/json"
22
23
24 def get_app_id() -> str:
25     """環境変数 ESTAT_APPID を取得する（未設定なら終了）。"""
26     app_id = os.environ.get("ESTAT_APPID")
27     if not app_id:
28         sys.exit(
29             "環境変数 ESTAT_APPID が未設定です。'.env' を用意し、"
30             "'set -a; source .env; set +a' を実行してください。"
31         )
32     return app_id
33
34
35 def as_list(x) -> list:
36     """単一オブジェクトのときはリストに包む（要素数1だと配列にならないAPI仕様への対策）。"""
37     return x if isinstance(x, list) else ([ if x is None else [x]])
38
39
40 def text(x) -> str:
41     """{'@...':..., '$':本文'} 形式の値を文字列に正規化する。"""
42     return x["$"] if isinstance(x, dict) else (" if x is None else str(x))
43
44
45 def _get(func: str, params: dict) -> dict:
46     res = requests.get(f"{BASE}/{func}", params=params, timeout=120)
47     res.raise_for_status()
48     return res.json()
49
50
51 def get_stats_list(app_id: str, **params) -> list[dict]:
52     body = _get("getStatsList", {"appId": app_id, **params})["GET_STATS_LIST"]
53     if body["RESULT"]["STATUS"] != 0:
54         sys.exit(f"getStatsList エラー: {body['RESULT']['ERROR_MSG']}")
55     return as_list(body["DATALIST_INF"].get("TABLE_INF", []))
56
57
58 def get_meta_info(app_id: str, stats_data_id: str) -> list[dict]:
59     body = _get("getMetaInfo", {"appId": app_id, "statsDataId": stats_data_id})["GET_META_INFO"]
60     if body["RESULT"]["STATUS"] != 0:
61         sys.exit(f"getMetaInfo エラー: {body['RESULT']['ERROR_MSG']}")
62     return as_list(body["METADATA_INF"]["CLASS_INF"]["CLASS_OBJ"])
63
64
65 def get_stats_data(app_id: str, stats_data_id: str, *, fetch_all: bool = False, **params) -> dict:
66     """getStatsData を呼ぶ。fetch_all=True なら NEXT_KEY を辿って全件を結合する。"""
67     base = {"appId": app_id, "statsDataId": stats_data_id, **params}
68     body = _get("getStatsData", base)["GET_STATS_DATA"]
69     if body["RESULT"]["STATUS"] != 0:
70         sys.exit(f"getStatsData エラー: {body['RESULT']['ERROR_MSG']}")

```

```

71 sdata = body["STATISTICAL_DATA"]
72 values = as_list(sdata.get("DATA_INF", {}).get("VALUE"))
73 next_key = sdata["RESULT_INF"].get("NEXT_KEY")
74 while fetch_all and next_key:
75     body = _get("getStatsData", {**base, "startPosition": next_key})["GET_STATS_DATA"]
76     if body["RESULT"]["STATUS"] != 0:
77         sys.exit(f"getStatsData エラー: {body['RESULT']['ERROR_MSG']}")
78     sd2 = body["STATISTICAL_DATA"]
79     values += as_list(sd2.get("DATA_INF", {}).get("VALUE"))
80     next_key = sd2["RESULT_INF"].get("NEXT_KEY")
81 sdata.setdefault("DATA_INF", {})[ "VALUE" ] = values
82 return sdata
83
84
85 def stats_data_to_frame(sdata: dict) -> pd.DataFrame:
86     """STATISTICAL_DATA を tidy な DataFrame にする。
87
88     返る列は予測可能な名前にそろえる:
89     - 各次元 id (tab, cat01, area, time ...) には「名称」が入る
90     - 同名 + '_code' 列に元のコードが入る
91     - '値' に数値、'単位' に単位
92     """
93     objs = as_list(sdata["CLASS_INF"]["CLASS_OBJ"])
94     code2name = {o["@id"]: {c["@code"]: c["@name"]} for c in as_list(o["CLASS"])} for o in objs}
95
96     raw = pd.DataFrame(as_list(sdata["DATA_INF"]["VALUE"]))
97     out = {}
98     for col in raw.columns:
99         if col == "$":
100             out["値"] = pd.to_numeric(raw[col], errors="coerce")
101         elif col == "@unit":
102             out["単位"] = raw[col]
103         elif col == "@annotation":
104             out["annotation"] = raw[col]
105         elif col.startswith("@"):
106             key = col[1:] # 例: '@cat01' → 'cat01'
107             out[f"{key}_code"] = raw[col]
108             if key in code2name:
109                 out[key] = raw[col].map(code2name[key])
110     return pd.DataFrame(out)

```

scripts/jpfont.py

matplotlib の日本語フォント設定

```

1 """matplotlib をグラフ内で日本語表示できるようにするモジュール。
2
3 import するだけで、システムにインストールされた日本語フォント
4 (Noto Sans CJK JP / IPAexGothic / Takao など) を自動的に選んで設定する。
5 外部の日本語化専用パッケージに依存すると環境差が出ることがあるため、
6 本書ではこの軽量モジュールを用いる。
7 """
8 import matplotlib
9 from matplotlib import font_manager
10

```

```

11 # 優先的に使いたい日本語フォント（先に見つかったものを採用）
12 _PREFERRED = [
13     "Noto Sans CJK JP", "Noto Sans JP", "IPAexGothic", "IPAPGothic",
14     "TakaoPGothic", "BIZ UDPGothic", "Yu Gothic", "Hiragino Sans",
15     "Meiryo", "MS Gothic", "VL PGothic",
16 ]
17
18
19 def set_japanese_font() -> str | None:
20     """利用可能な日本語フォントを1つ選んで matplotlib に設定する。"""
21     available = {f.name for f in font_manager.fontManager.ttflist}
22     for name in _PREFERRED:
23         if name in available:
24             matplotlib.rcParams["font.family"] = name
25             matplotlib.rcParams["axes.unicode_minus"] = False # 軸のマイナス記号化け対策
26             return name
27     return None
28
29
30 # import された時点で自動設定する
31 set_japanese_font()

```

ヒント

stats_data_to_frame は、各次元（tab, cat01, area, time …）について「名称の列」と「_code を付けたコードの列」を作る。これにより、どの統計表でも `df["time"]・df["値"]` のように予測可能な列名で扱える。値に注釈コードが付いている場合は `df["annotation"]` に残す。

21 グループ A：API 操作の基礎

21.1 例題 1：キーワードで統計表を検索する

この例題のポイント

API：getStatsList / 主なパラメータ：searchWord, limit

最初の一步として、キーワード「人口」を含む統計表を検索し、各表の統計表 ID (@id) と表題を一覧する。応答の DATALIST_INF.TABLE_INF に表の配列が入る。

scripts/ex01_search_statslist.py

getStatsList でキーワード検索

```

1 # /// script
2 # requires-python = ">=3.11"
3 # dependencies = ["requests"]
4 # ///
5 """例題1: getStatsList でキーワード検索し、統計表の一覧を表示する。
6
7 e-Stat API の最初の一步。キーワード「人口」を含む統計表を検索し、

```

```

8 各表に付く統計表ID (@id = statsDataId) と表題を表示する。
9
10 実行:
11     set -a; source .env; set +a          # ESTAT_APPID を環境変数に展開
12     uv run scripts/ex01_search_statslist.py
13     """
14 import os
15 import sys
16
17 import requests
18
19 BASE = "https://api.e-stat.go.jp/rest/3.0/app/json"
20
21
22 def get_app_id() -> str:
23     """環境変数 ESTAT_APPID を取得する (未設定なら終了)。"""
24     app_id = os.environ.get("ESTAT_APPID")
25     if not app_id:
26         sys.exit(
27             "環境変数 ESTAT_APPID が未設定です。'.env' を用意し、"
28             "'set -a; source .env; set +a' を実行してから再度実行してください。"
29         )
30     return app_id
31
32
33 def text(x) -> str:
34     """値が {'@...':..., '$': '本文'} の辞書のことがあるので文字列に正規化する。"""
35     return x["$"] if isinstance(x, dict) else (" if x is None else str(x))
36
37
38 def main() -> None:
39     app_id = get_app_id()
40
41     # --- リクエストのパラメータ ---
42     params = {
43         "appId": app_id,
44         "searchWord": "人口",    # 「人口」を含む統計表を検索
45         "limit": 10,            # 先頭10件だけ取得
46     }
47     res = requests.get(f"{BASE}/getStatsList", params=params, timeout=60)
48     res.raise_for_status()
49     body = res.json()["GET_STATS_LIST"]
50
51     # --- STATUS が 0 以外ならエラー ---
52     status = body["RESULT"]["STATUS"]
53     if status != 0:
54         sys.exit(f"APIエラー (STATUS={status}): {body['RESULT']['ERROR_MSG']}")
55
56     info = body["DATALIST_INF"]
57     print(f"ヒット総数: {info.get('NUMBER', '不明')} 件 (先頭 {params['limit']} 件を表示) \n")
58
59     # 1件だけのときは配列にならないのでリストに包む
60     tables = info["TABLE_INF"]

```

```

61 tables = tables if isinstance(tables, list) else [tables]
62
63 for t in tables:
64     stat_name = text(t.get("STAT_NAME"))      # 政府統計名
65     title = text(t.get("TITLE"))              # 統計表の表題
66     survey = text(t.get("SURVEY_DATE"))       # 調査年月
67     print(f"[ID:{t['@id']}] {stat_name}")
68     print(f"        表題: {title} (調査:{survey})")
69
70
71 if __name__ == "__main__":
72     main()

```

実行結果（抜粋）

ヒット総数: 25016 件（先頭 10 件を表示）

```

[ID:0000150041] 人口推計
    表題: 人口及び人口増加（29）、男女別（3）人口数…全国（調査:199310）
[ID:0000150271] 人口推計
    表題: [参考表]年齢各歳（94）、男女別（3）、…全国（調査:200610）
...

```

21.2 例題 2：政府統計コードで絞り、CSV に保存する

この例題のポイント

API：getStatsList / パラメータ：statsCode=00200521（国勢調査）

検索結果を pandas の DataFrame に整え、Excel で開けるよう BOM 付き UTF-8 の CSV に保存する。必要な列だけを取り出すのがコツ。

scripts/ex02_search_by_statscode.py

検索結果を CSV に保存

```

1 # /// script
2 # requires-python = ">=3.11"
3 # dependencies = ["requests", "pandas"]
4 # ///
5 """例題2: 政府統計コードで絞って検索し、一覧を pandas で整形して CSV 保存する。
6
7 statsCode=00200521（国勢調査）の統計表を検索し、主要な項目を DataFrame にして
8 output/ex02_statslist_census.csv に保存する。Excel で開けるよう BOM 付き UTF-8 とする。
9
10 実行:
11     set -a; source .env; set +a
12     uv run scripts/ex02_search_by_statscode.py
13 """
14 import os
15 import sys
16
17 import pandas as pd
18 import requests
19

```

```

20 BASE = "https://api.e-stat.go.jp/rest/3.0/app/json"
21
22
23 def get_app_id() -> str:
24     app_id = os.environ.get("ESTAT_APPID")
25     if not app_id:
26         sys.exit("環境変数 ESTAT_APPID が未設定です。'.env' を設定してください。")
27     return app_id
28
29
30 def text(x) -> str:
31     return x["$"] if isinstance(x, dict) else (" if x is None else str(x))
32
33
34 def main() -> None:
35     app_id = get_app_id()
36
37     params = {
38         "appId": app_id,
39         "statsCode": "00200521", # 国勢調査 (政府統計コード)
40         "limit": 100,
41     }
42     res = requests.get(f"{BASE}/getStatsList", params=params, timeout=60)
43     res.raise_for_status()
44     body = res.json()["GET_STATS_LIST"]
45     if body["RESULT"]["STATUS"] != 0:
46         sys.exit(f"APIエラー: {body['RESULT']['ERROR_MSG']}")
47
48     tables = body["DATALIST_INF"]["TABLE_INF"]
49     tables = tables if isinstance(tables, list) else [tables]
50
51     # 必要な列だけ取り出して表 (DataFrame) にする
52     rows = []
53     for t in tables:
54         rows.append(
55             {
56                 "統計表ID": t["@id"],
57                 "政府統計名": text(t.get("STAT_NAME")),
58                 "提供統計名": text(t.get("STATISTICS_NAME")),
59                 "表題": text(t.get("TITLE")),
60                 "周期": text(t.get("CYCLE")),
61                 "調査年月": text(t.get("SURVEY_DATE")),
62                 "公開日": text(t.get("OPEN_DATE")),
63             }
64         )
65     df = pd.DataFrame(rows)
66
67     os.makedirs("output", exist_ok=True)
68     out = "output/ex02_statslist_census.csv"
69     df.to_csv(out, index=False, encoding="utf-8-sig") # BOM付きで Excel 対応
70
71     print(f"国勢調査の統計表: {len(df)} 件取得")
72     print(df[["統計表ID", "表題", "調査年月"]].head(10).to_string(index=False))

```

```
70     print(f"\n→ 全件を {out} に保存しました。")
71
72
73 if __name__ == "__main__":
74     main()
75
```

実行結果（抜粋）

国勢調査の統計表: 100 件取得

統計表ID	表題	調査年月
000000300001	男女の別（性別）（3），年齢5歳階級（23），人口	198010
...		

→ 全件を output/ex02_statslist_census.csv に保存しました。

21.3 例題 3：分野と複合キーワードで横断検索する

この例題のポイント

API: `getStatsList` / パラメータ: `statsField`, `searchWord(AND/OR)`, `collectArea`

統計分野コード (statsField) での絞り込み、空白による AND 検索、OR 演算子、集計地域区分 (collectArea) の指定を組み合わせる。

```

scripts/ex03_search_by_field.py
分野・AND/OR・地域区分で検索

1 # /// script
2 # requires-python = ">=3.11"
3 # dependencies = ["requests"]
4 # ///
5 """例題3: 統計分野 (statsField) と複合キーワードで横断検索する。
6
7 statsField は統計分野コード (2桁=大分類)。ここでは「03 労働・賃金」分野から、
8 キーワードを AND 結合 (空白区切り) して検索する例を示す。
9 searchKind / collectArea の使い方もコメントで示す。
10
11 実行:
12     set -a; source .env; set +a
13     uv run scripts/ex03_search_by_field.py
14 """
15 import os
16 import sys
17
18 import requests
19
20 BASE = "https://api.e-stat.go.jp/rest/3.0/app/json"
21
22
23 def get_app_id() -> str:
24     app_id = os.environ.get("ESTAT_APPID")
25     if not app_id:
26         sys.exit("環境変数 ESTAT_APPID が未設定です。'.env' を設定してください。")
27     return app_id

```

```

29
30 def text(x) -> str:
31     return x["$"] if isinstance(x, dict) else (" if x is None else str(x))
32
33
34 def search(app_id: str, **extra) -> dict:
35     """getStatsList を呼んで DATALIST_INF を返す共通関数。"""
36     params = {"appId": app_id, "limit": 10, **extra}
37     res = requests.get(f"{BASE}/getStatsList", params=params, timeout=60)
38     res.raise_for_status()
39     body = res.json()["GET_STATS_LIST"]
40     if body["RESULT"]["STATUS"] != 0:
41         sys.exit(f"APIエラー: {body['RESULT']['ERROR_MSG']}")
42     return body["DATALIST_INF"]
43
44
45 def show(info: dict, header: str) -> None:
46     print(f"\n=== {header} ===")
47     print(f"ヒット総数: {info.get('NUMBER', '不明')} 件")
48     tables = info.get("TABLE_INF", [])
49     tables = tables if isinstance(tables, list) else [tables]
50     for t in tables[:5]:
51         print(f" ID:{t['@id']} {text(t.get('STATISTICS_NAME'))} / {text(t.get('TITLE'))}")
52
53
54 def main() -> None:
55     app_id = get_app_id()
56
57     # (1) 分野「03 労働・賃金」× キーワード AND (空白区切り)
58     show(
59         search(app_id, statsField="03", searchWord="賃金 産業"),
60         "分野=労働・賃金, 「賃金」AND「産業」",
61     )
62
63     # (2) OR 検索 ("|" でなく OR と書く)
64     show(
65         search(app_id, searchWord="完全失業率 OR 有効求人倍率"),
66         "「完全失業率」OR「有効求人倍率」",
67     )
68
69     # (3) 市区町村別データに限定 (collectArea=3)。searchKind=2 は小地域・地域メッシュ。
70     show(
71         search(app_id, searchWord="人口", collectArea="3"),
72         "「人口」かつ 市区町村別集計 (collectArea=3)",
73     )
74
75
76 if __name__ == "__main__":
77     main()

```

実行結果（抜粋）

```
=== 分野=労働・賃金, 「賃金」 AND 「産業」 ===
ヒット総数: 573 件
[ID:0003029886] 賃金構造基本統計調査 … 年齢階級別きまって支給する現金給与額…
=== 「完全失業率」 OR 「有効求人倍率」 ===
ヒット総数: 35 件
=== 「人口」かつ 市区町村別集計 (collectArea=3) ===
ヒット総数: 1255 件
```

21.4 例題 4：メタ情報（分類の定義）を取得する

この例題のポイント

API：getMetaInfo ／ 着目点：分類軸（CLASS_OBJ）と階層レベル

統計表 ID を 1 件見つけ、その表が持つ分類軸（次元）と、各分類で指定できるコード・階層レベル（@level）を一覧する。これが getStatsData の絞り込みの設計図になる。

scripts/ex04_getmetainfo.py

getMetaInfo で分類を調べる

```
1 # /// script
2 # requires-python = ">=3.11"
3 # dependencies = ["requests"]
4 # ///
5 """例題4: getMetaInfo で統計表のメタ情報（分類の定義）を取得する。
6
7 まず getStatsList で対象の統計表IDを1件見つけ、その表の「どんな項目があり、
8 どんなコードを指定できるか」を getMetaInfo で取得して表示する。
9 各分類の階層レベル（@level）も確認する。
10
11 実行:
12     set -a; source .env; set +a
13     uv run scripts/ex04_getmetainfo.py
14 """
15 import os
16 import sys
17
18 import requests
19
20 BASE = "https://api.e-stat.go.jp/rest/3.0/app/json"
21
22
23 def get_app_id() -> str:
24     app_id = os.environ.get("ESTAT_APPID")
25     if not app_id:
26         sys.exit("環境変数 ESTAT_APPID が未設定です。'.env' を設定してください。")
27     return app_id
28
29
30 def text(x) -> str:
31     return x["$"] if isinstance(x, dict) else (" " if x is None else str(x))
32
```

```

33
34 def as_list(x) -> list:
35     """単一オブジェクトのときはリストに包む。"""
36     return x if isinstance(x, list) else [x]
37
38
39 def find_one_table(app_id: str, stats_code: str) -> tuple[str, str]:
40     """statsCode に該当する統計表を1件見つけ、(ID, 表題) を返す。"""
41     params = {"appId": app_id, "statsCode": stats_code, "limit": 1}
42     body = requests.get(f"{BASE}/getStatsList", params=params, timeout=60).json()["GET_STATS_LIST"]
43     if body["RESULT"]["STATUS"] != 0:
44         sys.exit(f"検索エラー: {body['RESULT']['ERROR_MSG']}")
45     t = as_list(body["DATALIST_INF"]["TABLE_INF"])[0]
46     return t["@id"], text(t.get("TITLE"))
47
48
49 def main() -> None:
50     app_id = get_app_id()
51
52     # 消費者物価指数 (00200573) の統計表を1件見つける
53     stats_data_id, title = find_one_table(app_id, "00200573")
54     print(f"対象の統計表: [ID:{stats_data_id}] {title}\n")
55
56     # その表のメタ情報を取得
57     params = {"appId": app_id, "statsDataId": stats_data_id}
58     body = requests.get(f"{BASE}/getMetaInfo", params=params, timeout=60).json()["GET_META_INFO"]
59     if body["RESULT"]["STATUS"] != 0:
60         sys.exit(f"メタ取得エラー: {body['RESULT']['ERROR_MSG']}")
61
62     class_objs = as_list(body["METADATA_INF"]["CLASS_INF"]["CLASS_OBJ"])
63     print(f"この表は {len(class_objs)} 個の分類軸 (次元) を持つ:\n")
64     for obj in class_objs:
65         classes = as_list(obj["CLASS"])
66         print(f"● {obj['@id']:6s} = {obj['@name']} (選択肢 {len(classes)} 件) ")
67         for c in classes[:3]: # 先頭3件だけ例示
68             lv = c.get("@level") or "-"
69             print(f"        code={c['@code']:>10s} level={lv} {c['@name']}")
70         if len(classes) > 3:
71             print("        ...")
72
73
74 if __name__ == "__main__":
75     main()

```

実行結果 (抜粋)

対象の統計表: [ID:0002050001] 消費者物価指数 (平成17年基準)

この表は 4 個の分類軸 (次元) を持つ:

- tab = 表章項目 (選択肢 7 件)
- cat01 = 平成17年基準品目 (選択肢 802 件)
 - code= 0001 level=1 0001 総合
- area = 地域 (選択肢 65 件)

● time = 時間軸 (年・月) (選択肢 587 件)

ヒント

この例題は `getStatsList` の先頭 1 件を使ってメタ情報の構造を観察するため、古い基準年の表が出ることがある。実データの分析では、例題 7 以降のように目的に合う統計表 ID と分類コードを確認してから使う。

21.5 例題 5：取得件数を先に確認する (`cntGetFlg`)

この例題のポイント

API：`getStatsData` / パラメータ：`cntGetFlg=Y`

巨大な表を全件取得する前に、`cntGetFlg=Y` で件数 (`TOTAL_NUMBER`) だけを問い合わせ、ページングが必要かを判断する。

scripts/ex05_count_records.py

件数だけを確認する

```
1 # /// script
2 # requires-python = ">=3.11"
3 # dependencies = ["requests"]
4 # ///
5 """例題5: cntGetFlg=Y で「取得件数」だけを先に確認する。
6
7 巨大な統計表をいきなり全件取得すると時間も通信量もかかる。getStatsData に
8 cntGetFlg=Y を付けると、データ本体を返さずに該当件数 (TOTAL_NUMBER) だけを返す。
9 これで規模を把握し、limit やページングの方針を決められる。
10
11 実行:
12     set -a; source .env; set +a
13     uv run scripts/ex05_count_records.py
14 """
15 import os
16 import sys
17
18 import requests
19
20 BASE = "https://api.e-stat.go.jp/rest/3.0/app/json"
21
22
23 def get_app_id() -> str:
24     app_id = os.environ.get("ESTAT_APPID")
25     if not app_id:
26         sys.exit("環境変数 ESTAT_APPID が未設定です。'.env' を設定してください。")
27     return app_id
28
29
30 def text(x) -> str:
31     return x["$"] if isinstance(x, dict) else (" " if x is None else str(x))
```

```

32
33
34 def find_one_table(app_id: str, stats_code: str) -> tuple[str, str]:
35     params = {"appId": app_id, "statsCode": stats_code, "limit": 1}
36     body = requests.get(f"{BASE}/getStatsList", params=params, timeout=60).json()["GET_STATS_LIST"]
37     t = body["DATALIST_INF"]["TABLE_INF"]
38     t = t[0] if isinstance(t, list) else t
39     return t["@id"], text(t.get("TITLE"))
40
41
42 def main() -> None:
43     app_id = get_app_id()
44
45     # 労働力調査 (00200531) の表を1件見つける
46     stats_data_id, title = find_one_table(app_id, "00200531")
47     print(f"対象の統計表: [ID:{stats_data_id}] {title}\n")
48
49     # cntGetFlg=Y: 件数だけを問い合わせる (データ本体は返らない)
50     params = {
51         "appId": app_id,
52         "statsDataId": stats_data_id,
53         "cntGetFlg": "Y",
54         "metaGetFlg": "N",
55     }
56     body = requests.get(f"{BASE}/getStatsData", params=params, timeout=60).json()["GET_STATS_DATA"]
57     if body["RESULT"]["STATUS"] != 0:
58         sys.exit(f"APIエラー: {body['RESULT']['ERROR_MSG']}")
59
60     total = int(body["STATISTICAL_DATA"]["RESULT_INF"]["TOTAL_NUMBER"])
61     print(f"この表の総データ件数 (セル数) : {total:,} 件")
62
63     limit = 100_000 # 1回あたりの既定上限
64     pages = -(-total // limit) # 切り上げ
65     print(f"1回 {limit:,} 件取得とすると、全件には約 {pages} 回のリクエストが必要。")
66     if total <= limit:
67         print("→ 1回のリクエストで全件取得できる規模です。")
68     else:
69         print("→ NEXT_KEY を使ったページネーションが必要です (例題11参照)。")
70
71
72 if __name__ == "__main__":
73     main()

```

実行結果 (抜粋)

対象の統計表: [ID:0003217721] 就業状態, 年齢階級別15歳以上人口(2018年1-3月期～)
 この表の総データ件数 (セル数) : 41,474 件
 → 1回のリクエストで全件取得できる規模です。

21.6 例題 6：生の JSON 構造を観察する

この例題のポイント

API：getStatsData / 着目点：RESULT_INF / CLASS_INF / DATA_INF

少量 (limit=3) だけ取得して、レスポンスの入れ子構造を実際に目で確かめる。メタ (CLASS_OBJ) とデータ (VALUE) の対応 (第 7 節) を再確認しよう。

scripts/ex06_inspect_json.py

レスポンス構造を観察する

```
1 # /// script
2 # requires-python = ">=3.11"
3 # dependencies = ["requests"]
4 # ///
5 """例題6: getStatsData の生 JSON 構造を観察する。
6
7 少量 (limit=3) だけ取得し、レスポンスの入れ子構造 (RESULT / RESULT_INF /
8 CLASS_INF / DATA_INF) を実際に目で見て理解する。メタ情報 (CLASS_OBJ) と
9 データ (VALUE) の対応を確かめるのが狙い。
10
11 実行:
12     set -a; source .env; set +a
13     uv run scripts/ex06_inspect_json.py
14 """
15 import json
16 import os
17 import sys
18
19 import requests
20
21 BASE = "https://api.e-stat.go.jp/rest/3.0/app/json"
22
23
24 def get_app_id() -> str:
25     app_id = os.environ.get("ESTAT_APPID")
26     if not app_id:
27         sys.exit("環境変数 ESTAT_APPID が未設定です。'.env' を設定してください。")
28     return app_id
29
30
31 def text(x) -> str:
32     return x["$"] if isinstance(x, dict) else (" " if x is None else str(x))
33
34
35 def as_list(x) -> list:
36     return x if isinstance(x, list) else [x]
37
38
39 def main() -> None:
40     app_id = get_app_id()
```

```

42 # 消費者物価指数の表を1件見つける
43 p = {"appId": app_id, "statsCode": "00200573", "limit": 1}
44 lst = requests.get(f"{BASE}/getStatsList", params=p, timeout=60).json()["GET_STATS_LIST"]
45 table = as_list(lst["DATALIST_INF"]["TABLE_INF"])[0]
46 sid = table["@id"]
47 print(f"対象の統計表: [ID:{sid}] {text(table.get('TITLE'))}\n")
48
49 # 先頭3件だけ取得
50 params = {"appId": app_id, "statsDataId": sid, "limit": 3}
51 sdata = requests.get(f"{BASE}/getStatsData", params=params, timeout=60).json()
52 sdata = sdata["GET_STATS_DATA"]["STATISTICAL_DATA"]
53
54 print("■ RESULT_INF (件数情報) ")
55 print(json.dumps(sdata["RESULT_INF"], ensure_ascii=False, indent=2))
56
57 print("\n■ CLASS_INF の最初の次元 (CLASS_OBJ[0]) の冒頭")
58 obj0 = as_list(sdata["CLASS_INF"]["CLASS_OBJ"])[0]
59 classes = as_list(obj0["CLASS"])
60 preview = {"@id": obj0["@id"], "@name": obj0["@name"], "CLASS(先頭2件)": classes[:2]}
61 print(json.dumps(preview, ensure_ascii=False, indent=2))
62
63 print("\n■ DATA_INF の VALUE (数値セル) 先頭3件")
64 values = as_list(sdata["DATA_INF"]["VALUE"])
65 print(json.dumps(values[:3], ensure_ascii=False, indent=2))
66 print("\n→ VALUE の @cat01 等のコードを、CLASS_OBJ の同じ @id から名称に引ける (例題12)。")
67
68
69 if __name__ == "__main__":
70     main()

```

実行結果 (抜粋)

```

■ RESULT_INF (件数情報)
{ "TOTAL_NUMBER": 10785241, "FROM_NUMBER": 1, "TO_NUMBER": 3, "NEXT_KEY": 4 }
■ DATA_INF の VALUE (数値セル) 先頭3件
[ { "@tab": "1", "@cat01": "0001", "@area": "13A01", "@time": "2011001212", "$": "99.1" }, ... ]

```

22 グループ B：getStatsData の絞り込み技法

22.1 例題 7：時間軸で絞る (cdTimeFrom・lvTime)

この例題のポイント

使う統計：消費者物価指数（2020 年基準）／ ID：0003427113 ／ パラメータ：cdTimeFrom, lvTime

「総合・全国」の月次指数を 2023 年 1 月以降に絞って取得する。月次コードは YYYY00MMMM 形式で、lvTime=4 により月次だけを選ぶ。

```

1 # /// script
2 # requires-python = ">=3.11"
3 # dependencies = ["requests", "pandas"]
4 # ///
5 """例題7: 時間軸で絞り込む (cdTimeFrom)。
6
7 消費者物価指数 (2020年基準, 統計表ID 0003427113) から、「総合」「全国」の
8 月次指数を 2023年1月以降だけ取得する。時間軸コードの範囲指定 (cdTimeFrom) と
9 階層レベル指定 (lvTime=4 で月次のみ) を使う。
10
11 実行:
12     set -a; source .env; set +a
13     uv run scripts/ex07_filter_time.py
14 """
15 import estat_api as e
16
17 SID = "0003427113" # 消費者物価指数 (2020年基準)
18
19
20 def main() -> None:
21     app_id = e.get_app_id()
22
23     sdata = e.get_stats_data(
24         app_id, SID,
25         cdCat01="0001",      # 品目: 総合
26         cdArea="00000",     # 地域: 全国
27         cdTab="1",          # 表章項目: 指数
28         lvTime="4",         # 時間軸の階層レベル4 = 月次のみ
29         cdTimeFrom="2023000101", # 2023年1月以降 (月次コードは YYYY00MMM 形式)
30     )
31     df = e.stats_data_to_frame(sdata).sort_values("time_code")
32
33     print(f"取得した月数: {len(df)} か月")
34     print(" (消費者物価指数 総合・全国・2020年平均=100, 直近12か月) ")
35     print(df[["time", "値"]].tail(12).to_string(index=False))
36
37
38 if __name__ == "__main__":
39     main()

```

実行結果 (抜粋)

```

取得した月数: 41 か月
(消費者物価指数 総合・全国・2020年平均=100, 直近12か月)
   time    値
2025年6月 111.7
...
2026年5月 113.5

```

22.2 例題 8：地域で絞る (lvArea・cdArea)

この例題のポイント

使う統計：人口推計（都道府県別人口）／ ID：0003448232 ／ パラメータ：lvArea=2, cdArea

lvArea=2 で都道府県だけを取り出し、cdArea で特定県（愛知=23000・東京=13000）に絞る方法を示す。地域コードは「全国=00000、都道府県=XX000」の体系。

scripts/ex08_filter_area.py

地域で絞り込む

```
1 # /// script
2 # requires-python = ">=3.11"
3 # dependencies = ["requests", "pandas"]
4 # ///
5 """例題8: 地域で絞り込む (lvArea と cdArea)。
6
7 人口推計（令和2年国勢調査基準，統計表ID 0003448232「都道府県，男女別人口」）から、
8 都道府県別の総人口を取り出す。
9 lvArea=2 で都道府県だけに、cdArea で特定県だけに絞る方法を示す。
10
11 実行:
12     set -a; source .env; set +a
13     uv run scripts/ex08_filter_area.py
14 """
15 import estat_api as e
16
17 SID = "0003448232" # 人口推計 都道府県，男女別人口－総人口，日本人人口
18
19
20 def main() -> None:
21     app_id = e.get_app_id()
22
23     # (1) lvArea=2 → 都道府県だけ（全国を除く）。男女計・総人口の最新年。
24     sd = e.get_stats_data(app_id, SID,
25                           cdTab="001", cdCat01="000", cdCat02="001", lvArea="2")
26     df = e.stats_data_to_frame(sd)
27     latest = df["time_code"].max()
28     df = df[df["time_code"] == latest]
29     print(f"対象年: {df['time'].iloc[0]} / 都道府県数: {len(df)}")
30     print("人口の多い上位5都道府県（単位：千人）：")
31     print(df.sort_values("値", ascending=False)[["area", "値"]].head(5).to_string(index=False))
32
33     # (2) cdArea で特定の都道府県だけ（愛知県=23000，東京都=13000）
34     sd2 = e.get_stats_data(app_id, SID,
35                            cdTab="001", cdCat01="000", cdCat02="001",
36                            cdArea="23000,13000")
37     df2 = e.stats_data_to_frame(sd2)
38     df2 = df2[df2["time_code"] == df2["time_code"].max()]
39     print(f"ncdArea で愛知県・東京都だけを取得（最新年）：")
40     print(df2[["area", "値", "単位"]].to_string(index=False))
41
```

```

42
43 if __name__ == "__main__":
44     main()

```

実行結果（抜粋）

対象年：2024年 / 都道府県数：47

人口の多い上位5都道府県（単位：千人）：

area	値
東京都	14178
神奈川県	9225
大阪府	8757

cdArea で愛知県・東京都だけを取得（最新年）：

東京都	14178 千人
愛知県	7460 千人

22.3 例題 9：分類事項で絞る（cdCat02）

この例題のポイント

使う統計：労働力調査 / ID：0003005865 / パラメータ：cdCat02（就業状態、カンマ区切りで複数）

分類事項（就業状態）をカンマ区切りで複数指定し、就業率と完全失業率を同時に取得する。

scripts/ex09_filter_category.py

分類で絞り込む

```

1 # /// script
2 # requires-python = ">=3.11"
3 # dependencies = ["requests", "pandas"]
4 # ///
5 """例題9：分類事項で絞り込む（cdCat02）。
6
7 労働力調査（統計表ID 0003005865「労働力人口比率、就業率及び完全失業率」）から、
8 「就業状態」分類（cat02）を使って、就業率と完全失業率を取り出す。
9 複数コードはカンマ区切りで一度に指定できる。
10
11 実行：
12     set -a; source .env; set +a
13     uv run scripts/ex09_filter_category.py
14 """
15 import estat_api as e
16
17 SID = "0003005865"
18
19
20 def main() -> None:
21     app_id = e.get_app_id()
22
23     sdata = e.get_stats_data(
24         app_id, SID,
25         cdTab="02",          # 表章項目：率(%)

```

```

26     cdCat01="000",      # 産業: 全産業
27     cdCat02="13,08",   # 就業状態: 13=就業, 08=完全失業者 (カンマで複数)
28     cdCat03="0",       # 性別: 総数
29     cdTimeFrom="2024000101", # 2024年1月以降 (この表の時間軸は月次・階層なし)
30 )
31 df = e.stats_data_to_frame(sdata).sort_values("time_code")
32
33 # cat02 (就業状態) ごとに直近の値を表示
34 for state, sub in df.groupby("cat02"):
35     latest = sub.iloc[-1]
36     print(f"{state}: 直近 {latest['time']} = {latest['値']}{latest['単位']} ({len(sub)}か月分取得)")
37
38
39 if __name__ == "__main__":
40     main()

```

実行結果 (抜粋)

完全失業者: 直近 2026年4月 = 2.7% (28か月分取得)
 就業: 直近 2026年4月 = 62.6% (28か月分取得)

22.4 例題 10：階層レベルで絞る (lvCat01)

この例題のポイント

使う統計：消費者物価指数 (2020 年基準) / ID：0003427113 / パラメータ：lvCat01

品目分類は階層構造を持つ。lvCat01 を変えると取得される品目の粒度が変わることを、件数の変化で確かめる。

scripts/ex10_filter_level.py

階層レベルで絞り込む

```

1  # /// script
2  # requires-python = ">=3.11"
3  # dependencies = ["requests", "pandas"]
4  # ///
5  """例題10: 階層レベルで絞り込む (lvCat01)。
6
7  分類事項は階層構造を持つ。消費者物価指数の「品目」 (cat01) は
8  レベル1 (総合・10大費目など) → レベル3 (中分類) → さらに細目…と深くなる。
9  lvCat01 を変えると、取得される品目の粒度が変わることを確かめる。
10
11  実行:
12      set -a; source .env; set +a
13      uv run scripts/ex10_filter_level.py
14  """
15  import estat_api as e
16
17  SID = "0003427113" # 消費者物価指数 (2020年基準)
18
19
20  def main() -> None:

```

```

21 app_id = e.get_app_id()
22
23 print("lvCat01（品目の階層レベル）を変えたときに取得される品目数：\n")
24 for lv in ["1", "1-2", "1-3", "1-4"]:
25     sdata = e.get_stats_data(
26         app_id, SID,
27         cdArea="00000", cdTab="1",
28         lvTime="4", cdTimeFrom="2025000101", # 2025年以降の月次に限定して件数を抑える
29         lvCat01=lv,
30     )
31     df = e.stats_data_to_frame(sdata)
32     n_items = df["cat01_code"].nunique()
33     print(f"    lvCat01={lv:4s} → 品目 {n_items:4d} 種類")
34
35 print("\nlvCat01=1（最上位）の品目一覧の例：")
36 sdata = e.get_stats_data(
37     app_id, SID, cdArea="00000", cdTab="1",
38     lvTime="4", cdTimeFrom="2026000101", lvCat01="1",
39 )
40 df = e.stats_data_to_frame(sdata)
41 for name in df["cat01"].drop_duplicates().head(12):
42     print(f"    {name}")
43
44
45 if __name__ == "__main__":
46     main()

```

実行結果（抜粋）

```

lvCat01（品目の階層レベル）を変えたときに取得される品目数：
lvCat01=1      → 品目   46 種類
lvCat01=1-2    → 品目   60 種類
lvCat01=1-3    → 品目  113 種類
lvCat01=1-4    → 品目  162 種類

```

22.5 例題 11：ページネーション（NEXT_KEY）

この例題のポイント

使う統計：消費者物価指数（巨大表）／ ID:0003427113 ／ パラメータ：limit, startPosition, NEXT_KEY

1300 万件超の表に対し、limit=1000 で取得しながら NEXT_KEY を startPosition に渡してページを送る仕組みを確認する（デモのため 3 ページで打ち切り）。

scripts/ex11_pagination.py

NEXT_KEY でページ送り

```

1 # /// script
2 # requires-python = ">=3.11"
3 # dependencies = ["requests"]
4 # ///
5 """例題11: ページネーション（NEXT_KEY と startPosition）。

```

```

5
6
7 1回のリクエストで返る件数には上限がある（既定10万件）。上限を超える表は、
8 応答の RESULT_INF.NEXT_KEY を次の startPosition に渡して繰り返し取得する。
9 ここでは巨大な表（消費者物価指数）に対し、limit=1000 で3ページだけ取得して
10 ページ送りの仕組みを確認する。
11
12 実行：
13     set -a; source .env; set +a
14     uv run scripts/ex11_pagination.py
15
16 """
17 import os
18 import sys
19
20 import requests
21
22 URL = "https://api.e-stat.go.jp/rest/3.0/app/json/getStatsData"
23 SID = "0003427113" # 消費者物価指数（2020年基準、非常に大きい表）
24
25 def main() -> None:
26     app_id = os.environ.get("ESTAT_APPID")
27     if not app_id:
28         sys.exit("環境変数 ESTAT_APPID が未設定です。'.env' を設定してください。")
29
30     start = 1
31     page = 0
32     collected = 0
33     while True:
34         params = {"appId": app_id, "statsDataId": SID, "limit": 1000, "startPosition": start}
35         body = requests.get(URL, params=params, timeout=120).json()["GET_STATS_DATA"]
36         if body["RESULT"]["STATUS"] != 0:
37             sys.exit(f"APIエラー: {body['RESULT']['ERROR_MSG']}")
38
39         info = body["STATISTICAL_DATA"]["RESULT_INF"]
40         values = body["STATISTICAL_DATA"]["DATA_INF"]["VALUE"]
41         collected += len(values)
42         page += 1
43         print(
44             f"ページ{page}: TOTAL={int(info['TOTAL_NUMBER']):,} 件中 "
45             f"{info['FROM_NUMBER']}～{info['TO_NUMBER']} を取得 "
46             f"(NEXT_KEY={info.get('NEXT_KEY')})"
47         )
48
49         next_key = info.get("NEXT_KEY")
50         if not next_key or page >= 3: # NEXT_KEY が無ければ完了。ここではデモのため3ページで打ち切り
51             break
52         start = next_key
53
54     print(f"\nこのデモでは {collected:,} 件を取得した（全件取得するには打ち切りを外す）。")
55
56
57 if __name__ == "__main__":
58     main()

```

実行結果（抜粋）

ページ1: TOTAL=13,496,127 件中 1～1000 を取得 (NEXT_KEY=1001)
ページ2: TOTAL=13,496,127 件中 1001～2000 を取得 (NEXT_KEY=2001)
ページ3: TOTAL=13,496,127 件中 2001～3000 を取得 (NEXT_KEY=3001)

22.6 例題 12：データとメタを別々に取得して結合する

この例題のポイント

使う統計：消費者物価指数／ ID：0003427113 ／ パラメータ：metaGetFlg=N + getMetaInfo

メタ情報は 1 回だけ取得して辞書化し、データは metaGetFlg=N で軽量に取得してあとから結合する。estat_api の内部処理を分解して理解する。

scripts/ex12_join_meta.py

メタを再利用して結合

```
1 # /// script
2 # requires-python = ">=3.11"
3 # dependencies = ["requests", "pandas"]
4 # ///
5 """例題12: データとメタ情報を別々に取得して結合する (metaGetFlg=N)。
```

```

36 raw = pd.DataFrame(e.as_list(sdata["DATA_INF"]["VALUE"]))
37 print("\nメタ抜きの生データ（コードのみ）の先頭：")
38 print(raw.head(3).to_string(index=False))
39
40 # (3) 自前で結合：@time コードを (1) の辞書で名称に変換
41 raw["時点"] = raw["@time"].map(code2name["time"])
42 raw["品目"] = raw["@cat01"].map(code2name["cat01"])
43 raw["値"] = pd.to_numeric(raw["$"], errors="coerce")
44 print("\nコードを名称に結合した結果（直近5件）：")
45 print(raw.sort_values("@time")[["時点", "品目", "値"]].tail(5).to_string(index=False))
46
47
48 if __name__ == "__main__":
49     main()

```

実行結果（抜粋）

メタ情報の次元: ['tab', 'cat01', 'area', 'time']
コードを名称に結合した結果（直近5件）：

	時点	品目	値
2026年1月	0001	総合	112.9
2026年2月	0001	総合	112.2
2026年3月	0001	総合	112.7
2026年4月	0001	総合	113.0
2026年5月	0001	総合	113.5

22.7 例題 13：CSV と JSON を比べる

この例題のポイント

使う統計：消費者物価指数／ ID：0003427113 ／ API：getSimpleStatsData（CSV）と
getStatsData（JSON）

同じ条件を CSV 簡易版と JSON の両方で取得し、数値が一致することを確認する。CSV は pandas.read_csv に直接渡せて手軽である。

scripts/ex13_csv_vs_json.py

CSV と JSON の取得を比較

```

1 # /// script
2 # requires-python = ">=3.11"
3 # dependencies = ["requests", "pandas"]
4 # ///
5 """例題13: CSV(getSimpleStatsData) と JSON の取得を比べる。
6
7 同じ条件（消費者物価指数 総合・全国・月次）を、CSV簡易版とJSONの両方で取得して、
8 得られる数値が一致することを確認する。CSVは pandas.read_csv に直接渡せて手軽。
9
10 実行:
11     set -a; source .env; set +a
12     uv run scripts/ex13_csv_vs_json.py
13 """
14 import io

```

```

15
16 import pandas as pd
17 import requests
18
19 import estat_api as e
20
21 CSV_URL = "https://api.e-stat.go.jp/rest/3.0/app/getSimpleStatsData"
22 SID = "0003427113"
23 COND = dict(cdCat01="0001", cdArea="00000", cdTab="1", lvTime="4", cdTimeFrom="2025000101")
24
25
26 def main() -> None:
27     app_id = e.get_app_id()
28
29     # --- CSV (簡易版)。sectionHeaderFlg=2 で説明ヘッダ行を省く ---
30     params = {"appId": app_id, "statsDataId": SID, "sectionHeaderFlg": "2", **COND}
31     text = requests.get(CSV_URL, params=params, timeout=120).text
32     csv_df = pd.read_csv(io.StringIO(text)).sort_values("time_code")
33     print("■ CSV(getSimpleStatsData) の列:", list(csv_df.columns))
34     print(csv_df[["time_code", "時間軸 (年・月)", "value"]].tail(3).to_string(index=False))
35
36     # --- JSON (estat_api 経由) ---
37     sdata = e.get_stats_data(app_id, SID, **COND)
38     json_df = e.stats_data_to_frame(sdata).sort_values("time_code")
39     print("\n■ JSON の直近3件:")
40     print(json_df[["time", "値"]].tail(3).to_string(index=False))
41
42     # --- 値の一致を確認 (同じ最新月どうし) ---
43     csv_last = float(csv_df.iloc[-1]["value"])
44     json_last = float(json_df.iloc[-1]["値"])
45     print(f"\n最新値の一致確認: CSV={csv_last} / JSON={json_last} → "
46           f"'一致' if csv_last == json_last else '不一致'")
47
48
49 if __name__ == "__main__":
50     main()

```

実行結果 (抜粋)

```

■ CSV(getSimpleStatsData) の列: ['tab_code', '表章項目', 'cat01_code', ..., 'value', 'annotation']
■ JSON の直近3件: 2026年5月 113.5
最新値の一致確認: CSV=113.5 / JSON=113.5 → 一致

```

23 グループ C：実データの取得と分析・可視化

23.1 例題 14：人口ピラミッド

この例題のポイント

使う統計：人口推計（年齢 5 歳階級・男女別）／ ID：0003443840

男女・年齢 5 歳階級別の人口を取得し、左右対称の人口ピラミッドを描く。「総数」や「(再掲)」の区分は二重計上になるため除外するのが要点。

scripts/ex14_population_pyramid.py

人口ピラミッドを描く

```
1 # /// script
2 # requires-python = ">=3.11"
3 # dependencies = ["requests", "pandas", "matplotlib"]
4 # ///
5 """例題14: 国勢調査ベースの人口推計から「人口ピラミッド」を描く。
6
7 人口推計（令和2年国勢調査基準，統計表ID 0003443840）から、男女・年齢5歳階級別の
8 総人口を取得し、左右対称の人口ピラミッド（横棒グラフ）を作図する。
9
10 実行:
11     set -a; source .env; set +a
12     uv run scripts/ex14_population_pyramid.py
13 """
14 import os
15
16 import matplotlib
17 matplotlib.use("Agg")
18 import matplotlib.pyplot as plt
19 import jpfont # noqa: F401 matplotlib の日本語フォントを設定 (jpfont.py)
20
21 import estat_api as e
22
23 SID = "0003443840" # 人口推計 年齢 (5歳階級), 男女別人口
24
25
26 def main() -> None:
27     app_id = e.get_app_id()
28
29     sdata = e.get_stats_data(
30         app_id, SID,
31         cdTab="001",          # 人口
32         cdCat01="01",        # 確定値
33         cdCat02="001,002",   # 男女別: 001=男, 002=女
34         cdCat04="001",       # 総人口
35     )
36     df = e.stats_data_to_frame(sdata)
37
```

```

38 # 最新時点・5歳階級だけに絞る（「総数」や「(再掲) …」は二重計上になるので除外)
39 df = df[df["time_code"] == df["time_code"].max()]
40 df = df[~df["cat03"].str.contains("総数|再掲")].sort_values("cat03_code")
41 period = df["time"].iloc[0]
42
43 men = df[df["cat02"] == "男"]
44 women = df[df["cat02"] == "女"]
45 ages = men["cat03"].tolist()
46 y = range(len(ages))
47
48 fig, ax = plt.subplots(figsize=(7, 6))
49 ax.barh(y, -men["値"].to_numpy(), color="#4C72B0", label="男")
50 ax.barh(y, women["値"].to_numpy(), color="#C44E52", label="女")
51 ax.set_yticks(list(y))
52 ax.set_yticklabels(ages, fontsize=7)
53 ax.set_xlabel("人口 (千人) ")
54 ax.set_title(f"日本の人口ピラミッド ({period}) ")
55 # x軸の負値を絶対値表示に（男側の符号を隠す）
56 ax.xaxis.set_major_formatter(plt.FuncFormatter(lambda x, _ : f"{abs(int(x)):,}"))
57 ax.legend()
58 ax.grid(axis="x", alpha=0.3)
59
60 os.makedirs("figures", exist_ok=True)
61 out = "figures/ex14_population_pyramid.pdf"
62 fig.savefig(out, bbox_inches="tight")
63 total = df["値"].sum()
64 print(f"{period} 時点の総人口（男女計）： {total:,.0f} 千人")
65 print(f"→ 人口ピラミッドを {out} に保存しました。")
66
67
68 if __name__ == "__main__":
69     main()

```

実行結果（抜粋）

2026年1月 時点の総人口（男女計）： 122,982 千人
→ 人口ピラミッドを figures/ex14_population_pyramid.pdf に保存しました。

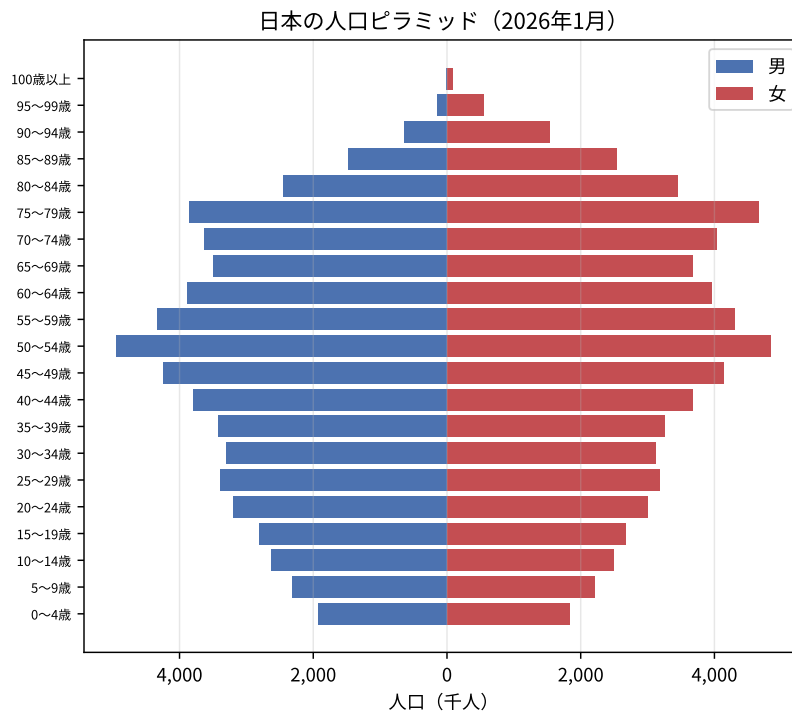


図2 例題14の出力：日本の人口ピラミッド

23.2 例題15：都道府県別人口ランキング

この例題のポイント

使う統計：人口推計（都道府県別人口）／ ID：0003448232

最新年の都道府県別総人口を取得し、上位15都道府県を横棒グラフにする。

scripts/ex15_prefecture_ranking.py

都道府県ランキングを描く

```

1 # /// script
2 # requires-python = ">=3.11"
3 # dependencies = ["requests", "pandas", "matplotlib"]
4 # ///
5 """例題15: 都道府県別人口のランキングを棒グラフにする。
6
7 人口推計（統計表ID 0003448232）から最新年の都道府県別総人口を取得し、
8 上位15都道府県を横棒グラフで表示する。
9
10 実行:
11     set -a; source .env; set +a
12     uv run scripts/ex15_prefecture_ranking.py
13 """
14 import os
15
16 import matplotlib

```

```

17 matplotlib.use("Agg")
18 import matplotlib.pyplot as plt
19 import jpfont # noqa: F401 matplotlib の日本語フォントを設定 (jpfont.py)
20
21 import estat_api as e
22
23 SID = "0003448232" # 人口推計 都道府県, 男女別人口
24
25
26 def main() -> None:
27     app_id = e.get_app_id()
28
29     sdata = e.get_stats_data(app_id, SID,
30                             cdTab="001", cdCat01="000", cdCat02="001", lvArea="2")
31     df = e.stats_data_to_frame(sdata)
32     df = df[df["time_code"] == df["time_code"].max()]
33     period = df["time"].iloc[0]
34
35     top = df.sort_values("値", ascending=False).head(15).iloc[::-1]
36
37     fig, ax = plt.subplots(figsize=(7, 6))
38     ax.barh(top["area"], top["値"], color="#55A868")
39     ax.set_xlabel("人口 (千人) ")
40     ax.set_title(f"都道府県別人口 上位15 ({period}) ")
41     for yi, v in enumerate(top["値"]):
42         ax.text(v, yi, f" {v:,.0f}", va="center", fontsize=8)
43     ax.grid(axis="x", alpha=0.3)
44
45     os.makedirs("figures", exist_ok=True)
46     out = "figures/ex15_prefecture_ranking.pdf"
47     fig.savefig(out, bbox_inches="tight")
48     print(f"対象年: {period}")
49     print(top.sort_values("値", ascending=False)[["area", "値"]].head(5).to_string(index=False))
50     print(f"→ ランキング図を {out} に保存しました。")
51
52
53 if __name__ == "__main__":
54     main()

```

実行結果 (抜粋)

対象年: 2024年

area	値
東京都	14178
神奈川県	9225
大阪府	8757

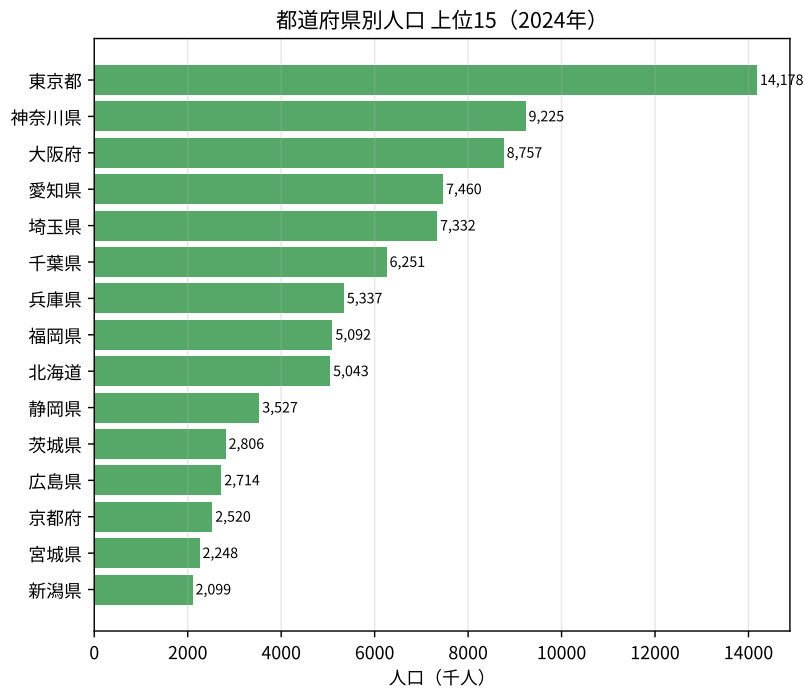


図 3 例題 15 の出力：都道府県別人口 上位 15

23.3 例題 16：出生数と合計特殊出生率

この例題のポイント

使う統計：人口動態調査（確定数・出生）／ ID：0003411595

出生数（棒）と合計特殊出生率（折れ線）を二軸で重ねて、少子化の推移を可視化する。

scripts/ex16_births_trend.py

出生数と出生率を描く

```

1 # /// script
2 # requires-python = ">=3.11"
3 # dependencies = ["requests", "pandas", "matplotlib"]
4 # ///
5 """例題16: 出生数と合計特殊出生率の年次推移を描く。
6
7 人口動態調査（確定数・出生，統計表ID 0003411595）から、出生数と合計特殊出生率を
8 取得し、棒グラフ（出生数）と折れ線（合計特殊出生率）を二軸で重ねて作図する。
9
10 実行:
11     set -a; source .env; set +a
12     uv run scripts/ex16_births_trend.py
13 """
14 import os
15
16 import matplotlib
17 matplotlib.use("Agg")

```

```

18 import matplotlib.pyplot as plt
19 import jpfont # noqa: F401 matplotlib の日本語フォントを設定 (jpfont.py)
20
21 import estat_api as e
22
23 SID = "0003411595" # 人口動態調査 出生（年次別）
24
25
26 def main() -> None:
27     app_id = e.get_app_id()
28
29     sdata = e.get_stats_data(app_id, SID, cdCat01="00100,00150",
30                             cdTimeFrom="1980000000") # 1980年以降
31     df = e.stats_data_to_frame(sdata)
32     df["年"] = df["time"].str.replace("年", "").astype(int)
33
34     births = df[df["cat01"] == "出生数_総数"].sort_values("年")
35     tfr = df[df["cat01"] == "合計特殊出生率"].sort_values("年")
36
37     fig, ax1 = plt.subplots(figsize=(8, 4.5))
38     ax1.bar(births["年"], births["値"] / 10000, color="#8C9EC4", label="出生数")
39     ax1.set_ylabel("出生数（万人）")
40     ax1.set_xlabel("年")
41
42     ax2 = ax1.twinx()
43     ax2.plot(tfr["年"], tfr["値"], color="#C44E52", marker="o", ms=3, label="合計特殊出生率")
44     ax2.set_ylabel("合計特殊出生率")
45
46     ax1.set_title("出生数と合計特殊出生率の推移")
47     fig.legend(loc="upper right", bbox_to_anchor=(0.9, 0.88))
48
49     os.makedirs("figures", exist_ok=True)
50     out = "figures/ex16_births_trend.pdf"
51     fig.savefig(out, bbox_inches="tight")
52     latest = births.iloc[-1]
53     print(f"最新 {latest['年']}年: 出生数 {latest['値']:.0f} 人, "
54           f"合計特殊出生率 {tfr.iloc[-1]['値']}")
55     print(f"→ 図を {out} に保存しました。")
56
57
58 if __name__ == "__main__":
59     main()

```

実行結果（抜粋）

最新 2024年: 出生数 686,173 人, 合計特殊出生率 1.15

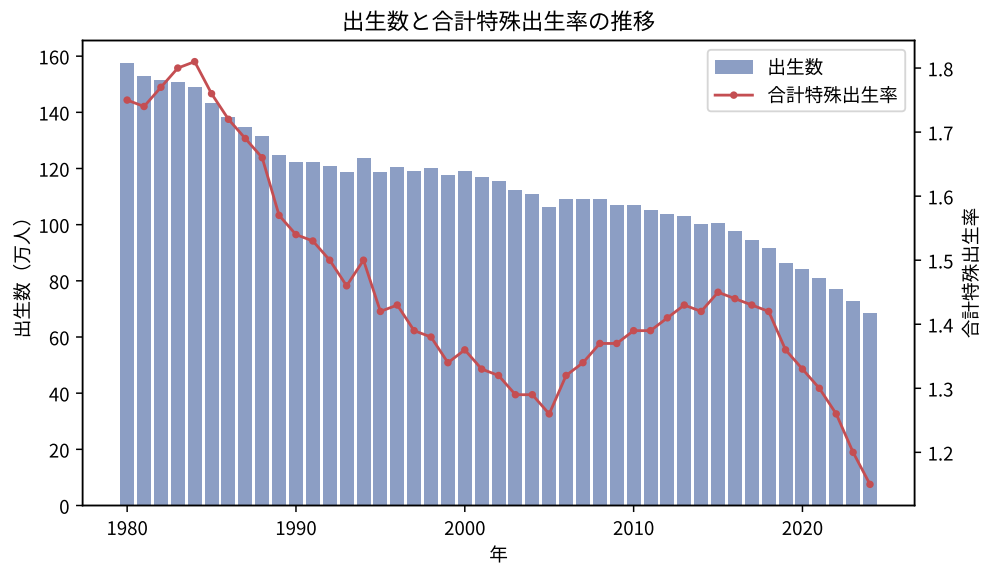


図4 例題16の出力：出生数と合計特殊出生率の推移

23.4 例題17：3大都市圏の転入超過

この例題のポイント

使う統計：住民基本台帳人口移動報告／ID：0003404102

東京圏・名古屋圏・大阪圏の転入超過数の年次推移を比較し、東京一極集中を可視化する。

scripts/ex17_migration.py

転入超過の推移を描く

```

1 # /// script
2 # requires-python = ">=3.11"
3 # dependencies = ["requests", "pandas", "matplotlib"]
4 # ///
5 """例題17: 3大都市圏の転入超過数の推移を描く。
6
7 住民基本台帳人口移動報告（統計表ID 0003404102）から、東京圏・名古屋圏・大阪圏の
8 転入超過数の年次推移を取得し、折れ線グラフで比較する（東京一極集中の可視化）。
9
10 実行:
11     set -a; source .env; set +a
12     uv run scripts/ex17_migration.py
13 """
14 import os
15
16 import matplotlib
17 matplotlib.use("Agg")
18 import matplotlib.pyplot as plt
19 import jpfont # noqa: F401 matplotlib の日本語フォントを設定 (jpfont.py)
20
21 import estat_api as e

```

```

22
23 SID = "0003404102" # 住民基本台帳人口移動報告 3大都市圏
24
25
26 def main() -> None:
27     app_id = e.get_app_id()
28
29     sdata = e.get_stats_data(
30         app_id, SID,
31         cdCat01="002,003,004", # 東京圏・名古屋圏・大阪圏
32         cdCat02="003", # 転入超過数
33     )
34     df = e.stats_data_to_frame(sdata)
35     df["年"] = df["time"].str.extract(r"(\d+)").astype(int)
36
37     fig, ax = plt.subplots(figsize=(8, 4.5))
38     for area, sub in df.groupby("cat01"):
39         sub = sub.sort_values("年")
40         ax.plot(sub["年"], sub["値"] / 10000, marker="", label=area)
41     ax.axhline(0, color="gray", lw=0.8)
42     ax.set_xlabel("年")
43     ax.set_ylabel("転入超過数 (万人)")
44     ax.set_title("3大都市圏の転入超過数の推移")
45     ax.legend()
46     ax.grid(alpha=0.3)
47
48     os.makedirs("figures", exist_ok=True)
49     out = "figures/ex17_migration.pdf"
50     fig.savefig(out, bbox_inches="tight")
51     tokyo = df[df["cat01"] == "東京圏"].sort_values("年").iloc[-1]
52     print(f"最新 {tokyo['年']}年 東京圏の転入超過数: {tokyo['値']:, .0f} 人")
53     print(f"→ 図を {out} に保存しました。")
54
55
56 if __name__ == "__main__":
57     main()

```

実行結果 (抜粋)

最新 2019年 東京圏の転入超過数: 145,576 人

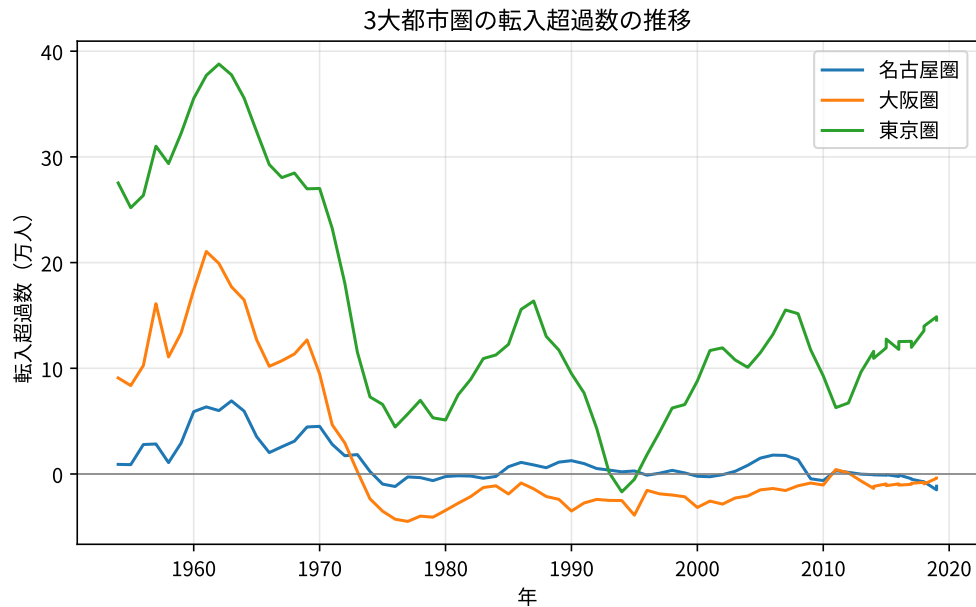


図5 例題17の出力：3大都市圏の転入超過数の推移

23.5 例題18：完全失業率の月次推移

この例題のポイント

使う統計：労働力調査／ID：0003005865

完全失業率（男女総数・原数値）の月次系列を2000年から取得して折れ線にする。

scripts/ex18_unemployment.py

完全失業率を描く

```
1 # /// script
2 # requires-python = ">=3.11"
3 # dependencies = ["requests", "pandas", "matplotlib"]
4 # ///
5 """例題18: 完全失業率の月次推移を描く。
6
7 労働力調査（統計表ID 0003005865）から、完全失業率（男女総数）の月次系列を
8 取得し、折れ線グラフにする（原数値）。
9
10 実行:
11     set -a; source .env; set +a
12     uv run scripts/ex18_unemployment.py
13 """
14 import os
15
16 import matplotlib
17 matplotlib.use("Agg")
18 import matplotlib.pyplot as plt
19 import jpfont # noqa: F401 matplotlib の日本語フォントを設定 (jpfont.py)
```

```

20 import pandas as pd
21
22 import estat_api as e
23
24 SID = "0003005865"
25
26
27 def main() -> None:
28     app_id = e.get_app_id()
29
30     sdata = e.get_stats_data(
31         app_id, SID,
32         cdTab="02",      # 率(%)
33         cdCat01="000",   # 全産業
34         cdCat02="08",    # 就業状態: 完全失業者 → 完全失業率
35         cdCat03="0",     # 性別: 総数
36     )
37     df = e.stats_data_to_frame(sdata).sort_values("time_code")
38     # 月次コード YYYY00MMM から年月を作る
39     df["年月"] = pd.to_datetime(
40         df["time_code"].str[:4] + "-" + df["time_code"].str[-2:], format="%Y-%m", errors="coerce"
41     )
42     df = df.dropna(subset=["年月"])
43
44     fig, ax = plt.subplots(figsize=(8, 4))
45     ax.plot(df["年月"], df["値"], color="#4C72B0", lw=1)
46     ax.set_ylabel("完全失業率 (%) ")
47     ax.set_title("完全失業率の推移 (月次・原数値) ")
48     ax.grid(alpha=0.3)
49
50     os.makedirs("figures", exist_ok=True)
51     out = "figures/ex18_unemployment.pdf"
52     fig.savefig(out, bbox_inches="tight")
53     last = df.iloc[-1]
54     print(f"取得期間: {df['年月'].min():%Y-%m} ～ {df['年月'].max():%Y-%m} ({len(df)}か月) ")
55     print(f"直近の完全失業率: {last['値']}% ({last['年月']:%Y年%m月}) ")
56     print(f"→ 図を {out} に保存しました。")
57
58
59 if __name__ == "__main__":
60     main()

```

実行結果 (抜粋)

取得期間: 2000-01 ～ 2026-04 (316か月)
直近の完全失業率: 2.7% (2026年04月)



図 6 例題 18 の出力：完全失業率の推移

23.6 例題 19：現金給与総額の推移

この例題のポイント

使う統計：毎月勤労統計調査（実数原表）／ ID：0003138108

現金給与総額（5 人以上・調査産業計）の水準と前年同月比を描く。なお、この原表は旧産業分類のため 2010 年 1 月～2015 年 11 月が対象である。統計系列が分類改定で分割される実例として扱う。

scripts/ex19_cash_earnings.py

現金給与総額を描く

```
1 # /// script
2 # requires-python = ">=3.11"
3 # dependencies = ["requests", "pandas", "matplotlib"]
4 # ///
5 """例題19: 毎月勤労統計から現金給与総額の推移を描く。
6
7 毎月勤労統計調査（全国調査・実数原表、統計表ID 0003138108）から、
8 事業所規模5人以上・調査産業計・就業形態計の現金給与総額（月次）を取得し、
9 水準と前年同月比を作図する。
10 ※この原表は旧産業分類（2007年改定）に基づくため2010年1月～2015年11月をカバーする。
11 産業分類の改定で系列が分割される実例。最新値は新分類の別表を参照する。
12
13 実行:
14     set -a; source .env; set +a
15     uv run scripts/ex19_cash_earnings.py
16 """
17 import os
18
19 import matplotlib
```

```

20 matplotlib.use("Agg")
21 import matplotlib.pyplot as plt
22 import jpfont # noqa: F401 matplotlib の日本語フォントを設定 (jpfont.py)
23 import pandas as pd
24
25 import estat_api as e
26
27 SID = "0003138108"
28
29
30 def main() -> None:
31     app_id = e.get_app_id()
32
33     sdata = e.get_stats_data(
34         app_id, SID,
35         cdTab="741",      # 現金給与総額 (円)
36         cdCat01="001",    # 性別: 計
37         cdCat02="00",     # 就業形態: 計
38         cdCat03="T",      # 事業所規模: 5人以上
39         cdCat04="TL",     # 産業: 調査産業計
40     )
41     df = e.stats_data_to_frame(sdata).sort_values("time_code")
42     df["年月"] = pd.to_datetime(
43         df["time_code"].str[:4] + "-" + df["time_code"].str[-2:], format="%Y-%m", errors="coerce"
44     )
45     df = df.dropna(subset=["年月"]).reset_index(drop=True)
46     df["前年同月比"] = df["値"].pct_change(12) * 100
47
48     fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(8, 6), sharex=True)
49     ax1.plot(df["年月"], df["値"] / 10000, color="#4C72B0", lw=1)
50     ax1.set_ylabel("現金給与総額 (万円) ")
51     ax1.set_title("現金給与総額 (5人以上・調査産業計) ")
52     ax1.grid(alpha=0.3)
53     ax2.plot(df["年月"], df["前年同月比"], color="#C44E52", lw=1)
54     ax2.axhline(0, color="gray", lw=0.8)
55     ax2.set_ylabel("前年同月比 (%) ")
56     ax2.grid(alpha=0.3)
57
58     os.makedirs("figures", exist_ok=True)
59     out = "figures/ex19_cash_earnings.pdf"
60     fig.savefig(out, bbox_inches="tight")
61     last = df.iloc[-1]
62     print(f"直近 {last['年月']:%Y年%m月}: 現金給与総額 {last['値']:, .0f} 円, "
63           f"前年同月比 {last['前年同月比']:+.1f}%")
64     print(f"→ 図を {out} に保存しました。")
65
66
67 if __name__ == "__main__":
68     main()

```

実行結果 (抜粋)

直近 2015年11月: 現金給与総額 274,414 円, 前年同月比 -1.0%

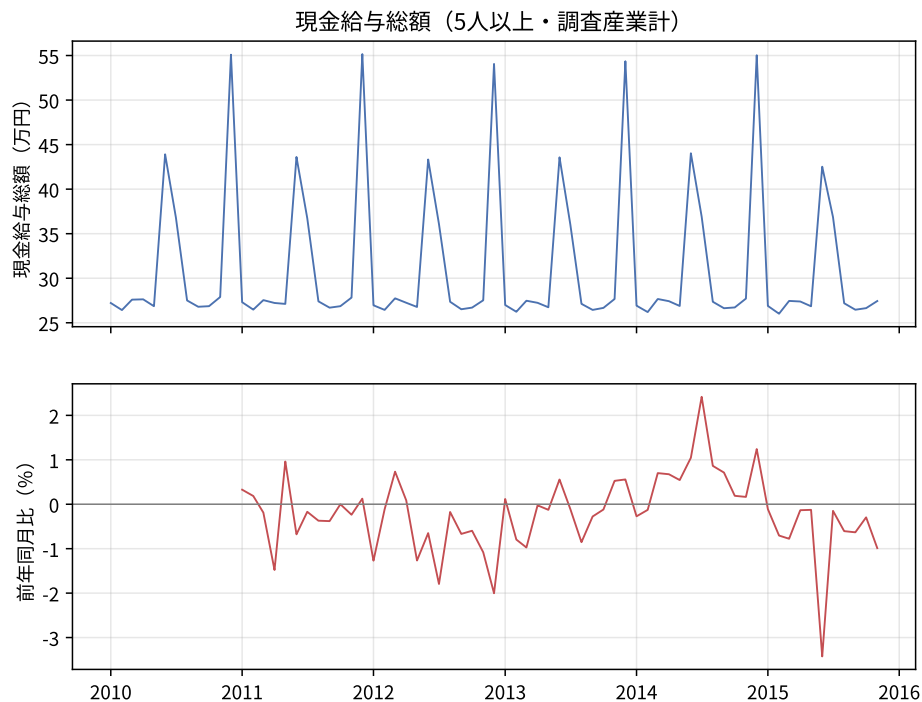


図 7 例題 19 の出力：現金給与総額と前年同月比

23.7 例題 20：年齢階級別の賃金プロファイル

この例題のポイント

使う統計：賃金構造基本統計調査／ ID：0003029886

男女別・年齢階級別の「きまって支給する現金給与額」を取得し、賃金カーブを描く。cdCat05（産業計）・cdCat06（民・公計）まで指定しないと多重に取得される点に注意。

scripts/ex20_wage_by_age.py

年齢別賃金を描く

```
1 # /// script
2 # requires-python = ">=3.11"
3 # dependencies = ["requests", "pandas", "matplotlib"]
4 # ///
5 """例題20: 賃金構造基本統計調査から年齢階級別の賃金プロファイルを描く。
6
7 統計表ID 0003029886（一般労働者・産業計）から、男女別・年齢階級別の
8 「きまって支給する現金給与額」を取得し、年齢に対する賃金カーブを作図する。
9
10 実行:
11     set -a; source .env; set +a
12     uv run scripts/ex20_wage_by_age.py
13 """
14 import os
15
```

```

16 import matplotlib
17 matplotlib.use("Agg")
18 import matplotlib.pyplot as plt
19 import jpfont # noqa: F401 matplotlib の日本語フォントを設定 (jpfont.py)
20
21 import estat_api as e
22
23 SID = "0003029886" # 賃金構造基本統計調査 年齢階級別
24
25
26 def main() -> None:
27     app_id = e.get_app_id()
28
29     sdata = e.get_stats_data(
30         app_id, SID,
31         cdTab="40", # きまって支給する現金給与額 (千円)
32         cdCat01="01", # 企業規模計 (10人以上)
33         cdCat02="02,03", # 性別: 男, 女
34         cdCat03="01", # 学歴計
35         cdCat05="01", # 産業分類: 産業計
36         cdCat06="01", # 民・公区分: 民営+公営
37     )
38     df = e.stats_data_to_frame(sdata)
39     if "time_code" in df: # 年次があれば最新年に
40         df = df[df["time_code"] == df["time_code"].max()]
41     df = df[df["cat04"] != "年齢計"].sort_values("cat04_code")
42
43     fig, ax = plt.subplots(figsize=(8, 4.5))
44     for sex, sub in df.groupby("cat02"):
45         ax.plot(sub["cat04"], sub["値"], marker="o", ms=3, label=sex)
46     ax.set_ylabel("きまって支給する現金給与額 (千円) ")
47     ax.set_title("年齢階級別の賃金プロファイル (一般労働者・産業計) ")
48     ax.tick_params(axis="x", labelrotation=60, labelsz=7)
49     ax.legend()
50     ax.grid(alpha=0.3)
51
52     os.makedirs("figures", exist_ok=True)
53     out = "figures/ex20_wage_by_age.pdf"
54     fig.savefig(out, bbox_inches="tight")
55     peak = df.loc[df["値"].idxmax()]
56     print(f"最も賃金が高い区分: {peak['cat02']}・{peak['cat04']} = {peak['値']} 千円")
57     print(f"→ 図を {out} に保存しました。")
58
59
60 if __name__ == "__main__":
61     main()

```

実行結果 (抜粋)

最も賃金が高い区分: 男・50～54歳 = 437.2 千円

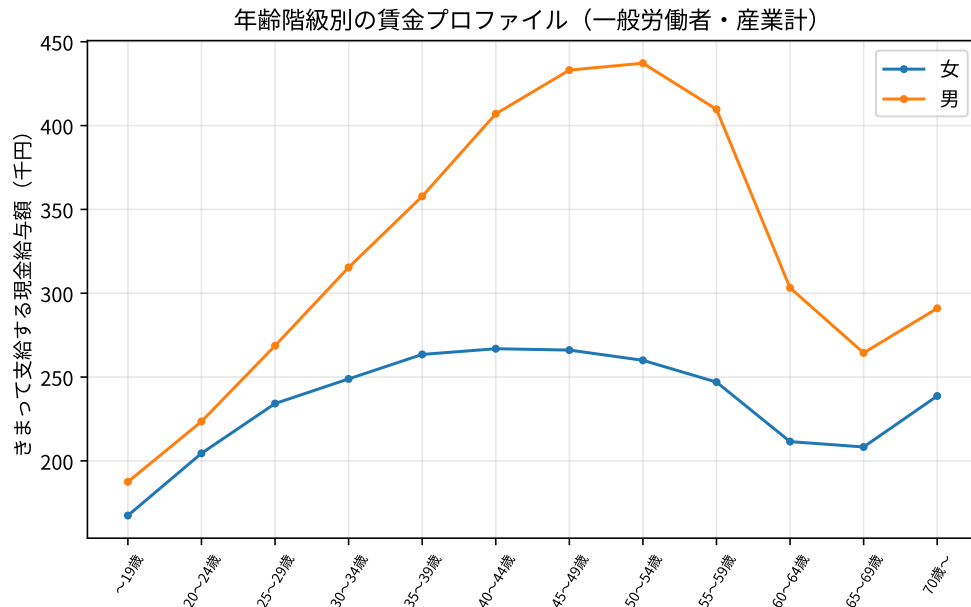


図 8 例題 20 の出力：年齢階級別の賃金プロファイル

23.8 例題 21：消費者物価指数の前年同月比（インフレ率）

この例題のポイント

使う統計：消費者物価指数（2020 年基準）／ ID：0003427113 ／ パラメータ：cdTab=3（前年同月比）

総合と「生鮮食品を除く総合（コア）」の前年同月比を比較する。cdTab=3 が前年同月比。

scripts/ex21_cpi_inflation.py

インフレ率を描く

```
# /// script
# requires-python = ">=3.11"
# dependencies = ["requests", "pandas", "matplotlib"]
# ///
"""例題21: 消費者物価指数の前年同月比（インフレ率）を描く。

消費者物価指数（2020年基準，統計表ID 0003427113）の「前年同月比」を、
総合と「生鮮食品を除く総合（コア）」の2系列について取得し、折れ線で比較する。

実行:
    set -a; source .env; set +a
    uv run scripts/ex21_cpi_inflation.py
"""
import os

import matplotlib
matplotlib.use("Agg")
```

```

18 import matplotlib.pyplot as plt
19 import jpfont # noqa: F401 matplotlib の日本語フォントを設定 (jpfont.py)
20 import pandas as pd
21
22 import estat_api as e
23
24 SID = "0003427113"
25
26
27 def main() -> None:
28     app_id = e.get_app_id()
29
30     sdata = e.get_stats_data(
31         app_id, SID,
32         cdTab="3",          # 前年同月比
33         cdCat01="0001,0161", # 総合 / 生鮮食品を除く総合
34         cdArea="00000",     # 全国
35         lvTime="4",         # 月次
36         cdTimeFrom="2021000101",
37     )
38     df = e.stats_data_to_frame(sdata).sort_values("time_code")
39     df["年月"] = pd.to_datetime(
40         df["time_code"].str[:4] + "-" + df["time_code"].str[8:10], format="%Y-%m", errors="coerce"
41     )
42
43     fig, ax = plt.subplots(figsize=(8, 4.5))
44     for item, sub in df.groupby("cat01"):
45         ax.plot(sub["年月"], sub["値"], marker="", label=item)
46     ax.axhline(0, color="gray", lw=0.8)
47     ax.axhline(2, color="orange", lw=0.8, ls="--", label="2%目標")
48     ax.set_ylabel("前年同月比 (%) ")
49     ax.set_title("消費者物価指数 前年同月比 (全国) ")
50     ax.legend()
51     ax.grid(alpha=0.3)
52
53     os.makedirs("figures", exist_ok=True)
54     out = "figures/ex21_cpi_inflation.pdf"
55     fig.savefig(out, bbox_inches="tight")
56     latest = df.sort_values("年月").groupby("cat01").tail(1)
57     print("直近の前年同月比：")
58     print(latest[["年月", "cat01", "値"]].to_string(index=False))
59     print(f"→ 図を {out} に保存しました。")
60
61
62 if __name__ == "__main__":
63     main()

```

実行結果 (抜粋)

直近の前年同月比：

2026-05-01 0161 生鮮食品を除く総合 1.4

2026-05-01 0001 総合 1.5

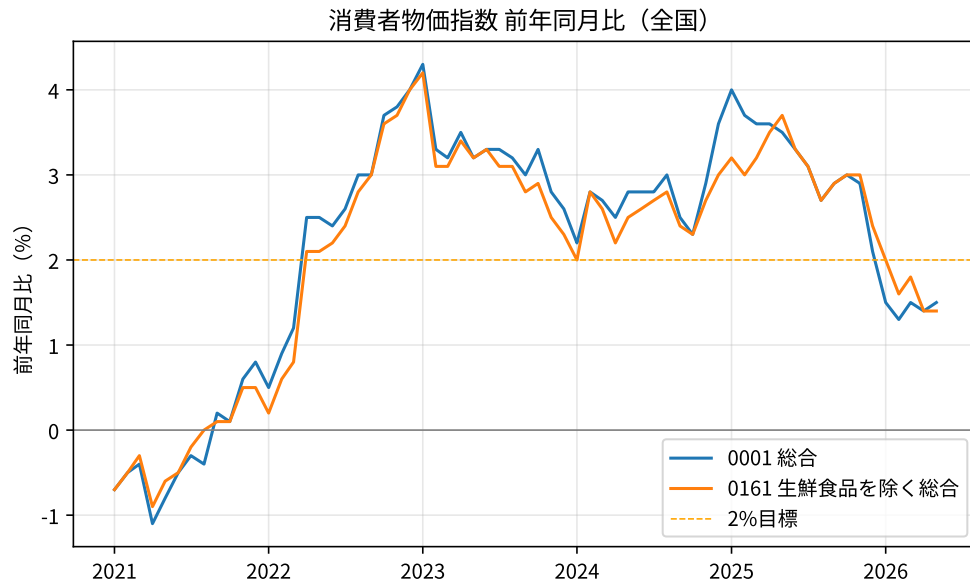


図9 例題 21 の出力：消費者物価指数 前年同月比

23.9 例題 22：費目別消費支出の構成

この例題のポイント

使う統計：家計調査（品目分類 2020 年改定）／ ID：0003348239

二人以上の世帯の 10 大費目別消費支出を取得し、構成比を円グラフにする。cdCat02（世帯区分）・cdArea（全国）の指定を忘れると多重集計になる。

scripts/ex22_household_spending.py

費目別支出を描く

```
1 # /// script
2 # requires-python = ">=3.11"
3 # dependencies = ["requests", "pandas", "matplotlib"]
4 # ///
5 """例題22: 家計調査から費目別消費支出の構成を描く。
6
7 家計調査（家計収支編・二人以上の世帯、品目分類2020年改定、統計表ID 0003348239）から、
8 10大費目別の消費支出（最新月）を取得し、構成比を円グラフにする。
9
10 実行:
11     set -a; source .env; set +a
12     uv run scripts/ex22_household_spending.py
13 """
14 import os
15
16 import matplotlib
17 matplotlib.use("Agg")
18 import matplotlib.pyplot as plt
```

```

19 import jpfont # noqa: F401 matplotlib の日本語フォントを設定 (jpfont.py)
20
21 import estat_api as e
22
23 SID = "0003348239"
24 # 10大費目の品目コード (1 食料 ~ 10 その他の消費支出)
25 HIMOKU = ["010000000", "020000000", "030000000", "040000000", "050000000",
26           "060000000", "070000000", "080000000", "090000000", "100000000"]
27
28
29 def main() -> None:
30     app_id = e.get_app_id()
31
32     sdata = e.get_stats_data(
33         app_id, SID,
34         cdCat01=",".join(HIMOKU),
35         cdCat02="03", # 世帯区分: 二人以上の世帯 (2000年~)
36         cdArea="00000", # 全国
37     )
38     df = e.stats_data_to_frame(sdata)
39
40     # 最新年に絞る
41     df = df[df["time_code"] == df["time_code"].max()]
42     period = df["time"].iloc[0]
43     df = df.sort_values("値", ascending=False)
44
45     # 費目名の先頭の番号を取り、見やすいラベルに
46     labels = [name.split(maxsplit=1)[-1] for name in df["cat01"]]
47
48     fig, ax = plt.subplots(figsize=(7, 6))
49     ax.pie(df["値"], labels=labels, autopct="%1.0f%%",
50           pctdistance=0.8, textprops={"fontsize": 8})
51     ax.set_title(f"二人以上の世帯の費目別消費支出の構成 ({period}) ")
52
53     os.makedirs("figures", exist_ok=True)
54     out = "figures/ex22_household_spending.pdf"
55     fig.savefig(out, bbox_inches="tight")
56     print(f"対象: {period} (二人以上の世帯・全国) / 10費目計: {df['値'].sum():.0f} 円")
57     print(df[["cat01", "値"]].to_string(index=False))
58     print(f"→ 円グラフを {out} に保存しました。")
59
60
61 if __name__ == "__main__":
62     main()

```

実行結果 (抜粋)

対象: 2024年 (二人以上の世帯・全国) / 10費目計: 3,602,915 円

1	食料	1079228
10	その他の消費支出	567737
7	交通・通信	500766

二人以上の世帯の費目別消費支出の構成（2024年）

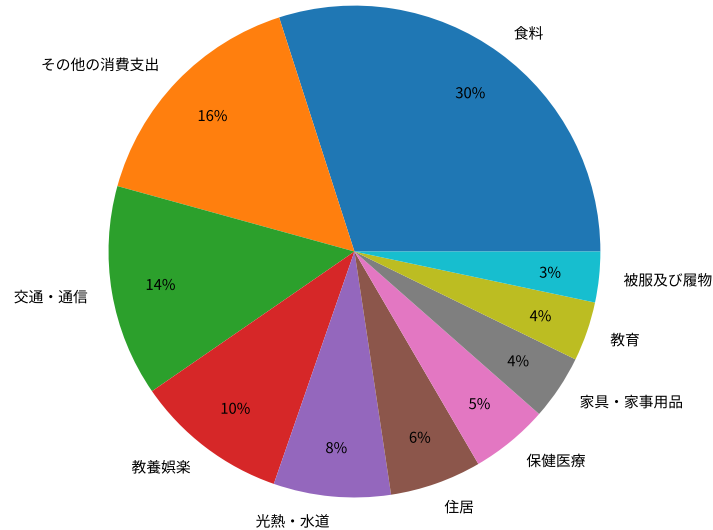


図 10 例題 22 の出力：費目別消費支出の構成

23.10 例題 23：実質 GDP 成長率（年率換算）

この例題のポイント

使う統計：四半期別 GDP 速報（実質季節調整系列）／ ID：0003376098

四半期の実質 GDP（季節調整値）から前期比を年率換算する。 年率 = $((GDP_t / GDP_{t-1})^4 - 1) \times 100$ 。 ※本表は公表時点アーカイブ（過去の値）であり対象期間に注意。

scripts/ex23_gdp_growth.py

実質 GDP 成長率を描く

```

1 # /// script
2 # requires-python = ">=3.11"
3 # dependencies = ["requests", "pandas", "matplotlib"]
4 # ///
5 """例題23: 実質GDP（季節調整系列）から年率換算の成長率を計算する。
6
7 四半期別GDP速報の「実質季節調整系列・実額」（統計表ID 0003376098）から
8 国内総生産（支出側）の四半期系列を取得し、前期比を年率換算して作図する。
9 年率換算成長率 =  $((GDP_t / GDP_{t-1})^4 - 1) \times 100$ 
10 ※この表は公表時点アーカイブ（過去の値）であり、系列の基準年・期間に注意。
11
12 実行:
13     set -a; source .env; set +a
14     uv run scripts/ex23_gdp_growth.py
15 """
16 import os

```

```

17
18 import matplotlib
19 matplotlib.use("Agg")
20 import matplotlib.pyplot as plt
21 import jpfont # noqa: F401
22
23 import estat_api as e
24
25 SID = "0003376098" # 四半期別GDP速報 実質季節調整系列 実額
26
27
28 def main() -> None:
29     app_id = e.get_app_id()
30
31     sdata = e.get_stats_data(app_id, SID, fetch_all=True)
32     df = e.stats_data_to_frame(sdata)
33
34     # 需要項目 (cat01) から「国内総生産(支出側)」の系列だけを取り出す
35     g = df[df["cat01"] == "国内総生産(支出側)"].sort_values("time_code").reset_index(drop=True)
36     g["年率(%)"] = ((g["値"] / g["値"].shift(1)) ** 4 - 1) * 100
37
38     recent = g.dropna().tail(40)
39     fig, ax = plt.subplots(figsize=(9, 4))
40     colors = ["#C44E52" if v < 0 else "#4C72B0" for v in recent["年率(%)"]]
41     ax.bar(range(len(recent)), recent["年率(%)"], color=colors)
42     step = max(1, len(recent) // 10)
43     ax.set_xticks(range(0, len(recent), step))
44     ax.set_xticklabels(recent["time"].iloc[::step], rotation=60, fontsize=7)
45     ax.axhline(0, color="gray", lw=0.8)
46     ax.set_ylabel("実質GDP成長率 (前期比年率, %) ")
47     ax.set_title("実質GDP成長率 (季節調整済・前期比年率) ")
48     ax.grid(axis="y", alpha=0.3)
49
50     os.makedirs("figures", exist_ok=True)
51     out = "figures/ex23_gdp_growth.pdf"
52     fig.savefig(out, bbox_inches="tight")
53     last = g.dropna().iloc[-1]
54     print(f"最新 {last['time']}: 実質GDP {last['値']:, .0f} 10億円, 前期比年率 {last['年率(%)']:+.1f}%")
55     print(f"→ 図を {out} に保存しました。")
56
57
58 if __name__ == "__main__":
59     main()

```

実行結果 (抜粋)

最新 2018年10～12月期: 実質GDP 535,333 10億円, 前期比年率 +1.9%

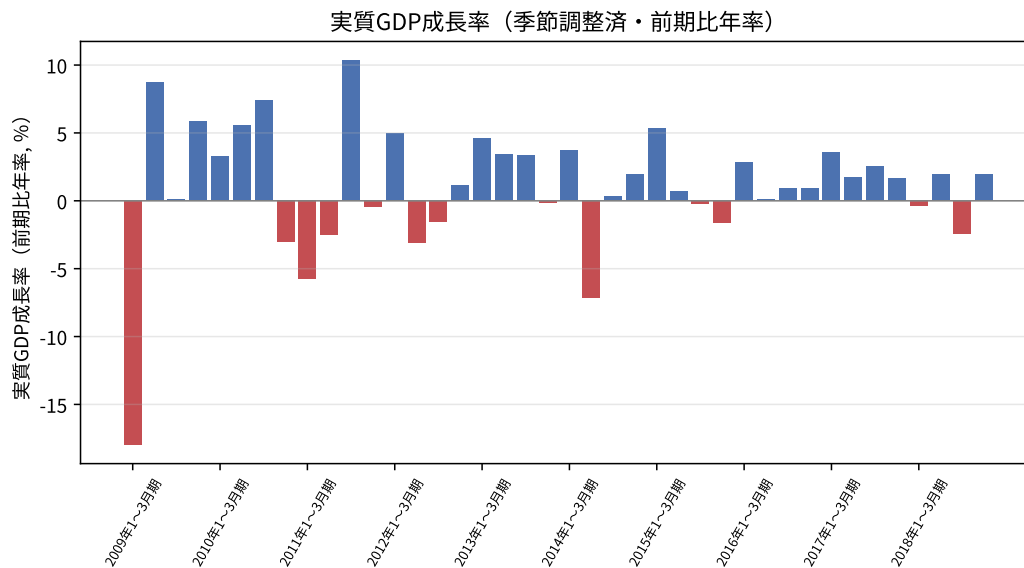


図 11 例題 23 の出力：実質 GDP 成長率（前期比年率）

23.11 例題 24：鉱工業生産指数（IIP）

この例題のポイント

使う統計：鉱工業生産・出荷・在庫指数（2020 年基準）／ ID：0004052177

鉱工業（総合）の季節調整済生産指数を取得して推移を描く。この表は時間軸ラベルが YYYYMM 形式で、ウェイト行などの非日付行を除外して使う。

```
scripts/ex24_iip.py 鉱工業生産指数を描く

1 # /// script
2 # requires-python = ">=3.11"
3 # dependencies = ["requests", "pandas", "matplotlib"]
4 # ///
5 """例題24: 鉱工業生産指数（IIP）の推移を描く。
6
7 鉱工業生産・出荷・在庫指数（2020年基準，統計表ID 0004052177「業種別季節調整済指数・
8 付加価値額生産」）から、鉱工業（総合）の生産指数を取得して折れ線にする。
9
10 実行:
11     set -a; source .env; set +a
12     uv run scripts/ex24_iip.py
13
14 """
15 import os
16
17 import matplotlib
18 matplotlib.use("Agg")
19 import matplotlib.pyplot as plt
20 import jpfont # noqa: F401
```

```

20 import pandas as pd
21
22 import estat_api as e
23
24 SID = "0004052177"
25
26
27 def main() -> None:
28     app_id = e.get_app_id()
29
30     sdata = e.get_stats_data(app_id, SID, cdCat01="0001000", fetch_all=True) # 鉱工業
31     df = e.stats_data_to_frame(sdata)
32
33     # この表の「時間軸」は YYYYMM の月次ラベル（「付加生産ウエイト」等の非日付行を除く）
34     df = df[df["time"].str.match(r"^\d{6}$")].copy()
35     df["年月"] = pd.to_datetime(df["time"], format="%Y%m")
36     df = df.sort_values("年月")
37
38     fig, ax = plt.subplots(figsize=(9, 4))
39     ax.plot(df["年月"], df["値"], color="#4C72B0", lw=1.2)
40     ax.axhline(100, color="gray", lw=0.8, ls="--", label="2020年=100")
41     ax.set_ylabel("生産指数（2020年=100, 季節調整済）")
42     ax.set_title("鉱工業生産指数の推移")
43     ax.legend()
44     ax.grid(alpha=0.3)
45
46     os.makedirs("figures", exist_ok=True)
47     out = "figures/ex24_iip.pdf"
48     fig.savefig(out, bbox_inches="tight")
49     last = df.iloc[-1]
50     print(f"取得期間: {df['年月'].min():%Y-%m} ~ {df['年月'].max():%Y-%m}")
51     print(f"直近 {last['年月']:%Y年%m月} の鉱工業生産指数: {last['値']}")
52     print(f"→ 図を {out} に保存しました。")
53
54
55 if __name__ == "__main__":
56     main()

```

実行結果（抜粋）

取得期間: 2018-01 ~ 2026-03
直近 2026年03月 の鉱工業生産指数: 102.0

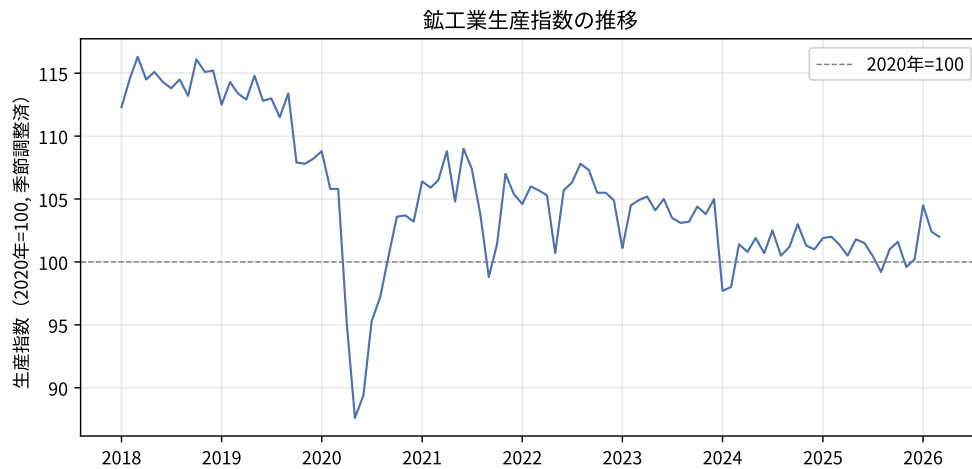


図 12 例題 24 の出力：鉱工業生産指数の推移

23.12 例題 25：業種別商業販売額指数

この例題のポイント

使う統計：商業動態統計調査（確報）／ ID：0004026181

商業計・卸売業・小売業の季節調整済商業販売額指数（月次）を取得して比較する。

scripts/ex25_retail_sales.py

商業販売額指数を描く

```

1 # /// script
2 # requires-python = ">=3.11"
3 # dependencies = ["requests", "pandas", "matplotlib"]
4 # ///
5 """例題25: 商業動態統計の業種別商業販売額指数を描く。
6
7 商業動態統計調査（確報，統計表ID 0004026181「業種別商業販売額指数」）から、
8 商業計・卸売業・小売業の季節調整済指数（月次）を取得して比較する。
9
10 実行:
11     set -a; source .env; set +a
12     uv run scripts/ex25_retail_sales.py
13 """
14 import os
15
16 import matplotlib
17 matplotlib.use("Agg")
18 import matplotlib.pyplot as plt
19 import jpfont # noqa: F401
20 import pandas as pd
21
22 import estat_api as e
23
24 SID = "0004026181"

```

```

25 WANT = ["商業計", "卸売業計", "小売業計"]
26
27
28 def main() -> None:
29     app_id = e.get_app_id()
30
31     sdata = e.get_stats_data(
32         app_id, SID,
33         cdCat02="01080300", # 季節調整済指数
34         fetch_all=True,
35     )
36
37     df = e.stats_data_to_frame(sdata)
38     df = df[df["cat01"].isin(WANT)]
39     # 月次 (YYYY年M月) だけに絞る
40     df = df[df["time"].str.contains(r"\d+\d+\d+", regex=True)].copy()
41     df["年月"] = pd.to_datetime(
42         df["time"].str.replace("年", "-").str.replace("月", ""), format="%Y-%m", errors="coerce"
43     )
44     df = df.dropna(subset=["年月"]).sort_values("年月")
45
46     fig, ax = plt.subplots(figsize=(9, 4))
47     for name, sub in df.groupby("cat01"):
48         ax.plot(sub["年月"], sub["値"], marker="", label=name)
49     ax.set_ylabel("商業販売額指数 (2020年=100, 季節調整済)")
50     ax.set_title("業種別 商業販売額指数の推移")
51     ax.legend()
52     ax.grid(alpha=0.3)
53
54     os.makedirs("figures", exist_ok=True)
55     out = "figures/ex25_retail_sales.pdf"
56     fig.savefig(out, bbox_inches="tight")
57     print(f"取得業種: {sorted(df['cat01'].unique())}")
58     print(f"期間: {df['年月'].min():%Y-%m} ~ {df['年月'].max():%Y-%m}")
59     print(f"→ 図を {out} に保存しました。")
60
61 if __name__ == "__main__":
62     main()

```

実行結果 (抜粋)

取得業種: ['卸売業計', '商業計', '小売業計']
 期間: 2022-11 ~ 2024-01

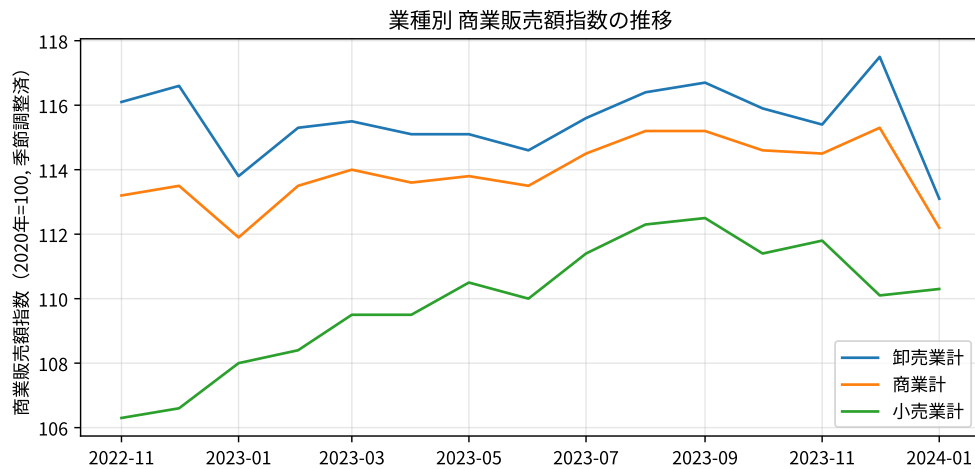


図 13 例題 25 の出力：業種別商業販売額指数

23.13 例題 26：経常利益と設備投資

この例題のポイント

使う統計：法人企業統計調査（時系列・全産業）／ ID：0003061946

全規模・全産業の経常利益と設備投資（四半期）を二軸で重ねて、企業収益と投資の関係を見る。

scripts/ex26_corporate.py

経常利益と設備投資を描く

```

1 # /// script
2 # requires-python = ">=3.11"
3 # dependencies = ["requests", "pandas", "matplotlib"]
4 # ///
5 """例題26: 法人企業統計から経常利益と設備投資の推移を描く。
6
7 法人企業統計調査（時系列データ・全産業、統計表ID 0003061946）から、
8 全規模・全産業の経常利益と設備投資（四半期）を取得して二軸で作図する。
9
10 実行:
11     set -a; source .env; set +a
12     uv run scripts/ex26_corporate.py
13 """
14 import os
15
16 import matplotlib
17 matplotlib.use("Agg")
18 import matplotlib.pyplot as plt
19 import jpfont # noqa: F401
20
21 import estat_api as e
22
23 SID = "0003061946"
24

```

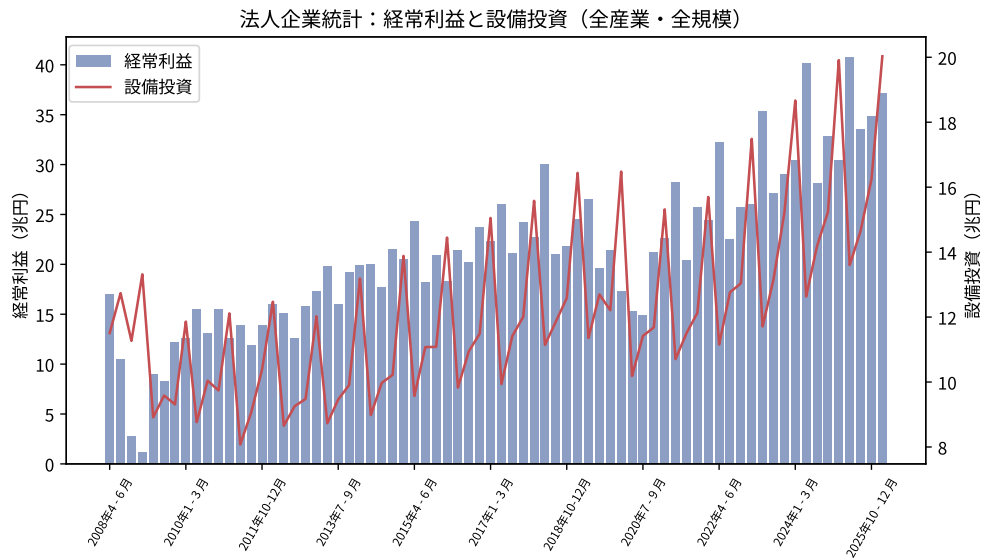
```

25
26 def main() -> None:
27     app_id = e.get_app_id()
28
29     sdata = e.get_stats_data(
30         app_id, SID,
31         cdCat01="086,040",    # 086=経常利益(当期末), 040=設備投資
32         cdCat03="26",        # 規模: 全規模
33         fetch_all=True,
34     )
35     df = e.stats_data_to_frame(sdata).sort_values("time_code")
36     keiri = df[df["cat01"].str.contains("経常利益")]
37     setubi = df[df["cat01"].str.contains("設備投資")]
38
39     x = range(len(keiri))
40     fig, ax1 = plt.subplots(figsize=(9, 4.5))
41     ax1.bar(x, keiri["値"] / 1e6, color="#8C9EC4", label="経常利益")
42     ax1.set_ylabel("経常利益 (兆円) ")
43     ax2 = ax1.twinx()
44     ax2.plot(x, setubi["値"].to_numpy() / 1e6, color="#C44E52", label="設備投資")
45     ax2.set_ylabel("設備投資 (兆円) ")
46     step = max(1, len(keiri) // 10)
47     ax1.set_xticks(list(x)[::step])
48     ax1.set_xticklabels(keiri["time"].iloc[::step], rotation=60, fontsize=7)
49     ax1.set_title("法人企業統計：経常利益と設備投資（全産業・全規模）")
50     fig.legend(loc="upper left", bbox_to_anchor=(0.12, 0.88))
51
52     os.makedirs("figures", exist_ok=True)
53     out = "figures/ex26_corporate.pdf"
54     fig.savefig(out, bbox_inches="tight")
55     last = keiri.iloc[-1]
56     print(f"最新 {last['time']}: 経常利益 {last['値'] / 1e6:.1f} 兆円")
57     print(f"→ 図を {out} に保存しました。")
58
59
60 if __name__ == "__main__":
61     main()

```

実行結果（抜粋）

最新 2026年1 - 3 月: 経常利益 37.1 兆円



23.14 例題 27：進学率の長期推移

この例題のポイント

使う統計：学校基本調査（年次統計・進学率）／ ID：0003147040

高等学校等への進学率と大学・短大等への現役進学率の長期時系列を描く。

scripts/ex27_advancement_rate.py

進学率の推移を描く

```

1 # /// script
2 # requires-python = ">=3.11"
3 # dependencies = ["requests", "pandas", "matplotlib"]
4 # ///
5 """例題27: 学校基本調査から進学率の長期推移を描く。
6
7 学校基本調査（年次統計，統計表ID 0003147040「進学率（1948年～）」から、
8 高等学校等への進学率と大学・短大等への進学率の長期時系列を取得して作図する。
9
10 実行:
11     set -a; source .env; set +a
12     uv run scripts/ex27_advancement_rate.py
13 """
14 import os
15
16 import matplotlib
17 matplotlib.use("Agg")
18 import matplotlib.pyplot as plt
19 import jpfont # noqa: F401
20
21 import estat_api as e

```

```

22
23 SID = "0003147040"
24
25
26 def main() -> None:
27     app_id = e.get_app_id()
28
29     sdata = e.get_stats_data(
30         app_id, SID,
31         cdCat01="0000000010",          # 性別: 計
32         cdCat02="0000000020,0000000040", # 高校等進学率, 大学・短大等への現役進学率
33         fetch_all=True,
34     )
35     df = e.stats_data_to_frame(sdata)
36     df["年"] = df["time"].str.extract(r"(\d+)").astype(int)
37     df = df.sort_values("年")
38
39     fig, ax = plt.subplots(figsize=(9, 4.5))
40     for name, sub in df.groupby("cat02"):
41         label = "高校等進学率" if "高等学校" in name else "大学・短大等進学率"
42         ax.plot(sub["年"], sub["値"], marker="", label=label)
43     ax.set_ylabel("進学率 (%) ")
44     ax.set_xlabel("年")
45     ax.set_title("進学率の長期推移")
46     ax.legend()
47     ax.grid(alpha=0.3)
48
49     os.makedirs("figures", exist_ok=True)
50     out = "figures/ex27_advancement_rate.pdf"
51     fig.savefig(out, bbox_inches="tight")
52     print(f"期間: {df['年'].min()}年 ~ {df['年'].max()}年")
53     latest_year = df["年"].max()
54     for name, sub in df.groupby("cat02"):
55         print(f" {name[:20]}…: {latest_year}年 = {sub[sub['年'] == latest_year]['値'].iloc[0]}%")
56     print(f"→ 図を {out} に保存しました。")
57
58
59 if __name__ == "__main__":
60     main()

```

実行結果 (抜粋)

期間: 1984年 ~ 2016年
 大学・短期大学等への現役進学率…: 2016年 = 55.0%
 高等学校等への進学率…: 2016年 = 98.7%

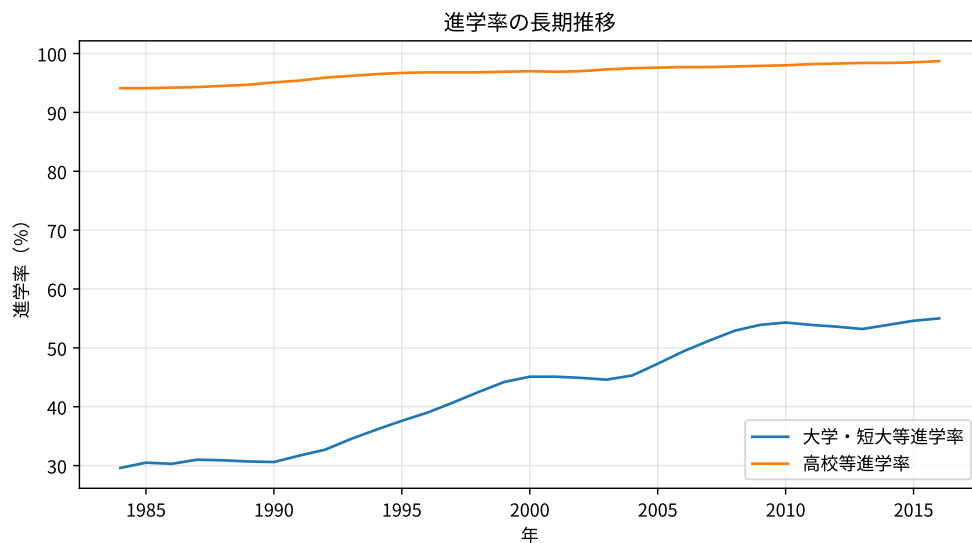


図 15 例題 27 の出力：進学率の長期推移

24 グループ D：応用

24.1 例題 28：データカタログで公開ファイルを探す (getDataCatalog)

この例題のポイント

API：getDataCatalog / パラメータ：searchWord, dataType=CSV

getStatsData が数値そのものを返すのに対し、getDataCatalog は e-Stat 上で公開されているファイル資源 (CSV・Excel 等) のカタログとダウンロード URL を返す。

scripts/ex28_datacatalog.py

データカタログを検索する

```

1 # /// script
2 # requires-python = ">=3.11"
3 # dependencies = ["requests"]
4 # ///
5 """例題28: getDataCatalog で公開ファイル (CSV/Excel等) を探す。
6
7 getStatsData が統計数値そのものを返すのに対し、getDataCatalog は e-Stat 上で
8 公開されているファイル資源 (CSV・Excel・PDF・DB等) のカタログ情報を返す。
9 ここでは「家計調査」のCSV資源を検索し、ダウンロード用URLを一覧する。
10
11 実行:
12     set -a; source .env; set +a
13     uv run scripts/ex28_datacatalog.py
14 """
15 import os

```

```

15 import sys
17
18 import requests
19
20 URL = "https://api.e-stat.go.jp/rest/3.0/app/json/getDataCatalog"
21
22
23 def as_list(x):
24     return x if isinstance(x, list) else [x]
25
26
27 def text(x):
28     return x["$"] if isinstance(x, dict) else (" " if x is None else str(x))
29
30
31 def main() -> None:
32     app_id = os.environ.get("ESTAT_APPID") or sys.exit("ESTAT_APPID 未設定")
33
34     params = {
35         "appId": app_id,
36         "searchWord": "家計調査",
37         "dataType": "CSV", # CSV形式の資源に限定
38         "limit": 5,
39     }
40     body = requests.get(URL, params=params, timeout=60).json()["GET_DATA_CATALOG"]
41     if body["RESULT"]["STATUS"] != 0:
42         sys.exit(f"APIエラー: {body['RESULT']['ERROR_MSG']}")
43
44     info = body["DATA_CATALOG_LIST_INF"]
45     print(f"カタログのヒット数: {info.get('NUMBER')} (先頭{params['limit']}件) \n")
46
47     for cat in as_list(info["DATA_CATALOG_INF"]):
48         dataset = cat.get("DATASET", {})
49         title = dataset.get("TITLE", {}).get("NAME", "") # TITLE は辞書 (NAME を取る)
50         stat = text(dataset.get("STAT_NAME"))
51         print(f"■ [{cat['@id']}] {stat} / {title[:50]}")
52         # 各カタログに紐づくダウンロード資源 (csv等) の名称とURL
53         resources = as_list(cat.get("RESOURCES", {}).get("RESOURCE", []))
54         for r in resources[:3]:
55             rname = r.get("TITLE", {}).get("NAME", "")
56             url = r.get("URL", "")
57             print(f"    - {rname[:40]}\n        {url}")
58
59
60 if __name__ == "__main__":
61     main()

```

実行結果 (抜粋)

カタログのヒット数: 6 (先頭5件)

■ [000001457786] 家計調査 / 家計調査_長期時系列表_2019年

- 品目分類 (全品目) (2015年改定): 支出金額 (年) ...

<https://www.e-stat.go.jp/stat-search/file-download?statInfId=000040270655&fileKind=1>

24.2 例題 29：複数表を一括取得する（getStatsDatas）

この例題のポイント

API：getStatsDatas（POST）／ パラメータ：statsDatasSpec（JSON 配列）

statsDatasSpec に複数の取得条件を JSON 配列で渡すと、1 回のリクエストで複数表を取得できる。応答は @requestNo で対応づく平坦な構造で返るため、表情報とデータを突き合わせる。ただし、一括取得は全リクエスト合計の返却セル数が大きすぎるとエラーになる。大量データの回収では、表ごとに getStatsData を呼び、NEXT_KEY で分割取得する。

scripts/ex29_bulk_stats.py

複数表を一括取得する

```
1 # /// script
2 # requires-python = ">=3.11"
3 # dependencies = ["requests", "pandas"]
4 # ///
5 """例題29: getStatsDatas で複数の統計表を一度に取得する。
6
7 statsDatasSpec に複数の取得条件（JSON配列）を渡すと、1回のリクエストで
8 複数表のデータをまとめて取得できる。ここでは消費者物価指数（総合・前年同月比）と
9 完全失業率の最新値を同時に取得する。
10
11 実行:
12     set -a; source .env; set +a
13     uv run scripts/ex29_bulk_stats.py
14 """
15 import json
16 import os
17 import sys
18
19 import requests
20
21 import estat_api as e
22
23 URL = "https://api.e-stat.go.jp/rest/3.0/app/json/getStatsDatas"
24
25
26 def main() -> None:
27     app_id = os.environ.get("ESTAT_APPID") or sys.exit("ESTAT_APPID 未設定")
28
29     spec = [
30         {"statsDataId": "0003427113", "cdTab": "3", "cdCat01": "0001",
31          "cdArea": "00000", "lvTime": "4", "cdTimeFrom": "2026000101"}, # CPI総合・前年同月比
32         {"statsDataId": "0003005865", "cdTab": "02", "cdCat01": "000",
33          "cdCat02": "08", "cdCat03": "0", "cdTimeFrom": "2026000101"}, # 完全失業率
34     ]
35     # getStatsDatas は POST で呼ぶ (statsDatasSpec を本文に渡す)
36     params = {"appId": app_id, "statsDatasSpec": json.dumps(spec, ensure_ascii=False)}
37     body = requests.post(URL, data=params, timeout=120).json()["GET_STATS_DATAS"]
38     if body["RESULT"]["STATUS"] != 0:
```

```

39     sys.exit(f"APIエラー: {body['RESULT']['ERROR_MSG']}")
40
41     # getStatsDatas は @requestNo で対応づく平坦な構造で返る
42     sdl = body["STATISTICAL_DATA_LIST"]
43     titles = {t["@requestNo"]: e.text(t.get("TITLE")) for t in e.as_list(sdl["TABLE_INF_LIST"]
44                                     ["TABLE_INF"])}
45
46     print(f"一度のリクエストで {len(titles)} 表を取得しました。 \n")
47     for di in e.as_list(sdl["DATA_INF_LIST"] ["DATA_INF"]):
48         rno = di["@requestNo"]
49         values = e.as_list(di["VALUE"])
50         latest = max(values, key=lambda v: v["@time"])
51         t = latest["@time"]
52         period = f"{t[:4]}年{t[8:10]}月" # 月次コード YYYY00MMMM
53         print(f"■ {titles[rno][:40]}")
54         print(f"    最新 {period}: {latest['$']}{latest.get('@unit', '')}")
55
56 if __name__ == "__main__":
57     main()

```

実行結果（抜粋）

一度のリクエストで 2 表を取得しました。

■ 消費者物価指数（2020年基準）

最新 2026年05月: 1.5%

■ 労働力人口比率、就業率及び完全失業率（2000年1月～）

最新 2026年04月: 2.7%

24.3 例題 30：再利用クライアントと英語メタ（lang=E）

この例題のポイント

使う統計：消費者物価指数・労働力調査／ 着目点：モジュール再利用, lang=E

estat_api を使って複数指標の最新値を集める「ダッシュボード」を作り、最後に lang=E で英語のメタ情報を取得する方法も示す。

scripts/ex30_mini_client.py

指標ダッシュボードと英語メタ

```

1 # /// script
2 # requires-python = ">=3.11"
3 # dependencies = ["requests", "pandas"]
4 # ///
5 """例題30: 共通モジュールを使った「景気指標ダッシュボード」と英語メタの取得。
6
7 これまで作った estat_api モジュールを使って、複数の統計から最新値を集めて
8 一覧表示する総合例。最後に lang="E" を指定して英語のメタ情報を取得する方法も示す。
9
10 実行:
11     set -a; source .env; set +a
12     uv run scripts/ex30_mini_client.py
13 """

```

```

14 import estat_api as e
15
16
17 def latest_value(app_id, sid, **cond) -> tuple[str, float, str]:
18     """指定条件で取得した系列の最新値（時点、値、単位）を返す。"""
19     sdata = e.get_stats_data(app_id, sid, **cond)
20     df = e.stats_data_to_frame(sdata).sort_values("time_code")
21     row = df.iloc[-1]
22     return row["time"], row["値"], row.get("単位", "")
23
24
25 def main() -> None:
26     app_id = e.get_app_id()
27
28     print("==== 主要経済指標ダッシュボード ====")
29     indicators = [
30         ("消費者物価指数 前年同月比（総合）", "0003427113",
31          dict(cdTab="3", cdCat01="0001", cdArea="00000", lvTime="4", cdTimeFrom="2025000101")),
32         ("完全失業率（%）", "0003005865",
33          dict(cdTab="02", cdCat01="000", cdCat02="08", cdCat03="0", cdTimeFrom="2025000101")),
34     ]
35     for label, sid, cond in indicators:
36         period, value, unit = latest_value(app_id, sid, **cond)
37         print(f"  {label:28s}: {value}{unit}  ({period})")
38
39     # --- lang="E" で英語のメタ情報を取得する ---
40     print("\n==== 英語メタの例（lang=E）====")
41     tables = e.get_stats_list(app_id, statsCode="00200573", limit=2, lang="E")
42     for t in tables:
43         print(f"  [{t['@id']}] {e.text(t.get('STATISTICS_NAME'))}")
44
45
46 if __name__ == "__main__":
47     main()

```

実行結果（抜粋）

```

==== 主要経済指標ダッシュボード ====
  消費者物価指数 前年同月比（総合）          : 1.5% （2026年5月）
  完全失業率（%）                          : 2.7% （2026年4月）
==== 英語メタの例（lang=E） ====
  [0002050001] 2005-Base Consumer Price Index

```

25 練習問題と解答：例題・応用編

問 1.問題 都道府県別人口を取得し、上位 5 都道府県を確認せよ。

解答 例題 8 を実行する。lvArea=2 により都道府県だけに絞られる。

実行結果（抜粋）

```
$ uv run scripts/ex08_filter_area.py
対象年: 2024年 / 都道府県数: 47
人口の多い上位5都道府県（単位：千人）：
area      値
東京都   14178
神奈川県  9225
大阪府    8757
愛知県    7460
埼玉県    7332
```

問 2.問題 都道府県別人口ランキングの図を再生成し、保存先を確認せよ。

解答 例題 15 を実行する。図は figures/ に保存される。

実行結果（抜粋）

```
$ uv run scripts/ex15_prefecture_ranking.py
対象年: 2024年
area      値
東京都   14178
神奈川県  9225
大阪府    8757
→ ランキング図を figures/ex15_prefecture_ranking.pdf に保存しました。
```

問 3.問題 完全失業率の月次系列について、取得期間と直近値を確認せよ。

解答 例題 18 を実行する。月次系列は 2000 年以降の長い時系列として取得される。

実行結果（抜粋）

```
$ uv run scripts/ex18_unemployment.py
取得期間: 2000-01 ~ 2026-04 (316か月)
直近の完全失業率: 2.7% (2026年04月)
→ 図を figures/ex18_unemployment.pdf に保存しました。
```

問 4.問題 消費者物価指数の前年同月比について、総合とコアの直近値を確認せよ。

解答 例題 21 を実行する。cdTab=3 が前年同月比を表す。

実行結果（抜粋）

```
$ uv run scripts/ex21_cpi_inflation.py
直近の前年同月比：
      年月      cat01      値
2026-05-01 0161 生鮮食品を除く総合 1.4
2026-05-01      0001 総合 1.5
→ 図を figures/ex21_cpi_inflation.pdf に保存しました。
```

問 5.問題 getStatsDatas で複数指標を一括取得し、各表の最新値を確認せよ。

解答 例題 29 を実行する。応答は @requestNo で表情報とデータを対応づける。

実行結果（抜粋）

```
$ uv run scripts/ex29_bulk_stats.py
一度のリクエストで 2 表を取得しました。
```

- 消費者物価指数（2020年基準）
最新 2026年05月：1.5%
- 労働力人口比率、就業率及び完全失業率（2000年1月～）
最新 2026年04月：2.7%

以上の 30 例で、`getStatsList`・`getMetaInfo`・`getStatsData`・`getSimpleStatsData`・`getDataCatalog`・`getStatsDatas` の主要機能と、時間・地域・分類・レベルによる絞り込み、ページング、コード変換、可視化、英語メタの取得までをひとつおとり扱った。これらを組み合わせれば、研究・実務で必要な公的統計データを自在に取得・分析できるはずである。

第 III 部 付録

A コーディングエージェント用プロンプト集

本付録は、Claude Code などのコーディングエージェントに指示して、第 2 部と同等のスクリプトを自動生成させるためのプロンプト集である。使い方は次のとおり。

- まず [A.1 節の共通の前提プロンプト](#) をエージェントに与え、作成方針（uv・PEP 723・ESTAT_APPID・整形・作図の流儀）を共有する。
- 続けて、作りたい例題に対応する個別プロンプト（[A.2 節以降](#)）を与える。
- 生成されたスクリプトを `uv run` で実行し、結果を確認する。

ヒント

個別プロンプトには統計表 ID（`statsDataId`）や分類コードを記してあるが、これらは調査年や基準改定で変わりうる。エージェントには「まず `getStatsList`/`getMetaInfo` で最新の ID とコードを確認してから実装する」よう促すと、より堅牢になる。

A.1 共通の前提プロンプト

共通プロンプト（最初に 1 回与える）

あなたは Python のコーディングエージェントです。これから、政府統計の総合窓口「e-Stat」の API 機能（バージョン 3.0）を使うスクリプトを作成します。以後すべてのスクリプトで、次の共通方針に従ってください。

- ・実行には uv を用い、ファイル先頭に PEP 723 のメタデータ (`# /// script ~# ///`) を置く。`requires-python = ">=3.11"` とし、使うパッケージ (`requests`, `pandas`, `matplotlib` など) を `dependencies` に列挙する。
- ・アプリケーション ID は `os.environ["ESTAT_APPID"]` から読み取り、未設定なら分かりやすいメッセージで終了する。ID をソースに直書きしない。
- ・JSON エンドポイント `https://api.e-stat.go.jp/rest/3.0/app/json/<機能名>` を使う。
- ・レスポンスの `RESULT.STATUS` を確認する。0 は正常、1 は該当データなし、2 は一括取得で一部リクエストに該当データなし、100 以上はエラーとして扱う。本例題のようにデータが存在する前提のスクリプトでは、0 以外なら `ERROR_MSG` を表示して終了してよい。`requests` には `timeout` を設定する。
- ・要素数 1 のとき配列にならない仕様に備え、「リストでなければリストに包む」処理を入れる。
- ・取得後は `pandas` で扱う。`CLASS_INF` の `CLASS_OBJ` (コード→名称対応) を使って `@cat01・@area・@time` 等を名称に変換し、列名は次元 ID、値は「値」、元コードは「(次元 ID) _code」とする。
- ・大きな表は `limit` と `NEXT_KEY (startPosition)` でページングする。
- ・秘匿値や該当なしを表す特殊文字 (`-`, `...`, `X` など) は、理由なく 0 に置換しない。必要な場合だけ `replaceSpChar` を指定し、注釈 (`@annotation`) があれば列として残す。
- ・グラフは `matplotlib` で描く。日本語フォントは `matplotlib.font_manager` でシステムのフォント (`Noto Sans CJK JP`, `IPAexGothic` 等) を探して設定する (環境差を避けるため、外部の日本語化専用パッケージに依存しない)。図は `figures/` に保存する。
- ・コードには日本語コメントを付け、`uv run スクリプト.py` で単体実行できるようにする。

A.2 グループ A：API 操作の基礎のプロンプト

例題 1：キーワードで統計表を検索

プロンプト例

`getStatsList` を使い、`searchWord` に「人口」を指定して統計表を検索し、先頭 10 件の統計表 ID (`@id`) と政府統計名・表題を表示するスクリプトを作ってください。ヒット総数 (`DATALIST_INF.NUMBER`) も表示してください。

例題 2：政府統計コードで絞って CSV 保存

プロンプト例

`getStatsList` で `statsCode=00200521` (国勢調査) の統計表を検索し、統計表 ID・統計名・表題・周期・調査年月・公開日を `pandas` の `DataFrame` にまとめて、`output/` に BOM 付き

UTF-8 の CSV として保存するスクリプトを作ってください。

例題 3：分野と複合キーワードで横断検索

プロンプト例

getStatsList で、(1) statsField=03（労働・賃金）と「賃金 産業」の AND 検索、(2)「完全失業率 OR 有効求人倍率」の OR 検索、(3)「人口」かつ collectArea=3（市区町村別）の 3 通りを実行し、それぞれの件数と先頭数件を表示するスクリプトを作ってください。

例題 4：メタ情報（分類の定義）を取得

プロンプト例

statsCode=00200573 の統計表を 1 件見つけ、その表のメタ情報を getMetaInfo で取得して、分類軸（CLASS_OBJ）ごとに名称・選択肢数・先頭数件のコード・階層レベル（@level）を表示するスクリプトを作ってください。

例題 5：取得件数を先に確認（cntGetFlg）

プロンプト例

statsCode=00200531 の表を 1 件見つけ、getStatsData に cntGetFlg=Y を付けて総件数（TOTAL_NUMBER）だけを取得し、既定上限 10 万件で何回のリクエストが必要かを計算して表示するスクリプトを作ってください。

例題 6：生の JSON 構造を観察

プロンプト例

消費者物価指数の表を limit=3 で取得し、RESULT_INF、CLASS_INF の先頭次元、DATA_INF の VALUE 先頭 3 件を json.dumps（日本語そのまま）で整形表示し、メタとデータの対応を観察するスクリプトを作ってください。

A.3 グループ B：getStatsData の絞り込みのプロンプト

例題 7：時間軸で絞る

プロンプト例

消費者物価指数（2020 年基準, statsDataId=0003427113）から、総合（cdCat01=0001）・全国（cdArea=00000）・指数（cdTab=1）の月次系列を、lvTime=4 と cdTimeFrom=2023000101 で 2023 年 1 月以降に絞って取得し、直近 12 か月を表示してください。

例題 8：地域で絞る

プロンプト例

人口推計 (statsDataId=0003448232) から、cdTab=001・cdCat01=000 (男女計)・cdCat02=001 (総人口) について、lvArea=2 で都道府県別の総人口を取得して上位を表示し、さらに cdArea=23000,13000 で愛知県・東京都だけを取得するスクリプトを作ってください。

例題 9：分類事項で絞る

プロンプト例

労働力調査 (statsDataId=0003005865) から、cdTab=02 (率)・cdCat01=000・cdCat03=0 で、就業状態 (cdCat02) に 13 (就業) と 08 (完全失業者) をカンマ指定して同時取得し、就業率と完全失業率の直近値を表示するスクリプトを作ってください。

例題 10：階層レベルで絞る

プロンプト例

消費者物価指数 (0003427113) について、lvCat01 を 1 / 1-2 / 1-3 / 1-4 と変えて取得される品目数の違いを表示し、lvCat01=1 のときの品目名を一覧するスクリプトを作ってください (cdArea=00000, cdTab=1, lvTime=4 で月次に限定)。

例題 11：ページネーション

プロンプト例

消費者物価指数 (0003427113, 巨大表) に対し、limit=1000 で取得しながら RESULT_INF.NEXT_KEY を startPosition に渡してページ送りし、各ページの FROM_NUMBER・TO_NUMBER・NEXT_KEY・TOTAL_NUMBER を表示するスクリプトを作ってください (デモとして 3 ページで打ち切り)。

例題 12：データとメタを別々に取得して結合

プロンプト例

消費者物価指数 (0003427113) について、getMetaInfo でコード→名称の辞書を 1 回だけ作り、getStatsData は metaGetFlg=N で軽量に取得して、あとから @time・@cat01 を名称に結合するスクリプトを作ってください。

例題 13：CSV と JSON を比較

プロンプト例

同じ条件 (消費者物価指数 総合・全国・月次) を、getSimpleStatsData (CSV, sectionHeaderFlg=2) と getStatsData (JSON) の両方で取得し、pandas.read_csv と JSON 整形の結果を比べて、最新値が一致することを確認するスクリプトを作ってください。

A.4 グループ C：取得と可視化のプロンプト

例題 14：人口ピラミッド

プロンプト例

人口推計 (statsDataId=0003443840) から、男女別 (cdCat02=001,002)・年齢 5 歳階級 (cat03、ただし「総数」と「(再掲)」は除外)・総人口 (cdCat04=001) の最新月を取得し、男を左・女を右にした人口ピラミッド (横棒グラフ) を作図して figures/ に保存するスクリプトを作ってください。

例題 15：都道府県別人口ランキング

プロンプト例

人口推計 (0003448232) から最新年の都道府県別総人口 (男女計・総人口, lvArea=2) を取得し、上位 15 都道府県を横棒グラフにして保存するスクリプトを作ってください。

例題 16：出生数と合計特殊出生率

プロンプト例

人口動態調査 (statsDataId=0003411595) から、出生数 (cdCat01=00100) と合計特殊出生率 (00150) の年次系列を取得し、出生数を棒グラフ・合計特殊出生率を折れ線として二軸で重ねた図を作るスクリプトを作ってください。

例題 17：3 大都市圏の転入超過

プロンプト例

住民基本台帳人口移動報告 (statsDataId=0003404102) から、東京圏・名古屋圏・大阪圏 (cdCat01=002,003,004) の転入超過数 (cdCat02=003) の年次推移を取得し、3 本の折れ線で比較する図を作るスクリプトを作ってください。

例題 18：完全失業率の月次推移

プロンプト例

労働力調査 (0003005865) から、完全失業率 (cdTab=02, cdCat02=08, cdCat03=0, 男女総数) の月次系列を取得し、年月を横軸にした折れ線グラフを作るスクリプトを作ってください。

例題 19：現金給与総額の推移

プロンプト例

毎月勤労統計調査 (statsDataId=0003138108) から、現金給与総額 (cdTab=741, cdCat01=001, cdCat02=00, cdCat03=T [5人以上], cdCat04=TL [調査産業計]) の月次系列を取得し、水準と前年同月比を上下 2 段で作図するスクリプトを作ってください (この原表は旧産業分類で 2010~2015 年を対象である点をコメントで明記)。

例題 20：年齢階級別の賃金プロフィール

プロンプト例

賃金構造基本統計調査 (statsDataId=0003029886) から、きまって支給する現金給与額 (cdTab=40) について、企業規模計 (cdCat01=01)・学歴計 (cdCat03=01)・産業計 (cdCat05=01)・民公計 (cdCat06=01) の条件で、男女別 (cdCat02=02,03) × 年齢階級 (cat04) の賃金カーブを折れ線で作図するスクリプトを作ってください。

例題 21：消費者物価指数の前年同月比

プロンプト例

消費者物価指数 (0003427113) の前年同月比 (cdTab=3) について、総合 (cdCat01=0001) と生鮮食品を除く総合 (0161) の 2 系列を全国 (00000)・月次で取得し、折れ線で比較する図 (2%目標の補助線付き) を作るスクリプトを作ってください。

例題 22：費目別消費支出の構成

プロンプト例

家計調査 (品目分類 2020 年改定, statsDataId=0003348239) から、二人以上の世帯 (cdCat02=03)・全国 (cdArea=00000) の 10 大費目 (食料~その他の消費支出の 0X0000000 コード) について最新年の消費支出を取得し、構成比を円グラフにするスクリプトを作ってください。

例題 23：実質 GDP 成長率 (年率換算)

プロンプト例

四半期別 GDP 速報の実質季節調整系列 (statsDataId=0003376098) から「国内総生産 (支出側)」の四半期系列を取得し、前期比を年率換算 ($((x_t/x_{t-1})^4 - 1) \times 100$) して棒グラフにするスクリプトを作ってください (公表時点アーカイブである旨をコメントに記載)。

例題 24：鉱工業生産指数

プロンプト例

鉱工業生産・出荷・在庫指数（2020 年基準, statsDataId=0004052177）から、鉱工業（cdCat01=0001000）の季節調整済生産指数を取得し、時間軸が YYYYMM 形式の行（ウエイト等の非日付行は除外）だけを使って推移を折れ線にするスクリプトを作ってください。

例題 25：業種別商業販売額指数

プロンプト例

商業動態統計調査（statsDataId=0004026181）から、商業計・卸売業計・小売業計について季節調整済指数（cdCat02=01080300）の月次系列を取得し、3 本の折れ線で比較する図を作るスクリプトを作ってください。

例題 26：経常利益と設備投資

プロンプト例

法人企業統計調査（時系列・全産業, statsDataId=0003061946）から、全規模（cdCat03=26）の経常利益（cdCat01=086）と設備投資（040）の四半期系列を取得し、二軸（棒+折れ線）で作図するスクリプトを作ってください。

例題 27：進学率の長期推移

プロンプト例

学校基本調査（年次統計, statsDataId=0003147040）から、性別計（cdCat01=0000000010）の高等学校等への進学率（cdCat02=0000000020）と大学・短大等への現役進学率（0000000040）の長期時系列を取得し、折れ線で作図するスクリプトを作ってください。

A.5 グループ D：応用のプロンプト

例題 28：データカタログで公開ファイルを探す

プロンプト例

getDataCatalog を使い、searchWord=家計調査・dataType=CSV で公開ファイルのカタログを検索し、各データセットの統計名・タイトル（DATASET.TITLE.NAME）と、紐づく資源（RESOURCE）のダウンロード URL を一覧表示するスクリプトを作ってください。

例題 29：複数表を一括取得（getStatsDatas）

プロンプト例

getStatsDatas を POST で呼び、statsDatasSpec に「消費者物価指数の前年同月比（0003427113）」と「完全失業率（0003005865）」の 2 条件を JSON 配列で渡して一括取得し、

@requestNo で表情報 (TABLE_INF_LIST) とデータ (DATA_INF_LIST) を突き合わせて各表の最新値を表示するスクリプトを作ってください。

例題 30：再利用クライアントと英語メタ

プロンプト例

これまでの共通処理 (appId 取得・getStatsData 呼び出し・コード→名称変換) を関数群にまとめたうえで、消費者物価指数の前年同月比と完全失業率の最新値を一覧する「景気指標ダッシュボード」を表示し、最後に lang=E を付けて getStatsList (statsCode=00200573) の英語メタ情報を取得して表示するスクリプトを作ってください。

B Flet で作る GUI アプリ

第 2 部までのスクリプトは、すべてコマンドラインで実行し、結果を端末やグラフファイルに出力するものだった。本付録では、同じ e-Stat API を GUI (画面) アプリから呼び出す例を 3 つ示す。GUI 化には Python だけでデスクトップ / Web / モバイルアプリを作れる Flet¹⁾ を用いる。

これまでの例題と大きく異なるのは、アプリケーション ID (appId) を画面の入力欄から受け取る点である。環境変数 ESTAT_APPID には依存せず、利用者が自分の ID をテキスト欄に貼り付けて使う。配布しやすく、ID をソースに残さない利点がある。

この例題のポイント

- 依存パッケージは flet と requests だけ。pandas や matplotlib は使わず、表は DataTable、棒グラフは Container の幅で描く。
- エントリーポイントは ft.run(main)。画面の組み立ては def main(page) の中で行い、部品 (コントロール) を page.add(...) で配置する。
- ボタン等のイベント処理は async def で書く。requests は通信が終わるまで戻らない (ブロッキング) ため、await asyncio.to_thread(...) で別スレッドに逃がし、待っている間も画面が固まらないようにする。
- 取得中は ProgressRing (くるくる) を表示し、エラーは画面上の Text に出す。レスポンスの RESULT.STATUS 確認や「1 件のとき配列にならない」対策は、第 1 部までと同じ考え方で行う。

実行方法 いずれも PEP 723 形式なので、uv run で必要なパッケージが自動的に用意される。デスクトップウィンドウで開くには次のようにする。

```
uv run scripts/flet_app1_search.py # デスクトップウィンドウで起動
```

1) <https://flet.dev/docs/>。Flet は Flutter を Python から扱うフレームワークであり、API は更新されることがある。

```
flet run scripts/flet_app1_search.py      # (flet を入れていれば) 同上
flet run --web scripts/flet_app1_search.py  # ブラウザで起動
```

appId は <https://www.e-stat.go.jp/api/> で無料登録すると取得できる。アプリ起動後、画面上部の「アプリケーション ID」欄に貼り付けてから操作する。

3つのアプリのつながり

3つは「①表を探す (getStatsList) → ②絞り込みコードを調べる (getMetaInfo) → ③データを取って可視化する (getStatsData)」という e-Stat 利用の典型的な流れに対応している。アプリ 1 で見つけた統計表 ID (statsDataId) をアプリ 2・3 の入力欄にコピーして使うとよい。

B.1 アプリ 1：統計表をキーワード検索する

この例題のポイント

getStatsList を呼ぶ最も基本的な GUI。キーワードと件数を入力して「検索」を押すと、ヒットした統計表をカードの一覧 (スクロール可能な ListView) に表示する。各カードには表題・政府統計名・統計表 ID・調査年月を載せ、ID は選択コピーできるようにしている。テキスト欄での Enter (on_submit) でも検索できる。

scripts/flet_app1_search.py

Flet で統計表を検索する

```
1 # /// script
2 # requires-python = ">=3.11"
3 # dependencies = ["flet", "requests"]
4 # ///
5 """Flet サンプル1: e-Stat API で統計表をキーワード検索する GUI アプリ。
6
7 appID (アプリケーションID) はアプリ起動後に画面上のテキスト欄へ入力する。
8 キーワードを入れて「検索」を押すと getStatsList を呼び、ヒットした統計表を
9 一覧 (スクロール可能なリスト) に表示する。
10
11 実行:
12     uv run scripts/flet_app1_search.py      # デスクトップウィンドウで起動
13     # もしくは
14     flet run scripts/flet_app1_search.py    # 同上
15     flet run --web scripts/flet_app1_search.py  # ブラウザで起動
16
17 appID は https://www.e-stat.go.jp/api/ で取得 (無料登録)。
18 """
19 import asyncio
20
21 import flet as ft
22 import requests
23
24 BASE = "https://api.e-stat.go.jp/rest/3.0/app/json"
25
```

```

26
27 def _text(x) -> str:
28     """{'@...':..., '$': '本文'} 形式のことがある値を文字列に正規化する。"""
29     return x["$"] if isinstance(x, dict) else (" if x is None else str(x))
30
31
32 def _as_list(x) -> list:
33     """要素1件だと配列にならない API 仕様への対策。"""
34     return x if isinstance(x, list) else [x]
35
36
37 def search_stats_list(app_id: str, word: str, limit: int) -> tuple[str, list[dict]]:
38     """getStatsList を同期的に呼ぶ（呼び出し側で別スレッド実行する）。
39
40     返り値は（ヒット総数の文字列、統計表のリスト）。エラーは例外で送出する。
41     """
42     params = {"appId": app_id, "searchWord": word, "limit": limit}
43     res = requests.get(f"{BASE}/getStatsList", params=params, timeout=60)
44     res.raise_for_status()
45     body = res.json()["GET_STATS_LIST"]
46     if body["RESULT"]["STATUS"] != 0:
47         raise RuntimeError(body["RESULT"]["ERROR_MSG"])
48     info = body["DATALIST_INF"]
49     total = str(info.get("NUMBER", "不明"))
50     return total, _as_list(info.get("TABLE_INF", []))
51
52
53 async def main(page: ft.Page) -> None:
54     page.title = "e-Stat 統計表検索"
55     page.window.width = 760
56     page.window.height = 720
57     page.scroll = ft.ScrollMode.AUTO
58
59     # --- 入力欄 ---
60     app_id_field = ft.TextField(
61         label="アプリケーションID (appId) ",
62         hint_text="e-Stat で取得した API キーを貼り付け",
63         password=True,
64         can_reveal_password=True,
65     )
66     word_field = ft.TextField(label="検索キーワード", value="人口", expand=True)
67     limit_field = ft.TextField(label="件数", value="20", width=90)
68
69     status = ft.Text(color=ft.Colors.RED)
70     spinner = ft.ProgressRing(visible=False, width=20, height=20)
71     results = ft.ListView(expand=True, spacing=6, padding=4)
72
73     async def on_search(e: ft.ControlEvent) -> None:
74         app_id = app_id_field.value.strip()
75         word = word_field.value.strip()
76         if not app_id:
77             status.value = "appId を入力してください。"
78         return

```

```

79     if not word:
80         status.value = "キーワードを入力してください。"
81         return
82     try:
83         limit = max(1, min(100, int(limit_field.value or "20")))
84     except ValueError:
85         status.value = "件数は整数で入力してください。"
86         return
87
88     # 検索中の表示にする
89     status.value = ""
90     results.controls.clear()
91     spinner.visible = True
92     search_btn.disabled = True
93     page.update()
94
95     try:
96         # requests はブロッキングなので別スレッドで実行し UI を固めない
97         total, tables = await asyncio.to_thread(search_stats_list, app_id, word, limit)
98     except requests.HTTPError as ex:
99         status.value = f"通信エラー: {ex}"
100    except Exception as ex: # API エラー含む
101        status.value = f"エラー: {ex}"
102    else:
103        status.value = f"ヒット総数 {total} 件 (先頭 {len(tables)} 件を表示) "
104        status.color = ft.Colors.GREEN
105        for t in tables:
106            stat_name = _text(t.get("STAT_NAME"))
107            title = _text(t.get("TITLE"))
108            survey = _text(t.get("SURVEY_DATE"))
109            results.controls.append(
110                ft.Card(
111                    content=ft.Container(
112                        padding=10,
113                        content=ft.Column(
114                            spacing=2,
115                            controls=[
116                                ft.Text(title or "(表題なし)", weight=ft.FontWeight.BOLD),
117                                ft.Text(stat_name, size=12, color=ft.Colors.BLUE_GREY),
118                                ft.Text(
119                                    f"統計表ID: {t.get('@id', '')}    調査年月: {survey or '-}'"
120                                ),
121                                ft.Text(
122                                    size=11,
123                                    selectable=True,
124                                ),
125                            ],
126                        ),
127                    ),
128                )
129            )
130    finally:
131        spinner.visible = False
132        search_btn.disabled = False

```

```

131         page.update()
132
133     search_btn = ft.Button("検索", icon=ft.Icons.SEARCH, on_click=on_search)
134     word_field.on_submit = on_search
135
136     page.add(
137         ft.Text("e-Stat 統計表検索", size=22, weight=ft.FontWeight.BOLD),
138         app_id_field,
139         ft.Row([word_field, limit_field, search_btn], vertical_alignment=ft.CrossAxisAlignment.END),
140     ),
141     ft.Row([spinner, status]),
142     ft.Divider(),
143     ft.Container(content=results, expand=True, height=420),
144 )
145
146 if __name__ == "__main__":
147     ft.run(main)

```

B.2 アプリ 2：メタ情報（分類の定義）を表示する

この例題のポイント

統計表 ID を入力して `getMetaInfo` を呼び、分類事項 (CLASS_OBJ) ごとに折りたたみパネル (ExpansionTile) を作る。パネルを開くと、その分類の「コード ↔ 名称 ↔ 単位」を DataTable で確認できる。アプリ 3 でデータを絞り込む際に指定する `cdTab・cdCat01・cdArea` などのコードを調べるのに役立つ。分類が大きい場合に画面が重くならないよう、1 分類あたりの表示件数に上限を設けている。

scripts/flet_app2_metadata.py

Flet でメタ情報を表示する

```

1  # /// script
2  # requires-python = ">=3.11"
3  # dependencies = ["flet", "requests"]
4  # ///
5  """Flet サンプル2: e-Stat API で統計表のメタ情報を表示する GUI アプリ。
6
7  appID は画面上のテキスト欄に入力する。統計表ID (statsDataId) を入れて
8  「メタ情報取得」を押すと getMetaInfo を呼び、分類事項 (CLASS_OBJ) ごとに
9  折りたたみパネルを作り、その中に「コード ↔ 名称」の表を表示する。
10
11  getStatsData を絞り込むときに使う cdTab / cdCat01 / cdArea などのコードを
12  調べるのに便利。統計表IDはサンプル1 (flet_app1_search.py) で探せる。
13
14  実行:
15      uv run scripts/flet_app2_metadata.py
16      flet run scripts/flet_app2_metadata.py
17  """
18  import asyncio
19

```

```

20 import flet as ft
21 import requests
22
23 BASE = "https://api.e-stat.go.jp/rest/3.0/app/json"
24
25
26 def _as_list(x) -> list:
27     return x if isinstance(x, list) else [x]
28
29
30 def get_meta_info(app_id: str, sid: str) -> tuple[str, list[dict]]:
31     """getMetaInfo を同期的に呼び（統計表タイトル, CLASS_OBJ のリスト）を返す。"""
32     params = {"appId": app_id, "statsDataId": sid}
33     res = requests.get(f"{BASE}/getMetaInfo", params=params, timeout=60)
34     res.raise_for_status()
35     body = res.json()["GET_META_INFO"]
36     if body["RESULT"]["STATUS"] != 0:
37         raise RuntimeError(body["RESULT"]["ERROR_MSG"])
38     meta = body["METADATA_INF"]
39     title = meta["TABLE_INF"]["TITLE"]
40     title = title["$"] if isinstance(title, dict) else str(title)
41     objs = _as_list(meta["CLASS_INF"]["CLASS_OBJ"])
42     return title, objs
43
44
45 # 1分類あたり表に出すコード数の上限（大きい分類で画面が重くなるのを防ぐ）
46 MAX_ROWS = 50
47
48
49 async def main(page: ft.Page) -> None:
50     page.title = "e-Stat メタ情報ビューア"
51     page.window.width = 820
52     page.window.height = 760
53     page.scroll = ft.ScrollMode.AUTO
54
55     app_id_field = ft.TextField(
56         label="アプリケーションID (appId) ",
57         password=True,
58         can_reveal_password=True,
59     )
60     sid_field = ft.TextField(
61         label="統計表ID (statsDataId) ",
62         hint_text="例: 0003448232",
63         expand=True,
64     )
65
66     status = ft.Text(color=ft.Colors.RED)
67     spinner = ft.ProgressRing(visible=False, width=20, height=20)
68     title_text = ft.Text("", size=15, weight=ft.FontWeight.BOLD, selectable=True)
69     panels = ft.Column(spacing=4)
70
71     def build_panel(obj: dict) -> ft.ExpansionTile:
72         """1つの分類事項 (CLASS_OBJ) を折りたたみパネルにする。"""

```

```

73     classes = _as_list(obj["CLASS"])
74     rows = [
75         ft.DataRow(
76             cells=[
77                 ft.DataCell(ft.Text(c.get("@code", ""), selectable=True)),
78                 ft.DataCell(ft.Text(c.get("@name", ""), selectable=True)),
79                 ft.DataCell(ft.Text(c.get("@unit", "") or "-")),
80             ]
81         )
82         for c in classes[:MAX_ROWS]
83     ]
84     table = ft.DataTable(
85         columns=[
86             ft.DataColumn(label=ft.Text("コード")),
87             ft.DataColumn(label=ft.Text("名称")),
88             ft.DataColumn(label=ft.Text("単位")),
89         ],
90         rows=rows,
91         column_spacing=24,
92     )
93     note = ""
94     if len(classes) > MAX_ROWS:
95         note = f" (全 {len(classes)} 件中、先頭 {MAX_ROWS} 件を表示) "
96     return ft.ExpansionTile(
97         title=ft.Text(f"@{obj.get('@id', '')} : {obj.get('@name', '')} ({len(classes)}件) "),
98         subtitle=ft.Text(note) if note else None,
99         controls=[ft.Container(content=table, padding=ft.Padding.only(left=12, bottom=8))],
100     )
101
102     async def on_fetch(e: ft.ControlEvent) -> None:
103         app_id = app_id_field.value.strip()
104         sid = sid_field.value.strip()
105         if not app_id:
106             status.value = "appId を入力してください。"
107             status.color = ft.Colors.RED
108             return
109         if not sid:
110             status.value = "統計表ID (statsDataId) を入力してください。"
111             status.color = ft.Colors.RED
112             return
113
114         status.value = ""
115         title_text.value = ""
116         panels.controls.clear()
117         spinner.visible = True
118         fetch_btn.disabled = True
119         page.update()
120
121         try:
122             title, objs = await asyncio.to_thread(get_meta_info, app_id, sid)
123         except requests.HTTPError as ex:
124             status.value = f"通信エラー: {ex}"
125             status.color = ft.Colors.RED

```

```

126     except Exception as ex:
127         status.value = f"エラー: {ex}"
128         status.color = ft.Colors.RED
129     else:
130         title_text.value = title
131         status.value = f"分類事項 {len(objs)} 種類を取得しました。"
132         status.color = ft.Colors.GREEN
133         for obj in objs:
134             panels.controls.append(build_panel(obj))
135     finally:
136         spinner.visible = False
137         fetch_btn.disabled = False
138         page.update()
139
140     fetch_btn = ft.Button("メタ情報取得", icon=ft.Icons.INFO_OUTLINE, on_click=on_fetch)
141     sid_field.on_submit = on_fetch
142
143     page.add(
144         ft.Text("e-Stat メタ情報ビューア", size=22, weight=ft.FontWeight.BOLD),
145         app_id_field,
146         ft.Row([sid_field, fetch_btn], vertical_alignment=ft.CrossAxisAlignment.END),
147         ft.Row([spinner, status]),
148         ft.Divider(),
149         title_text,
150         panels,
151     )
152
153
154 if __name__ == "__main__":
155     ft.run(main)

```

B.3 アプリ 3：統計データをランキング表示する

この例題のポイント

統計表 ID を入力して `getStatsData` を呼び、コードを名称に変換したうえで、最新の時点について値の大きい順に上位を横棒で表示する。棒グラフ専用のコントロールは使わず、最大値を基準にした `Container` の幅で棒を描いているため、依存は `flet・requests` だけで済む。表構造が表ごとに異なっても動くよう、「値・単位・時点」以外の次元名を連結してラベルにしている。

scripts/flet_app3_ranking.py

Flet で統計データをランキング表示する

```

1 # /// script
2 # requires-python = ">=3.11"
3 # dependencies = ["flet", "requests"]
4 # ///
5 """Flet サンプル3: e-Stat の統計データを取得してランキング表示する GUI アプリ。
6
7 appID は画面上的テキスト欄に入力する。統計表ID (statsDataId) を入れて

```

8 「データ取得」を押すと `getStatsData` を呼び、最新の時点について値の大きい順に
9 並べ、横棒（バー）と表で上位を表示する。

10
11 依存ライブラリは `flet` と `requests` だけ（`pandas/matplotlib` は使わず、棒は
12 `Container` の幅で描く）。統計表IDはサンプル1で検索、絞り込み用のコードは
13 サンプル2のメタ情報ビューアで調べられる。

14 実行:

```
15     uv run scripts/flet_app3_ranking.py
16     flet run scripts/flet_app3_ranking.py
17
18 """
19 import asyncio
20
21 import flet as ft
22 import requests
23
24 BASE = "https://api.e-stat.go.jp/rest/3.0/app/json"
25
26
27 def _as_list(x) -> list:
28     return x if isinstance(x, list) else [x]
29
30
31 def fetch_stats_data(app_id: str, sid: str, limit: int) -> list[dict]:
32     """getStatsData を呼び、コードを名称に直した tidy な行のリストを返す。
33
34     各行は {次元名: 名称, ..., '値': float, '単位': str, '時点': str} の辞書。
35     """
36     params = {"appId": app_id, "statsDataId": sid, "limit": limit}
37     res = requests.get(f"{BASE}/getStatsData", params=params, timeout=120)
38     res.raise_for_status()
39     body = res.json()["GET_STATS_DATA"]
40     if body["RESULT"]["STATUS"] != 0:
41         raise RuntimeError(body["RESULT"]["ERROR_MSG"])
42     sdata = body["STATISTICAL_DATA"]
43
44     # メタ (CLASS_OBJ) から「次元id → {コード: 名称}」と「次元id → 表示名」を作る
45     objs = _as_list(sdata["CLASS_INF"]["CLASS_OBJ"])
46     code2name = {o["@id"]: {c["@code"]: c["@name"]} for c in _as_list(o["CLASS"])} for o in objs}
47     id2label = {o["@id"]: o.get("@name", o["@id"]) for o in objs}
48
49     rows = []
50     for v in _as_list(sdata["DATA_INF"]["VALUE"]):
51         try:
52             value = float(v.get("$"))
53         except (TypeError, ValueError):
54             continue # 「-」など数値化できない値は除外
55         row = {"値": value, "単位": v.get("@unit", ""), "時点": ""}
56         for key, code in v.items():
57             if not key.startswith("@") or key == "@unit":
58                 continue
59             dim = key[1:] # 例: '@cat01' → 'cat01'
60             name = code2name.get(dim, {}).get(code, code)
```

```

61         row[id2label.get(dim, dim)] = name
62         if dim == "time":
63             row["時点"] = name
64         rows.append(row)
65     return rows
66
67
68 TOP_N = 15
69 BAR_FULL_WIDTH = 360 # 最大値のときのバーの幅(px)
70
71
72 async def main(page: ft.Page) -> None:
73     page.title = "e-Stat データ ランキング"
74     page.window.width = 880
75     page.window.height = 800
76     page.scroll = ft.ScrollMode.AUTO
77
78     app_id_field = ft.TextField(
79         label="アプリケーションID (appId) ",
80         password=True,
81         can_reveal_password=True,
82     )
83     sid_field = ft.TextField(label="統計表ID (statsDataId) ", hint_text="例: 0003448232", expand=
84         True)
85     limit_field = ft.TextField(label="取得件数", value="500", width=110)
86
87     status = ft.Text(color=ft.Colors.RED)
88     spinner = ft.ProgressRing(visible=False, width=20, height=20)
89     heading = ft.Text("", size=14, weight=ft.FontWeight.BOLD)
90     bars = ft.Column(spacing=6)
91
92     def label_for(row: dict) -> str:
93         """値・単位・時点以外の次元名を連結して 1 本のバーのラベルにする。"""
94         parts = [str(v) for k, v in row.items() if k not in ("値", "単位", "時点")]
95         return " / ".join(parts) if parts else "(系列)"
96
97     def render(rows: list[dict]) -> None:
98         bars.controls.clear()
99         if not rows:
100             heading.value = "数値データが見つかりませんでした。"
101             return
102
103         # 最新の時点だけに絞る (時点コードの最大=最も新しい)
104         latest = max((r["時点"] for r in rows), default="")
105         target = [r for r in rows if r["時点"] == latest] or rows
106         target = sorted(target, key=lambda r: r["値"], reverse=True)[:TOP_N]
107
108         unit = target[0]["単位"] or ""
109         heading.value = f"最新時点: {latest or '不明'} 上位 {len(target)} 件 (単位: {unit or
110             '-'}) "
111         vmax = max(r["値"] for r in target) or 1.0
112
113         for r in target:

```

```

112     w = max(2.0, BAR_FULL_WIDTH * r["値"] / vmax)
113     bars.controls.append(
114         ft.Row(
115             vertical_alignment=ft.CrossAxisAlignment.CENTER,
116             controls=[
117                 ft.Container(width=240, content=ft.Text(label_for(r), size=12, max_lines
118 =2)),
119                 ft.Container(
120                     width=w, height=20, bgcolor=ft.Colors.TEAL_400, border_radius=4
121 ),
122                 ft.Text(f"{r['値']:.0f}", size=12),
123             ],
124         )
125     )
126
127 async def on_fetch(e: ft.ControlEvent) -> None:
128     app_id = app_id_field.value.strip()
129     sid = sid_field.value.strip()
130     if not app_id or not sid:
131         status.value = "appID と統計表ID の両方を入力してください。"
132         status.color = ft.Colors.RED
133         return
134     try:
135         limit = max(1, min(100_000, int(limit_field.value or "500")))
136     except ValueError:
137         status.value = "取得件数は整数で入力してください。"
138         status.color = ft.Colors.RED
139         return
140
141     status.value = ""
142     heading.value = ""
143     bars.controls.clear()
144     spinner.visible = True
145     fetch_btn.disabled = True
146     page.update()
147
148     try:
149         rows = await asyncio.to_thread(fetch_stats_data, app_id, sid, limit)
150     except requests.HTTPError as ex:
151         status.value = f"通信エラー: {ex}"
152         status.color = ft.Colors.RED
153     except Exception as ex:
154         status.value = f"エラー: {ex}"
155         status.color = ft.Colors.RED
156     else:
157         status.value = f"{len(rows)} 件の数値データを取得しました。"
158         status.color = ft.Colors.GREEN
159         render(rows)
160     finally:
161         spinner.visible = False
162         fetch_btn.disabled = False
163         page.update()

```

```

164 fetch_btn = ft.Button("データ取得", icon=ft.Icons.BAR_CHART, on_click=on_fetch)
165 sid_field.on_submit = on_fetch
166
167 page.add(
168     ft.Text("e-Stat データ ランキング", size=22, weight=ft.FontWeight.BOLD),
169     app_id_field,
170     ft.Row([sid_field, limit_field, fetch_btn], vertical_alignment=ft.CrossAxisAlignment.END),
171     ft.Row([spinner, status]),
172     ft.Divider(),
173     heading,
174     bars,
175 )
176
177
178 if __name__ == "__main__":
179     ft.run(main)

```

さらに発展させるには

Flet には LineChart・BarChart などの本格的なグラフ部品や、Dropdown（アプリ 2 で調べたコードを選ばせる）、Tabs（複数表示の切り替え）、ファイル保存ダイアログなどもある。アプリ 2 で得た分類コードをアプリ 3 の絞り込み（cdTab 等）に渡せるよう連携させれば、第 2 部の各分析をそのまま対話的なダッシュボードに仕立てられる。

C FastAPI で作る Web アプリ

前付録の Flet がデスクトップ GUI だったのに対し、本付録では同じ e-Stat API を Web アプリから呼び出す例を 3 つ示す。サーバ側は Python の高速な Web フレームワーク **FastAPI**²⁾、画面の更新には **HTMX**³⁾ を用いる。ブラウザさえあれば動くため、配布や共同利用がしやすい。

この例題のポイント

- 各アプリは単一ファイルの PEP 723 スクリプト。依存は fastapi・uvicorn・jinja2・requests・python-multipart（アプリ 3 のみ pandas 追加）。画面の HTML は Jinja2 のテンプレート文字列としてソース内に持つ。
- サーバは HTML の断片を返し、HTMX が hx-post・hx-get で指定先（hx-target）に差し込む。ページ全体を再読み込みしない。
- appId は画面のフォーム欄から受け取り、各リクエストに添えて送る。環境変数 ESTAT_APPID があれば初期値に補完するが、ソースには残さない。
- レスポンスの RESULT.STATUS 確認や「1 件のとき配列にならない」対策（_as_list）は、

2) <https://fastapi.tiangolo.com/>

3) <https://htmx.org/>（CDN から読み込むため追加インストール不要）。HTML の属性だけで部分更新を実現し、JavaScript をほとんど書かずに対話的な画面を作る。

第 1 部までと同じ考え方で行う。

実行方法 いずれも PEP 723 形式なので `uv run` で必要なパッケージが自動的に用意される。起動後、表示される URL（既定 <http://127.0.0.1:8000>）をブラウザで開く。

```
uv run scripts/fastapi_app1_search.py    # 起動 -> ブラウザで 127.0.0.1:8000 を開く
```

appId は <https://www.e-stat.go.jp/api/> で無料登録すると取得できる。アプリ起動後、画面上部の「アプリケーション ID」欄に貼り付けてから操作する。

3つのアプリのつながり

3つは「①表を探す (`getStatsList`) → ②絞り込みコードを調べる (`getMetaInfo`) → ③データを取って可視化する (`getStatsData`)」という e-Stat 利用の典型的な流れに対応している。アプリ 1 で見つけた統計表 ID (`statsDataId`) をアプリ 2 に、アプリ 2 で調べた分類コードをアプリ 3 の絞り込み (`cdTab` 等) に渡して使う。

C.1 アプリ 1：統計表をキーワード検索する

この例題のポイント

最も基本的な FastAPI + HTMX アプリ。GET / がフォームを返し、検索ボタンの `hx-post="/search"` が `getStatsList` を呼んでヒット一覧の HTML 断片を `#results` に差し込む。FastAPI の Form 受け取り、HTMLResponse、HTMX の `hx-target` と `hx-indicator`（読み込み中表示）を学ぶ。

scripts/fastapi_app1_search.py

FastAPI で統計表を検索する

```
1 # /// script
2 # requires-python = ">=3.11"
3 # dependencies = ["fastapi", "uvicorn", "jinja2", "requests", "python-multipart"]
4 # ///
5 """FastAPI サンプル1: e-Stat 統計表をキーワード検索する Web アプリ (HTMX)。
6
7 appId は画面のフォーム欄に入力する（ソースに残さない）。検索すると getStatsList を
8 呼び、ヒットした統計表をカード一覧の HTML 断片で返し、HTMX が画面に差し込む。
9
10 実行:
11     uv run scripts/fastapi_app1_search.py
12     # 起動後ブラウザで http://127.0.0.1:8000 を開く
13 """
14 import os
15
16 import requests
17 import uvicorn
18 from fastapi import FastAPI, Form
19 from fastapi.responses import HTMLResponse
20 from jinja2 import Template
```

```

21
22 BASE = "https://api.e-stat.go.jp/rest/3.0/app/json"
23 DEFAULT_APPID = os.environ.get("ESTAT_APPID", "")
24
25 app = FastAPI(title="e-Stat 統計表検索")
26
27
28 def _text(x) -> str:
29     """{'@..':..., '$': '本文'} 形式のことがある値を文字列に正規化する。"""
30     return x["$"] if isinstance(x, dict) else (" " if x is None else str(x))
31
32
33 def _as_list(x) -> list:
34     """要素1件だと配列にならない API 仕様への対策。"""
35     return x if isinstance(x, list) else [x]
36
37
38 def search_stats_list(app_id: str, word: str, limit: int) -> tuple[str, list[dict]]:
39     """getStatsList を呼ぶ。返り値は (ヒット総数の文字列, 統計表のリスト)。"""
40     params = {"appId": app_id, "searchWord": word, "limit": limit}
41     res = requests.get(f"{BASE}/getStatsList", params=params, timeout=60)
42     res.raise_for_status()
43     body = res.json()["GET_STATS_LIST"]
44     if body["RESULT"]["STATUS"] != 0:
45         raise RuntimeError(body["RESULT"]["ERROR_MSG"])
46     info = body["DATALIST_INF"]
47     return str(info.get("NUMBER", "不明")), _as_list(info.get("TABLE_INF", []))
48
49
50 PAGE = Template(
51     """<!doctype html>
52     <html lang="ja"><head><meta charset="utf-8">
53     <meta name="viewport" content="width=device-width, initial-scale=1">
54     <title>e-Stat 統計表検索</title>
55     <script src="https://unpkg.com/htmx.org@2.0.4"></script>
56     <style>
57     body{font-family:system-ui,sans-serif;max-width:820px;margin:1.5rem auto;padding:0 1rem;color
58         :#222}
59     h1{font-size:1.4rem} label{font-size:.85rem;color:#555;display:block;margin:.4rem 0 .1rem}
60     input{padding:.45rem;border:1px solid #ccc;border-radius:6px;font-size:1rem}
61     .row{display:flex;gap:.6rem;align-items:flex-end;flex-wrap:wrap}
62     button{padding:.5rem 1rem;border:0;border-radius:6px;background:#005aa0;color:#fff;font-size:1rem;
63         cursor:pointer}
64     .card{border:1px solid #e0e0e0;border-radius:8px;padding:.7rem .9rem;margin:.5rem 0}
65     .card .t{font-weight:700} .card .s{color:#5a6b7b;font-size:.85rem}
66     .card .i{font-size:.8rem;color:#333;user-select:all} .err{color:#c0392b;font-weight:700}
67     .htmx-indicator{display:none} .htmx-request .htmx-indicator{display:inline}
68     </style></head>
69     <body>
70     <h1>e-Stat 統計表検索</h1>
71     <form hx-post="/search" hx-target="#results" hx-indicator="#spin">
72     <label>アプリケーションID (appId) </label>
73     <input type="password" name="appid" value="{{ default_appid }}" style="width:100%"

```

```

72     placeholder="e-Stat で取得した API キーを貼り付け">
73     <div class="row">
74         <div style="flex:1"><label>検索キーワード</label>
75         <input name="word" value="人口" style="width:100%"></div>
76         <div><label>件数</label><input name="limit" value="20" style="width:80px"></div>
77         <button type="submit">検索 <span id="spin" class="htmx-indicator">…</span></button>
78     </div>
79 </form>
80 <div id="results"></div>
81 </body></html>""
82 )
83
84 RESULTS = Template(
85     ""<p>ヒット総数 {{ total }} 件 (先頭 {{ tables|length }} 件を表示) </p>
86     {% for t in tables %}
87     <div class="card">
88         <div class="t">{{ t.title or "(表題なし)" }}</div>
89         <div class="s">{{ t.stat_name }}</div>
90         <div class="i">統計表ID: {{ t.id }} 調査年月: {{ t.survey or "-" }}</div>
91     </div>
92     {% endfor %}""
93 )
94
95
96 @app.get("/", response_class=HTMLResponse)
97 def index() -> str:
98     return PAGE.render(default_appid=DEFAULT_APPID)
99
100
101 @app.post("/search", response_class=HTMLResponse)
102 def search(appid: str = Form(""), word: str = Form(""), limit: str = Form("20")) -> str:
103     appid, word = appid.strip(), word.strip()
104     if not appid:
105         return '<p class="err">appId を入力してください。</p>'
106     if not word:
107         return '<p class="err">キーワードを入力してください。</p>'
108     try:
109         n = max(1, min(100, int(limit or "20")))
110     except ValueError:
111         return '<p class="err">件数は整数で入力してください。</p>'
112     try:
113         total, raw = search_stats_list(appid, word, n)
114     except requests.HTTPError as ex:
115         return f'<p class="err">通信エラー: {ex}</p>'
116     except Exception as ex: # API エラー含む
117         return f'<p class="err">エラー: {ex}</p>'
118     tables = [
119         {"id": t.get("@id", ""), "title": _text(t.get("TITLE")),
120          "stat_name": _text(t.get("STAT_NAME")), "survey": _text(t.get("SURVEY_DATE"))}
121         for t in raw
122     ]
123     return RESULTS.render(total=total, tables=tables)
124

```

```

125
126 if __name__ == "__main__":
127     uvicorn.run(app, host="127.0.0.1", port=8000)

```

C.2 アプリ 2：メタ情報（分類の定義）をブラウズする

この例題のポイント

統計表 ID を入力して `getMetaInfo` を呼び、分類事項（`CLASS_OBJ`）ごとの見出しを返す。各分類の「コード一覧」ボタンは `hx-get="/meta/class"` で、押したときにそのコード表だけを遅延読み込みする（HTMX の遅延ロード）。アプリ 3 で指定する `cdTab・cdCat01・cdArea` などのコード調べに役立つ。

scripts/fastapi_app2_metadata.py

FastAPI でメタ情報をブラウズする

```

1 # /// script
2 # requires-python = ">=3.11"
3 # dependencies = ["fastapi", "uvicorn", "jinja2", "requests", "python-multipart"]
4 # ///
5 """FastAPI サンプル2: 統計表のメタ情報（分類の定義）を表示する Web アプリ（HTMX）。
6
7 統計表IDを入れて getMetaInfo を呼び、分類事項（CLASS_OBJ）ごとの見出しを出す。
8 各分類の「コード一覧」ボタンを押すと HTMX が /meta/class を呼び、その分類の
9 「コード⇔名称⇔単位」表を遅延ロードして差し込む（アプリ3の絞り込みコード調べに使う）。
10
11 実行:
12     uv run scripts/fastapi_app2_metadata.py
13     # 起動後ブラウザで http://127.0.0.1:8000 を開く
14 """
15 import os
16
17 import requests
18 import uvicorn
19 from fastapi import FastAPI, Form
20 from fastapi.responses import HTMLResponse
21 from jinja2 import Template
22
23 BASE = "https://api.e-stat.go.jp/rest/3.0/app/json"
24 DEFAULT_APPID = os.environ.get("ESTAT_APPID", "")
25 MAX_CODES = 200 # 1分類あたりの表示上限（巨大分類で画面が重くならないように）
26
27 app = FastAPI(title="e-Stat メタ情報ブラウザ")
28
29
30 def _text(x) -> str:
31     return x["$"] if isinstance(x, dict) else ("" if x is None else str(x))
32
33
34 def _as_list(x) -> list:
35     return x if isinstance(x, list) else [x]
36

```

```

37
38 def get_meta_info(app_id: str, sid: str) -> tuple[str, list[dict]]:
39     """getMetaInfo を呼ぶ。 返り値は (表題, CLASS_OBJ のリスト)。"""
40     res = requests.get(f"{BASE}/getMetaInfo",
41                        params={"appId": app_id, "statsDataId": sid}, timeout=60)
42     res.raise_for_status()
43     body = res.json()["GET_META_INFO"]
44     if body["RESULT"]["STATUS"] != 0:
45         raise RuntimeError(body["RESULT"]["ERROR_MSG"])
46     info = body["METADATA_INF"]
47     title = _text(info["TABLE_INF"].get("TITLE"))
48     return title, _as_list(info["CLASS_INF"]["CLASS_OBJ"])
49
50
51 PAGE = Template(
52     """<!doctype html>
53 <html lang="ja"><head><meta charset="utf-8">
54 <meta name="viewport" content="width=device-width, initial-scale=1">
55 <title>e-Stat メタ情報ブラウザ</title>
56 <script src="https://unpkg.com/htmx.org@2.0.4"></script>
57 <style>
58   body{font-family:system-ui,sans-serif;max-width:860px;margin:1.5rem auto;padding:0 1rem;color
59     :#222}
60   h1{font-size:1.4rem} label{font-size:.85rem;color:#555;display:block;margin:.4rem 0 .1rem}
61   input{padding:.45rem;border:1px solid #ccc;border-radius:6px;font-size:1rem}
62   button{padding:.45rem .9rem;border:0;border-radius:6px;background:#005aa0;color:#fff;cursor:
63     pointer}
64   .cls{border:1px solid #e0e0e0;border-radius:8px;padding:.6rem .9rem;margin:.5rem 0}
65   .cls h3{margin:.1rem 0;font-size:1rem} .cls .id{color:#5a6b7b;font-size:.8rem}
66   table{border-collapse:collapse;width:100%;margin-top:.4rem;font-size:.85rem}
67   th,td{border:1px solid #ddd;padding:.25rem .5rem;text-align:left}
68   th{background:#f3f6f9} .err{color:#c0392b;font-weight:700}
69   .htmx-indicator{display:none} .htmx-request .htmx-indicator{display:inline}
70 </style></head>
71 <body>
72   <h1>e-Stat メタ情報ブラウザ</h1>
73   <form hx-post="/meta" hx-target="#meta" hx-indicator="#spin">
74     <label>アプリケーションID (appId) </label>
75     <input type="password" name="appid" value="{ default_appid }" style="width:100%"
76       placeholder="e-Stat で取得した API キーを貼り付け">
77     <label>統計表ID (statsDataId) </label>
78     <input name="sid" placeholder="例: 0003443840" style="width:100%">
79     <p><button type="submit">メタ情報を取得 <span id="spin" class="htmx-indicator">…</span></button>
80       </p>
81   </form>
82   <div id="meta"></div>
83 </body></html>"""
84 )
85
86 CLASSES = Template(
87     """<h2 style="font-size:1.1rem">{{ title or "(表題なし)" }}</h2>
88     {% for o in objs %}
89     <div class="cls">

```

```

87 <h3>{{ o.name }} <span class="id"> (id={{ o.id }}, {{ o.n }}件) </span></h3>
88 <button hx-get="/meta/class?appid={{ appid_q }}&sid={{ sid_q }}&id={{ o.id }}"
89       hx-target="#cls-{{ o.id }}" hx-swap="innerHTML">コード一覧</button>
90 <div id="cls-{{ o.id }}"></div>
91 </div>
92 {% endfor %}"""
93 )
94
95 CODES = Template(
96     """<table><tr><th>コード</th><th>名称</th><th>レベル</th><th>単位</th></tr>
97     {% for c in codes %}
98     <tr><td>{{ c.code }}</td><td>{{ c.name }}</td><td>{{ c.level }}</td><td>{{ c.unit }}</td></tr>
99     {% endfor %}
100 </table>{% if capped %}<p class="id">先頭 {{ codes|length }} 件のみ表示</p>{% endif %}"""
101 )
102
103
104 @app.get("/", response_class=HTMLResponse)
105 def index() -> str:
106     return PAGE.render(default_appid=DEFAULT_APPID)
107
108
109 @app.post("/meta", response_class=HTMLResponse)
110 def meta(appid: str = Form(""), sid: str = Form("")) -> str:
111     appid, sid = appid.strip(), sid.strip()
112     if not appid or not sid:
113         return '<p class="err">appId と統計表IDを入力してください。</p>'
114     try:
115         title, objs = get_meta_info(appid, sid)
116     except Exception as ex:
117         return f'<p class="err">エラー: {ex}</p>'
118     view = [{"id": o.get("@id", ""), "name": o.get("@name", ""),
119             "n": len(_as_list(o.get("CLASS", [])))} for o in objs]
120     return CLASSES.render(title=title, objs=view, appid_q=appid, sid_q=sid)
121
122
123 @app.get("/meta/class", response_class=HTMLResponse)
124 def meta_class(appid: str, sid: str, id: str) -> str:
125     try:
126         _, objs = get_meta_info(appid, sid)
127     except Exception as ex:
128         return f'<p class="err">エラー: {ex}</p>'
129     target = next((o for o in objs if o.get("@id") == id), None)
130     if target is None:
131         return '<p class="err">分類が見つかりません。</p>'
132     cls = _as_list(target.get("CLASS", []))
133     capped = len(cls) > MAX_CODES
134     codes = [{"code": c.get("@code", ""), "name": c.get("@name", ""),
135             "level": c.get("@level", "-"), "unit": c.get("@unit", "")}
136             for c in cls[:MAX_CODES]]
137     return CODES.render(codes=codes, capped=capped)
138
139

```

```

143 if __name__ == "__main__":
144     uvicorn.run(app, host="127.0.0.1", port=8000)

```

C.3 アプリ 3：統計データのダッシュボード

この例題のポイント

3 つの API を結びつける**本格アプリ**。統計表 ID と絞り込みコードを入力して `getStatsData` (NEXT_KEY ページング対応) を呼び、コードを名称に変換して `pandas` で整形。最新時点の上位を表と棒グラフ (`Chart.js`) で表示し、整形済みデータを **CSV ダウンロード** できる。グラフは差し替え後に発火する `htmx:afterSwap` で、断片に埋め込んだ JSON (`#chart-data`) を読んで描画する。

scripts/fastapi_app3_dashboard.py

FastAPI で統計データのダッシュボードを作る

```

1  # /// script
2  # requires-python = ">=3.11"
3  # dependencies = ["fastapi", "uvicorn", "jinja2", "requests", "python-multipart", "pandas"]
4  # ///
5  """FastAPI サンプル3: e-Stat 統計データのダッシュボード (HTMX + Chart.js + CSV)。
6
7  統計表IDと絞り込みコード (cdTab など) を入れて getStatsData を呼び、コードを名称に
8  変換して pandas で整形。最新時点の上位を「表」と「棒グラフ (Chart.js)」で表示し、
9  整形済みデータを CSV でダウンロードできる。getStatsList→getMetaInfo→getStatsData の
10  締めくくりとなる “本格アプリ”。アプリ1で見つけた統計表ID、アプリ2で調べたコードを使う。
11
12  実行:
13      uv run scripts/fastapi_app3_dashboard.py
14      # 起動後ブラウザで http://127.0.0.1:8000 を開く
15  """
16  import io
17  import json
18  import os
19  from urllib.parse import urlencode
20
21  import pandas as pd
22  import requests
23  import uvicorn
24  from fastapi import FastAPI, Form
25  from fastapi.responses import HTMLResponse, Response
26  from jinja2 import Template
27
28  BASE = "https://api.e-stat.go.jp/rest/3.0/app/json"
29  DEFAULT_APPID = os.environ.get("ESTAT_APPID", "")
30  MAX_PAGES = 10 # NEXT_KEY を辿る最大ページ数 (暴走防止)
31  TOP_N = 20     # グラフ・表に出す上位件数
32
33  app = FastAPI(title="e-Stat データ・ダッシュボード")
34
35

```

```

36 def _text(x) -> str:
37     return x["$"] if isinstance(x, dict) else (" if x is None else str(x))
38
39
40 def _as_list(x) -> list:
41     return x if isinstance(x, list) else [x]
42
43
44 def get_stats_data(app_id: str, sid: str, **filters) -> dict:
45     """getStatsData を呼び、NEXT_KEY を辿って (最大 MAX_PAGES) 全件結合する。"""
46     base = {"appId": app_id, "statsDataId": sid,
47             **{k: v for k, v in filters.items() if v}}
48
49     def fetch(params: dict) -> dict:
50         res = requests.get(f"{BASE}/getStatsData", params=params, timeout=120)
51         res.raise_for_status()
52         body = res.json()["GET_STATS_DATA"]
53         if body["RESULT"]["STATUS"] != 0:
54             raise RuntimeError(body["RESULT"]["ERROR_MSG"])
55         return body["STATISTICAL_DATA"]
56
57     sdata = fetch(base)
58     values = _as_list(sdata["DATA_INF"]["VALUE"])
59     next_key = sdata["RESULT_INF"].get("NEXT_KEY")
60     pages = 1
61     while next_key and pages < MAX_PAGES:
62         sd2 = fetch(**base, "startPosition": next_key)
63         values += _as_list(sd2["DATA_INF"]["VALUE"])
64         next_key = sd2["RESULT_INF"].get("NEXT_KEY")
65         pages += 1
66     sdata["DATA_INF"]["VALUE"] = values
67     return sdata
68
69
70 def to_table(sdata: dict) -> pd.DataFrame:
71     """STATISTICAL_DATA を tidy な DataFrame に。各次元名・<id>_code・値・単位。"""
72     objs = _as_list(sdata["CLASS_INF"]["CLASS_OBJ"])
73     code2name = {o["@id"]: {c["@code"]: c["@name"] for c in _as_list(o["CLASS"])}}
74     for o in objs:
75         raw = pd.DataFrame(_as_list(sdata["DATA_INF"]["VALUE"]))
76         out: dict[str, object] = {}
77         for col in raw.columns:
78             if col == "$":
79                 out["値"] = pd.to_numeric(raw[col], errors="coerce")
80             elif col == "@unit":
81                 out["単位"] = raw[col]
82             elif col.startswith("@"):
83                 key = col[1:]
84                 out[f"{key}_code"] = raw[col]
85                 if key in code2name:
86                     out[key] = raw[col].map(code2name[key])
87     return pd.DataFrame(out)
88

```

```

89
90 def build_series(df: pd.DataFrame) -> dict:
91     """最新時点の上位 TOP_N をグラフ用 {labels, values, unit, period} にする。"""
92     df = df.dropna(subset=["値"]).copy()
93     period = ""
94     if "time_code" in df.columns:
95         latest = df["time_code"].max()
96         df = df[df["time_code"] == latest]
97         period = str(df["time"].iloc[0]) if "time" in df.columns and len(df) else str(latest)
98     name_cols = [c for c in df.columns
99                  if c not in ("値", "単位", "time") and not c.endswith("_code")]
100     if name_cols:
101         labels = df[name_cols].astype(str).agg(" / ".join, axis=1)
102     else:
103         labels = df.index.astype(str)
104     df = df.assign(__label=labels).sort_values("値", ascending=False).head(TOP_N)
105     unit = str(df["単位"].iloc[0]) if "単位" in df.columns and len(df) else ""
106     return {"labels": df["__label"].tolist(),
107            "values": [float(v) for v in df["値"].tolist()],
108            "unit": unit, "period": period}
109
110
111 PAGE = Template(
112     """<!doctype html>
113 <html lang="ja"><head><meta charset="utf-8">
114 <meta name="viewport" content="width=device-width, initial-scale=1">
115 <title>e-Stat データ・ダッシュボード</title>
116 <script src="https://unpkg.com/htmx.org@2.0.4"></script>
117 <script src="https://cdn.jsdelivr.net/npm/chart.js@4.4.1/dist/chart.umd.min.js"></script>
118 <style>
119   body{font-family:system-ui,sans-serif;max-width:920px;margin:1.5rem auto;padding:0 1rem;color
120     :#222}
121   h1{font-size:1.4rem} label{font-size:.85rem;color:#555;display:block;margin:.4rem 0 .1rem}
122   input{padding:.45rem;border:1px solid #ccc;border-radius:6px;font-size:1rem}
123   .row{display:flex;gap:.6rem;flex-wrap:wrap} .row>div{flex:1;min-width:120px}
124   button{padding:.5rem 1rem;border:0;border-radius:6px;background:#005aa0;color:#fff;cursor:pointer}
125   table{border-collapse:collapse;width:100%;margin-top:.6rem;font-size:.85rem}
126   th,td{border:1px solid #ddd;padding:.25rem .5rem} th{background:#f3f6f9}
127   td.num{text-align:right;font-variant-numeric:tabular-nums} .err{color:#c0392b;font-weight:700}
128   .htmx-indicator{display:none} .htmx-request .htmx-indicator{display:inline}
129   a.csv{display:inline-block;margin:.5rem 0;color:#005aa0}
130 </style></head>
131 <body>
132 <h1>e-Stat データ・ダッシュボード</h1>
133 <form hx-post="/dashboard" hx-target="#panel" hx-indicator="#spin">
134   <label>アプリケーションID (appId) </label>
135   <input type="password" name="appid" value="{ default_appid }" style="width:100%"
136     placeholder="e-Stat で取得した API キーを貼り付け">
137   <label>統計表ID (statsDataId) </label>
138   <input name="sid" placeholder="例: 0003443840" style="width:100%">
139   <div class="row">
140     <div><label>cdTab</label><input name="cdTab" placeholder="表章(任意)"></div>
141     <div><label>cdCat01</label><input name="cdCat01" placeholder="分類1(任意)"></div>

```

```

141 <div><label>cdArea</label><input name="cdArea" placeholder="地域(任意)"></div>
142 <div><label>cdTime</label><input name="cdTime" placeholder="時間(任意)"></div>
143 </div>
144 <p><button type="submit">表示 <span id="spin" class="htmx-indicator">…</span></button></p>
145 </form>
146 <canvas id="chart" height="120"></canvas>
147 <div id="panel"></div>
148 <script>
149 let _chart = null;
150 function renderChart(){
151     const el = document.getElementById("chart-data");
152     if(!el) return;
153     const d = JSON.parse(el.textContent);
154     const ctx = document.getElementById("chart");
155     if(_chart) _chart.destroy();
156     _chart = new Chart(ctx, {
157         type: "bar",
158         data: { labels: d.labels,
159                 datasets: [{ label: (d.unit || "値") + " (" + d.period + ")",
160                             data: d.values, backgroundColor: "#4C72B0" }] },
161         options: { indexAxis: "y", responsive: true,
162                   plugins: { legend: { display: true } } }
163     });
164 }
165 document.body.addEventListener("htmx:afterSwap", function(e){
166     if(e.detail.target.id === "panel") renderChart();
167 });
168 </script>
169 </body></html>""
170 )
171
172 PANEL = Template(
173     ""<p>{{ title }} — {{ period }} 時点の上位 {{ rows|length }} 件</p>
174     <a class="csv" href="{{ csv_url }}">CSV ダウンロード</a>
175     <table><tr><th>項目</th><th>値</th><th>単位</th></tr>
176     {% for r in rows %}
177     <tr><td>{{ r.label }}</td><td class="num">{{ "{:,.0f}".format(r.value) }}</td><td>{{ unit }}</td>
178     </tr>
179     {% endfor %}
180     </table>
181     <script type="application/json" id="chart-data">{{ chart_json }}</script>""
182 )
183
184 @app.get("/", response_class=HTMLResponse)
185 def index() -> str:
186     return PAGE.render(default_appid=DEFAULT_APPID)
187
188
189 @app.post("/dashboard", response_class=HTMLResponse)
190 def dashboard(appid: str = Form(""), sid: str = Form(""),
191              cdTab: str = Form(""), cdCat01: str = Form(""),
192              cdArea: str = Form(""), cdTime: str = Form("")) -> str:

```

```

193 appid, sid = appid.strip(), sid.strip()
194 if not appid or not sid:
195     return '<p class="err">appId と統計表IDを入力してください。</p>'
196 filters = {"cdTab": cdTab, "cdCat01": cdCat01, "cdArea": cdArea, "cdTime": cdTime}
197 try:
198     sdata = get_stats_data(appid, sid, **filters)
199     df = to_table(sdata)
200     series = build_series(df)
201 except Exception as ex:
202     return f'<p class="err">エラー: {ex}</p>'
203 title = _text(sdata.get("TABLE_INF", {}).get("TITLE")) or sid
204 rows = [{"label": l, "value": v} for l, v in zip(series["labels"], series["values"])]
205 csv_url = "/download.csv?" + urlencode({"appid": appid, "sid": sid, **filters})
206 return PANEL.render(title=title, period=series["period"], unit=series["unit"],
207                     rows=rows, chart_json=json.dumps(series, ensure_ascii=False),
208                     csv_url=csv_url)
209
210
211 @app.get("/download.csv")
212 def download_csv(appid: str, sid: str, cdTab: str = "", cdCat01: str = "",
213                 cdArea: str = "", cdTime: str = "") -> Response:
214     filters = {"cdTab": cdTab, "cdCat01": cdCat01, "cdArea": cdArea, "cdTime": cdTime}
215     try:
216         sdata = get_stats_data(appid, sid, **filters)
217         df = to_table(sdata)
218     except Exception as ex:
219         return Response(f"error,{ex}\n", media_type="text/csv", status_code=502)
220     buf = io.StringIO()
221     df.to_csv(buf, index=False)
222     return Response(
223         buf.getvalue().encode("utf-8-sig"), # Excel 文字化け対策で BOM 付き
224         media_type="text/csv",
225         headers={"Content-Disposition": f'attachment; filename="{sid}.csv"'},
226     )
227
228
229 if __name__ == "__main__":
230     uvicorn.run(app, host="127.0.0.1", port=8000)

```

さらに発展させるには

ここから、複数系列の折れ線比較、地域メッシュ地図、分類コードを getMetaInfo から取得してプルダウンで選ばせる UI、結果のキャッシュ、ユーザごとの appId をセッションに保持する仕組み、コンテナ化してクラウドへデプロイ、などへ発展できる。第 2 部の各分析を、そのまま対話的な Web ダッシュボードに仕立てられる。